

## config/\_\_init\_\_.py

---

```
"""Configuration module for the simulator."""  
  
from .settings import Settings, SimulationMode  
  
__all__ = ['Settings', 'SimulationMode']
```

# config/settings.py

---

```
"""Settings and configuration for the simulator."""

from dataclasses import dataclass
from enum import Enum

class SimulationMode(Enum):
    """Simulation execution modes."""

    REAL_TIME = "real_time"
    BATCH = "batch"

@dataclass
class Settings:
    """
    Global settings for the port simulator.

    Attributes:
        timestep: Simulation timestep in seconds
        mode: Simulation mode (real-time or batch)
        db_path: Path to SQLite database file
        use_optimizer: Whether to use optimization for scheduling
        power_limit_mode: Whether to enforce contracted power limit without optimization (baseline mode)
        trip_schedule: Trip departure times as (hour_utc, slot_index) per day, e.g. ((9, 0), (14, 1))
    """

    timestep: int = 900 # Default: 15 minutes
    mode: SimulationMode = SimulationMode.BATCH
    db_path: str = "port_simulation.db"
    use_optimizer: bool = False # Default: use rule-based control
    power_limit_mode: bool = False # Default: no power limiting (unlimited charging)
    # Trip schedule: list of (hour_utc, slot_index), e.g. 9:00 slot 0, 14:00 slot 1
    trip_schedule: tuple = ((9, 0), (14, 1))

    def __post_init__(self):
        """Validate settings."""
        if self.timestep <= 0:
            raise ValueError("Timestep must be positive")
```

## database/\_\_init\_\_.py

---

```
"""Database management for the simulator."""  
  
from .db_manager import DatabaseManager  
  
__all__ = ['DatabaseManager']
```

# database/db\_manager.py

---

```
"""Database manager for storing simulation data."""

import sqlite3
from typing import Optional, List, Tuple
from contextlib import contextmanager

# Valid table names for data records
VALID_TABLES = ("measurements", "forecast", "scheduling")

# Default metrics to initialize with the database
# Format: (metric_name, unit, data_type)
DEFAULT_METRICS = (
    # Power metrics
    ("power_active_consumption", "kW", "float"),
    ("power_active_production", "kW", "float"),
    ("power_active_import", "kW", "float"),
    ("power_active_export", "kW", "float"),
    ("available_power", "kW", "float"),
    ("contracted_power", "kW", "float"),
    ("power_active", "kW", "float"),
    ("power_setpoint", "kW", "float"),
    # Battery/BESS metrics
    ("soc", "%", "float"),
    ("state", "", "str"),
    ("bess_discharge", "kW", "float"),
    ("bess_charge", "kW", "float"),
    ("energy_stored", "kWh", "float"),
    ("bess_available", "kWh", "float"),
    ("bess_capacity", "kWh", "float"),
    # Boat forecast metrics
    ("boat_required_energy", "kWh", "float"),
    ("boat_available", "", "int"),
    # Weather metrics
    ("temperature", "°C", "float"),
    ("humidity", "%", "float"),
    ("dew_point", "°C", "float"),
    ("precipitation", "mm", "float"),
    ("weather_code", "", "int"),
    ("cloud_cover", "%", "float"),
    ("wind_speed", "m/s", "float"),
    ("wind_direction", "°", "float"),
    # Solar radiation metrics
    ("ghi", "W/m²", "float"),
    ("direct_radiation", "W/m²", "float"),
    ("dhi", "W/m²", "float"),
    ("dni", "W/m²", "float"),
    # Energy/production metrics
    ("net_balance", "kW", "float"),
)

)

class DatabaseManager:
    """Manages SQLite database operations for the simulator."""

    def __init__(self, db_path: str):
        """
        Initialize database manager.

        Args:
            db_path: Path to SQLite database file
        """
        self.db_path = db_path
```

```

        self._connection: Optional[sqlite3.Connection] = None
        # Cache for source and metric IDs to avoid repeated lookups
        self._source_cache: dict[str, int] = {}
        self._metric_cache: dict[str, int] = {}

    def connect(self):
        """Establish database connection."""
        self._connection = sqlite3.connect(self.db_path)
        self._connection.row_factory = sqlite3.Row

    def close(self):
        """Close database connection."""
        if self._connection:
            self._connection.close()
            self._connection = None

    @contextmanager
    def get_connection(self):
        """Context manager for database connection."""
        conn = sqlite3.connect(self.db_path)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        except Exception as e:
            conn.rollback()
            raise e
        finally:
            conn.close()

    def initialize_schema(self):
        """Create database tables for simulation data."""
        with self.get_connection() as conn:
            cursor = conn.cursor()

            # Source table - registry of data sources
            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS source (
                    source_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    source_name TEXT NOT NULL UNIQUE,
                    source_type TEXT NOT NULL
                )
            """
            )

            # Metric table - registry of metrics
            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS metric (
                    metric_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    metric_name TEXT NOT NULL UNIQUE,
                    unit TEXT NOT NULL,
                    data_type TEXT NOT NULL
                )
            """
            )

            # Measurements table - for actual measured/simulated data
            cursor.execute(
                """
                CREATE TABLE IF NOT EXISTS measurements (
                    measurement_id INTEGER PRIMARY KEY AUTOINCREMENT,
                    timestamp TEXT NOT NULL,
                    source_id INTEGER NOT NULL,
                    metric_id INTEGER NOT NULL,
                    value TEXT NOT NULL,

```

```

        FOREIGN KEY (source_id) REFERENCES source(source_id),
        FOREIGN KEY (metric_id) REFERENCES metric(metric_id)
    )
    """
)

# Create indexes for measurements
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_measurements_timestamp
    ON measurements(timestamp)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_measurements_source_id
    ON measurements(source_id)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_measurements_metric_id
    ON measurements(metric_id)
    """
)

# Forecast table - for predicted/forecasted data
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS forecast (
        measurement_id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp TEXT NOT NULL,
        source_id INTEGER NOT NULL,
        metric_id INTEGER NOT NULL,
        value TEXT NOT NULL,
        FOREIGN KEY (source_id) REFERENCES source(source_id),
        FOREIGN KEY (metric_id) REFERENCES metric(metric_id)
    )
    """
)

# Create indexes for forecast
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_forecast_timestamp
    ON forecast(timestamp)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_forecast_source_id
    ON forecast(source_id)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_forecast_metric_id
    ON forecast(metric_id)
    """
)

# Scheduling table - for scheduled events/commands
cursor.execute(
    """
    CREATE TABLE IF NOT EXISTS scheduling (
        measurement_id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp TEXT NOT NULL,
        source_id INTEGER NOT NULL,

```

```

        metric_id INTEGER NOT NULL,
        value TEXT NOT NULL,
        FOREIGN KEY (source_id) REFERENCES source(source_id),
        FOREIGN KEY (metric_id) REFERENCES metric(metric_id)
    )
    """
)

# Create indexes for scheduling
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_scheduling_timestamp
    ON scheduling(timestamp)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_scheduling_source_id
    ON scheduling(source_id)
    """
)
cursor.execute(
    """
    CREATE INDEX IF NOT EXISTS idx_scheduling_metric_id
    ON scheduling(metric_id)
    """
)

conn.commit()

def initialize_default_metrics(self):
    """
    Initialize the database with default metrics.

    Inserts all metrics from DEFAULT_METRICS if they don't already exist.
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        for metric_name, unit, data_type in DEFAULT_METRICS:
            cursor.execute(
                """
                INSERT OR IGNORE INTO metric (metric_name, unit, data_type)
                VALUES (?, ?, ?)
                """
            )
            cursor.execute(
                """
                INSERT OR IGNORE INTO scheduling (source_id, metric_id, timestamp, value)
                VALUES (?, ?, ?, ?)
                """
            )
        conn.commit()

def _validate_table(self, table: str):
    """Validate that the table name is allowed."""
    if table not in VALID_TABLES:
        raise ValueError(f"Invalid table '{table}'. Must be one of: {VALID_TABLES}")

# Unified data record methods (measurements, forecast, scheduling)
def save_record(
    self, table: str, timestamp: str, source_id: int, metric_id: int, value: str
) -> int:
    """
    Save a single record to the specified table.

    Args:
        table: Table name ('measurements', 'forecast', or 'scheduling')
        timestamp: Timestamp (ISO format UTC string)
        source_id: Foreign key to source table
        metric_id: Foreign key to metric table
        value: Value as string

    Returns:
    """

```

```

        The measurement_id of the inserted record
    """
    self._validate_table(table)
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            f"""
                INSERT INTO {table} (timestamp, source_id, metric_id, value)
                VALUES (?, ?, ?, ?)
            """,
            (timestamp, source_id, metric_id, value),
        )
        return cursor.lastrowid

def save_records_batch(
    self, table: str, records: List[Tuple[str, int, int, str]]
):
    """
    Save multiple records in a single transaction.

    Args:
        table: Table name ('measurements', 'forecast', or 'scheduling')
        records: List of tuples (timestamp, source_id, metric_id, value)
    """
    self._validate_table(table)
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.executemany(
            f"""
                INSERT INTO {table} (timestamp, source_id, metric_id, value)
                VALUES (?, ?, ?, ?)
            """,
            records,
        )

def get_records(
    self,
    table: str,
    source_id: int = None,
    metric_id: int = None,
    start_time: str = None,
    end_time: str = None,
):
    """
    Retrieve records from the specified table with optional filtering.

    Args:
        table: Table name ('measurements', 'forecast', or 'scheduling')
        source_id: Filter by source_id (optional)
        metric_id: Filter by metric_id (optional)
        start_time: Filter by start timestamp ISO string (optional)
        end_time: Filter by end timestamp ISO string (optional)

    Returns:
        List of records
    """
    self._validate_table(table)
    query = f"SELECT * FROM {table} WHERE 1=1"
    params = []

    if source_id is not None:
        query += " AND source_id = ?"
        params.append(source_id)

    if metric_id is not None:
        query += " AND metric_id = ?"
        params.append(metric_id)

    if start_time is not None:

```



```

        query += " AND timestamp >= ?"
        params.append(start_time)

    if end_time is not None:
        query += " AND timestamp <= ?"
        params.append(end_time)

    query += " ORDER BY timestamp"

    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(query, params)
        return cursor.fetchall()

def clear_records(
    self, table: str, source_id: int = None, from_time: str = None
):
    """
    Clear records from the specified table.

    Args:
        table: Table name ('measurements', 'forecast', or 'scheduling')
        source_id: Optional source_id filter (clears only this source if provided)
        from_time: Optional timestamp filter (clears only records from this time onwards)
    """
    self._validate_table(table)
    with self.get_connection() as conn:
        cursor = conn.cursor()
        if source_id is not None and from_time is not None:
            cursor.execute(
                f"DELETE FROM {table} WHERE source_id = ? AND timestamp >= ?",
                (source_id, from_time),
            )
        elif source_id is not None:
            cursor.execute(
                f"DELETE FROM {table} WHERE source_id = ?", (source_id,)
            )
        elif from_time is not None:
            cursor.execute(
                f"DELETE FROM {table} WHERE timestamp >= ?", (from_time,)
            )
        else:
            cursor.execute(f"DELETE FROM {table}")

# Source table methods
def save_source(self, source_name: str, source_type: str) -> int:
    """
    Save a source to the database.

    Args:
        source_name: Name of the source (e.g., 'boat_001', 'charger_01')
        source_type: Type of source (e.g., 'boat', 'charger', 'battery')

    Returns:
        The source_id of the inserted record
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            INSERT INTO source (source_name, source_type)
            VALUES (?, ?)
            """,
            (source_name, source_type),
        )
        return cursor.lastrowid

def get_source(self, source_id: int = None, source_name: str = None):

```

```

"""
Retrieve a source by ID or name.

Args:
    source_id: Source ID (optional)
    source_name: Source name (optional)

Returns:
    Source record or None
"""
with self.get_connection() as conn:
    cursor = conn.cursor()
    if source_id is not None:
        cursor.execute("SELECT * FROM source WHERE source_id = ?", (source_id,))
    elif source_name is not None:
        cursor.execute("SELECT * FROM source WHERE source_name = ?", (source_name,))
    else:
        return None
    return cursor.fetchone()

def get_all_sources(self, source_type: str = None):
    """
    Retrieve all sources, optionally filtered by type.

    Args:
        source_type: Filter by source type (optional)

    Returns:
        List of source records
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        if source_type:
            cursor.execute("SELECT * FROM source WHERE source_type = ?", (source_type,))
        else:
            cursor.execute("SELECT * FROM source")
        return cursor.fetchall()

def delete_source(self, source_id: int):
    """
    Delete a source by ID.

    Args:
        source_id: Source ID to delete
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("DELETE FROM source WHERE source_id = ?", (source_id,))

# Metric table methods
def save_metric(self, metric_name: str, unit: str, data_type: str) -> int:
    """
    Save a metric to the database.

    Args:
        metric_name: Name of the metric (e.g., 'power', 'soc', 'temperature')
        unit: Unit of measurement (e.g., 'kW', '%', 'C')
        data_type: Data type (e.g., 'float', 'int', 'bool')

    Returns:
        The metric_id of the inserted record
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            """
            INSERT INTO metric (metric_name, unit, data_type)
            VALUES (?, ?, ?)
        """

```

```

        """
        (metric_name, unit, data_type),
    )
    return cursor.lastrowid

def get_metric(self, metric_id: int = None, metric_name: str = None):
    """
    Retrieve a metric by ID or name.

    Args:
        metric_id: Metric ID (optional)
        metric_name: Metric name (optional)

    Returns:
        Metric record or None
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        if metric_id is not None:
            cursor.execute("SELECT * FROM metric WHERE metric_id = ?", (metric_id,))
        elif metric_name is not None:
            cursor.execute("SELECT * FROM metric WHERE metric_name = ?", (metric_name,))
        else:
            return None
        return cursor.fetchone()

def get_all_metrics(self, data_type: str = None):
    """
    Retrieve all metrics, optionally filtered by data type.

    Args:
        data_type: Filter by data type (optional)

    Returns:
        List of metric records
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        if data_type:
            cursor.execute("SELECT * FROM metric WHERE data_type = ?", (data_type,))
        else:
            cursor.execute("SELECT * FROM metric")
        return cursor.fetchall()

def delete_metric(self, metric_id: int):
    """
    Delete a metric by ID.

    Args:
        metric_id: Metric ID to delete
    """
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("DELETE FROM metric WHERE metric_id = ?", (metric_id,))

# Helper methods for ID resolution
def get_or_create_source(self, source_name: str, source_type: str) -> int:
    """
    Get source_id for a source, creating it if it doesn't exist.

    Uses caching to avoid repeated database lookups.

    Args:
        source_name: Name of the source (e.g., 'port', 'SeaBreeze', 'FastCharger_A')
        source_type: Type of source (e.g., 'port', 'boat', 'charger', 'pv', 'bess', 'weather')

    Returns:
        The source_id
    """

```

```

"""
# Check cache first
if source_name in self._source_cache:
    return self._source_cache[source_name]

# Try to get existing source
source = self.get_source(source_name=source_name)
if source:
    source_id = source["source_id"]
    self._source_cache[source_name] = source_id
    return source_id

# Create new source
with self.get_connection() as conn:
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO source (source_name, source_type)
        VALUES (?, ?)
        """,
        (source_name, source_type),
    )
    source_id = cursor.lastrowid

self._source_cache[source_name] = source_id
return source_id

def get_metric_id(self, metric_name: str) -> int:
    """
    Get metric_id for a metric by name.

    Uses caching to avoid repeated database lookups.
    Assumes metric already exists (from DEFAULT_METRICS or manual creation).

    Args:
        metric_name: Name of the metric (e.g., 'power_active', 'soc')

    Returns:
        The metric_id

    Raises:
        ValueError: If metric doesn't exist
    """
    # Check cache first
    if metric_name in self._metric_cache:
        return self._metric_cache[metric_name]

    # Get existing metric
    metric = self.get_metric(metric_name=metric_name)
    if metric:
        metric_id = metric["metric_id"]
        self._metric_cache[metric_name] = metric_id
        return metric_id

    raise ValueError(f"Metric '{metric_name}' not found. Ensure it's in DEFAULT_METRICS or created manually")

def clear_caches(self):
    """Clear the source and metric ID caches."""
    self._source_cache.clear()
    self._metric_cache.clear()

```

## forecasting/\_\_init\_\_.py

---

```
"""Forecasting module for port energy consumption and production."""  
  
from .port_forecaster import PortForecaster, EnergyForecast  
  
__all__ = ["PortForecaster", "EnergyForecast"]
```

# forecasting/port\_forecaster.py

---

```
"""Port energy production and boat availability forecasting."""

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Dict, List, Optional

from database import DatabaseManager
from models import Boat, Port, Trip

@dataclass
class EnergyForecast:
    """Forecast for a single timestep."""

    timestamp: datetime
    # PV production in kW: per source and port sum
    power_active_production_kw_by_source: Dict[str, float]
    power_active_production_kw: float
    # Boat energy need until next departure (kWh)
    boat_required_energy_kwh: Dict[str, float]
    # Connectivity to charge: 0 = not available, 1 = available
    boat_available: Dict[str, int]

class PortForecaster:
    """Forecast PV production and boat availability for the port."""

    def __init__(
        self,
        port: Port,
        db_manager: DatabaseManager,
        timestep_seconds: int = 900,
        trip_schedule: tuple = ((9, 0), (14, 1)),
    ):
        self.port = port
        self.db_manager = db_manager
        self.timestep_seconds = timestep_seconds
        # (hour_utc, slot_index) per day
        self.trip_schedule = trip_schedule

    def generate_daily_forecast(
        self, forecast_date: datetime, trip_assignments: Dict[str, List[Trip]]
    ) -> List[EnergyForecast]:
        """
        Generate 24-hour forecast for the port.

        Forecasted variables:
        - power_active_production (kW) per PV and port sum
        - boat_required_energy (kWh) until next departure
        - boat_available (0 or 1) per boat
        """
        forecasts = []
        timesteps_per_day = int(24 * 3600 / self.timestep_seconds)
        weather_forecasts = self._get_weather_forecasts(forecast_date)

        for step in range(timesteps_per_day):
            timestamp = forecast_date + timedelta(
                seconds=step * self.timestep_seconds
            )

            by_source, total_kw = self._forecast_power_active_production(
                timestamp, weather_forecasts
```

```

    )
    boat_required_energy = self._forecast_boat_required_energy(
        timestamp, trip_assignments
    )
    boat_available = self._forecast_boat_available(
        timestamp, trip_assignments
    )

    forecast = EnergyForecast(
        timestamp=timestamp,
        power_active_production_kw_by_source=by_source,
        power_active_production_kw=total_kw,
        boat_required_energy_kwh=boat_required_energy,
        boat_available=boat_available,
    )
    forecasts.append(forecast)

return forecasts

def _forecast_power_active_production(
    self, timestamp: datetime, weather_forecasts: Dict[str, Dict[str, float]]
) -> tuple:
    """
    Forecast PV power production at timestamp (kW per source and total).
    """
    by_source: Dict[str, float] = {}
    total_kw = 0.0

    if not self.port.pv_systems:
        return by_source, total_kw

    ts_str = timestamp.strftime("%Y-%m-%d %H:00:00")
    conditions = weather_forecasts.get(
        ts_str, {"ghi": 0.0, "dni": 0.0, "dhi": 0.0, "temperature": 20.0}
    )

    for pv in self.port.pv_systems:
        production_kw = pv.calculate_production(
            ghi=conditions.get("ghi", 0.0),
            dni=conditions.get("dni", 0.0),
            dhi=conditions.get("dhi", 0.0),
            temperature=conditions.get("temperature", 20.0),
            timestamp=timestamp,
        )
        by_source[pv.name] = production_kw
        total_kw += production_kw

    return by_source, total_kw

def _forecast_boat_required_energy(
    self, timestamp: datetime, trip_assignments: Dict[str, List[Trip]]
) -> Dict[str, float]:
    """Boat energy need (kWh) until next departure deadline."""
    out: Dict[str, float] = {}
    for boat in self.port.boats:
        trips = trip_assignments.get(boat.name, [])
        next_trip = self._next_departure_trip(timestamp, trips)
        if next_trip is None:
            out[boat.name] = 0.0
        else:
            out[boat.name] = next_trip.estimate_energy_required(boat.k)
    return out

def _departure_times(self, timestamp: datetime) -> List[datetime]:
    """Departure times for the day of timestamp from trip_schedule."""
    return [
        timestamp.replace(hour=hour, minute=0, second=0, microsecond=0)
        for hour, _ in self.trip_schedule
    ]

```

```

]

def _next_departure_trip(
    self, timestamp: datetime, trips: List[Trip]
) -> Optional[Trip]:
    """Next trip (by departure time) after timestamp; None if no more trips."""
    if not trips:
        return None
    start_times = self._departure_times(timestamp)

    if len(trips) >= 1 and timestamp < start_times[0]:
        return trips[0]
    if len(trips) >= 2:
        dur0 = trips[0].duration
        t_trip0_end = start_times[0] + timedelta(seconds=dur0)
        if t_trip0_end <= timestamp < start_times[1]:
            return trips[1]
        if timestamp >= start_times[1]:
            return None # after last departure
    else:
        dur0 = trips[0].duration
        t_trip0_end = start_times[0] + timedelta(seconds=dur0)
        if timestamp >= t_trip0_end:
            return None
    return None

def _forecast_boat_available(
    self, timestamp: datetime, trip_assignments: Dict[str, List[Trip]]
) -> Dict[str, int]:
    """Boat availability to charge: 1 = available, 0 = not available."""
    out: Dict[str, int] = {}
    for boat in self.port.boats:
        trips = trip_assignments.get(boat.name, [])
        is_sailing = self._is_boat_sailing(timestamp, trips)
        in_window = self._in_charging_window(timestamp, trips, boat)
        out[boat.name] = 1 if (not is_sailing and in_window) else 0
    return out

def _is_boat_sailing(self, timestamp: datetime, trips: List[Trip]) -> bool:
    """True if boat is sailing at timestamp (within a trip)."""
    if not trips:
        return False
    start_times = self._departure_times(timestamp)
    for i, trip in enumerate(trips):
        if i >= len(start_times):
            break
        t_end = start_times[i] + timedelta(seconds=trip.duration)
        if start_times[i] <= timestamp < t_end:
            return True
    return False

def _in_charging_window(
    self, timestamp: datetime, trips: List[Trip], _boat: Boat
) -> bool:
    """True if timestamp is inside a charging availability window."""
    if not trips:
        return False
    start_times = self._departure_times(timestamp)

    if timestamp < start_times[0]:
        return True
    for i, trip in enumerate(trips):
        if i >= len(start_times):
            break
        t_dep = start_times[i]
        t_dep_end = t_dep + timedelta(seconds=trip.duration)
        if i + 1 < len(start_times):
            if t_dep_end <= timestamp < start_times[i + 1]:

```



```

        return True
    else:
        if timestamp >= t_dep_end:
            return True
    return False

def _get_weather_forecasts(
    self, forecast_date: datetime
) -> Dict[str, Dict[str, float]]:
    """
    Get weather forecasts for a specific date from database.
    """
    weather_data = {}

    start_str = forecast_date.strftime("%Y-%m-%d 00:00:00")
    end_str = (forecast_date + timedelta(days=1)).strftime(
        "%Y-%m-%d 00:00:00"
    )

    openmeteo_src = self.db_manager.get_or_create_source(
        "openmeteo", "weather"
    )
    metrics = ["ghi", "dni", "dhi", "temperature"]

    for metric in metrics:
        metric_id = self.db_manager.get_metric_id(metric)
        forecast_rows = self.db_manager.get_records(
            "forecast",
            source_id=openmeteo_src,
            metric_id=metric_id,
            start_time=start_str,
            end_time=end_str,
        )

        for row in forecast_rows:
            ts_str = row["timestamp"]
            if ts_str not in weather_data:
                weather_data[ts_str] = {}
            weather_data[ts_str][metric] = float(row["value"])

    return weather_data

def save_forecasts_to_db(
    self, forecasts: List[EnergyForecast], forecast_type: str = "port_energy" # pylint: disable=unused-
) -> None:
    """Save forecasts to DB: power_active_production (per PV + port), boat_required_energy, boat_availability"""
    forecast_data = []
    port_src = self.db_manager.get_or_create_source(self.port.name, "port")
    power_met = self.db_manager.get_metric_id("power_active_production")
    boat_required_met = self.db_manager.get_metric_id(
        "boat_required_energy"
    )
    boat_available_met = self.db_manager.get_metric_id("boat_available")

    for forecast in forecasts:
        ts_str = forecast.timestamp.strftime("%Y-%m-%d %H:%M:%S")

        for source_name, kw in forecast.power_active_production_kw_by_source.items():
            src = self.db_manager.get_or_create_source(
                source_name, "pv"
            )
            forecast_data.append(
                (ts_str, src, power_met, str(kw))
            )
        forecast_data.append(
            (
                ts_str,
                port_src,
                boat_required_met,
                str(forecast.boat_required_energy),
                boat_available_met,
                str(forecast.boat_availability)
            )
        )

```

```

        power_met,
        str(forecast.power_active_production_kw),
    )
)

for boat_name, kwh in forecast.boat_required_energy_kwh.items():
    boat_src = self.db_manager.get_or_create_source(
        boat_name, "boat"
    )
    forecast_data.append(
        (ts_str, boat_src, boat_required_met, str(kwh))
    )

for boat_name, avail in forecast.boat_available.items():
    boat_src = self.db_manager.get_or_create_source(
        boat_name, "boat"
    )
    forecast_data.append(
        (ts_str, boat_src, boat_available_met, str(avail))
    )

if forecast_data:
    self.db_manager.save_records_batch("forecast", forecast_data)

def print_forecast_summary(self, forecasts: List[EnergyForecast]) -> None:
    """Print a short summary of the forecasts."""
    if not forecasts:
        print(" No forecasts to display")
        return

    total_production_kwh = sum(
        f.power_active_production_kw * (self.timestep_seconds / 3600)
        for f in forecasts
    )
    peak_f = max(forecasts, key=lambda x: x.power_active_production_kw)

    print("\n ■ Forecast summary (24h):")
    print(f"    Total PV production: {total_production_kwh:.2f} kWh")
    print(
        f"    Peak PV power: {peak_f.power_active_production_kw:.2f} kW at {peak_f.timestamp.strftime('%Y-%m-%d %H:%M:%S')}"
    )

```

# main.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    charger1 = Charger(
        name="FastCharger_A", max_power=22, efficiency=0.95
    ) # Adjust max_power (kW) and efficiency

    # PV system configuration
    # Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
    pv_system = PV(
        name="Solar_Array_1",
        capacity=10.0, # Adjust PV capacity (kW DC)
        tilt=30.0, # Adjust panel tilt angle (degrees)
        azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
        efficiency=0.95, # Adjust system efficiency (0.0-1.0)
        latitude=port.lat,
        longitude=port.lon,
    )

    # BESS (Battery Energy Storage System) configuration
    # Modify BESS parameters or remove BESS entirely
    bess = BESS(
        name="Battery_Storage_1",
        capacity=100.0, # Adjust BESS capacity (kWh)
```

```

        max_charge_power=25.0, # Adjust max charge power (kW)
        max_discharge_power=25.0, # Adjust max discharge power (kW)
        efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
        soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
        soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
        initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
        control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
    )

    # Add components to port
    port.add_boat(boat1)
    port.add_charger(charger1)
    # No PV system or BESS
    # port.add_pv(pv_system)
    # port.add_bess(bess)

    print(f"\nPort: {port}")
    print(f"Boats: {boat1.name}")
    print(f"Chargers: {charger1.name}")
    # print(f"PV: {pv_system}")
    # print(f"BESS: {bess}")

    # Simulation settings
    # Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
    settings = Settings(
        timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
        mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
        db_path="5_vessels_no_opt_no_der.db", # Change database file path if needed
        use_optimizer=False, # Set to False to disable optimization (SCIP)
        power_limit_mode=True,
    )

    # Initialize database
    db_manager = DatabaseManager(settings.db_path)
    db_manager.initialize_schema()
    db_manager.initialize_default_metrics()

    # Create and run simulation
    # Modify start_date (None = current time) and days (simulation duration)
    sim = SimulationEngine(
        port=port,
        settings=settings,
        db_manager=db_manager,
        start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
        days=1, # Adjust simulation duration (days)
    )

    sim.run()

    print("\n" + "=" * 60)
    print(f"✓ Results saved to: {settings.db_path}")
    print("=" * 60)

if __name__ == "__main__":
    main()

```

## models/\_\_init\_\_.py

---

```
"""Models for the electric port simulator."""

from .port import Port
from .boat import Boat, BoatState
from .charger import Charger, ChargerState
from .trip import Trip, TripPoint
from .pv import PV
from .bess import BESS, BESSControlStrategy

__all__ = [
    "Port",
    "Boat",
    "BoatState",
    "Charger",
    "ChargerState",
    "Trip",
    "TripPoint",
    "PV",
    "BESS",
    "BESSControlStrategy",
]
```

# models/bess.py

---

```
"""BESS (Battery Energy Storage System) model."""

from dataclasses import dataclass
from enum import Enum

class BESSControlStrategy(Enum):
    """Control strategies for BESS operation."""
    DEFAULT = "default" # Charge from PV surplus, discharge when needed
    # Future strategies can be added here (e.g., PEAK_SHAVING, ARBITRAGE, etc.)

@dataclass
class BESS:
    """
    Represents a Battery Energy Storage System at the port.

    Attributes:
        name: Name/identifier of the BESS
        capacity: Total energy capacity in kWh
        max_charge_power: Maximum charging power in kW
        max_discharge_power: Maximum discharging power in kW
        efficiency: Round-trip efficiency (0-1)
        soc_min: Minimum state of charge (0-1), default 0.10 (10%)
        soc_max: Maximum state of charge (0-1), default 0.90 (90%)
        initial_soc: Initial state of charge (0-1), default 0.50 (50%)
        control_strategy: Control strategy for operation
        current_soc: Current state of charge (0-1)
        current_power: Current power (kW) - positive=charging, negative=discharging
    """

    name: str
    capacity: float # kWh
    max_charge_power: float # kW
    max_discharge_power: float # kW
    efficiency: float = 0.90 # 90% round-trip efficiency
    soc_min: float = 0.10 # 10% minimum SOC
    soc_max: float = 0.90 # 90% maximum SOC
    initial_soc: float = 0.50 # Start at 50%
    control_strategy: BESSControlStrategy = BESSControlStrategy.DEFAULT
    current_soc: float = 0.50
    current_power: float = 0.0 # kW (+ charging, - discharging)

    def __post_init__(self):
        """Validate BESS attributes and initialize state."""
        if self.capacity <= 0:
            raise ValueError("BESS capacity must be positive")
        if self.max_charge_power <= 0:
            raise ValueError("Max charge power must be positive")
        if self.max_discharge_power <= 0:
            raise ValueError("Max discharge power must be positive")
        if not 0 < self.efficiency <= 1:
            raise ValueError("Efficiency must be between 0 and 1")
        if not 0 <= self.soc_min < self.soc_max <= 1:
            raise ValueError("SOC limits must satisfy: 0 ≤ soc_min < soc_max ≤ 1")
        if not self.soc_min <= self.initial_soc <= self.soc_max:
            raise ValueError("Initial SOC must be between soc_min and soc_max")

        # Initialize current SOC
        self.current_soc = self.initial_soc

    def charge(self, power: float, timestep_seconds: float) -> float:
```

```

"""
Charge the battery with given power for a timestep.

Args:
    power: Charging power in kW (must be positive)
    timestep_seconds: Duration of the timestep in seconds

Returns:
    Actual power charged (may be less than requested if SOC limit reached)
"""
if power < 0:
    raise ValueError("Charging power must be positive")

# Limit to max charge power
actual_power = min(power, self.max_charge_power)

# Calculate energy to be added (with efficiency loss during charging)
energy_added = (actual_power * timestep_seconds / 3600.0) * self.efficiency # kWh

# Calculate new SOC
new_soc = self.current_soc + (energy_added / self.capacity)

# Check SOC limit
if new_soc > self.soc_max:
    # Reduce charging to hit exactly soc_max
    allowed_energy = (self.soc_max - self.current_soc) * self.capacity
    actual_power = (allowed_energy / self.efficiency) / (timestep_seconds / 3600.0)
    self.current_soc = self.soc_max
else:
    self.current_soc = new_soc

self.current_power = actual_power
return actual_power

def discharge(self, power: float, timestep_seconds: float) -> float:
    """
    Discharge the battery with given power for a timestep.

    Args:
        power: Discharging power in kW (must be positive)
        timestep_seconds: Duration of the timestep in seconds

    Returns:
        Actual power discharged (may be less than requested if SOC limit reached)
    """
    if power < 0:
        raise ValueError("Discharging power must be positive")

    # Limit to max discharge power
    actual_power = min(power, self.max_discharge_power)

    # Calculate energy to be removed (with efficiency loss during discharging)
    # When discharging, we lose energy due to efficiency
    energy_removed = actual_power * timestep_seconds / 3600.0 # kWh delivered
    energy_from_battery = energy_removed / self.efficiency # kWh removed from battery

    # Calculate new SOC
    new_soc = self.current_soc - (energy_from_battery / self.capacity)

    # Check SOC limit
    if new_soc < self.soc_min:
        # Reduce discharging to hit exactly soc_min
        allowed_energy = (self.current_soc - self.soc_min) * self.capacity
        actual_power = (allowed_energy * self.efficiency) / (timestep_seconds / 3600.0)
        self.current_soc = self.soc_min
    else:
        self.current_soc = new_soc
    self.current_power = -actual_power # Negative for discharging

```

```

        return actual_power

def get_available_energy(self) -> float:
    """
    Get available energy that can be discharged (kWh).

    Returns:
        Available energy in kWh
    """
    return (self.current_soc - self.soc_min) * self.capacity * self.efficiency

def get_available_charge_capacity(self) -> float:
    """
    Get available capacity for charging (kWh).

    Returns:
        Available charge capacity in kWh
    """
    return (self.soc_max - self.current_soc) * self.capacity / self.efficiency

def get_max_discharge_power_available(self, timestep_seconds: float) -> float:
    """
    Get maximum power that can be discharged in the current timestep.

    Args:
        timestep_seconds: Duration of the timestep in seconds

    Returns:
        Maximum discharge power in kW
    """
    # Limited by available energy
    available_energy = self.get_available_energy()
    energy_limited_power = available_energy / (timestep_seconds / 3600.0)

    # Limited by max discharge power
    return min(self.max_discharge_power, energy_limited_power)

def get_max_charge_power_available(self, timestep_seconds: float) -> float:
    """
    Get maximum power that can be charged in the current timestep.

    Args:
        timestep_seconds: Duration of the timestep in seconds

    Returns:
        Maximum charge power in kW
    """
    # Limited by available capacity
    available_capacity = self.get_available_charge_capacity()
    capacity_limited_power = available_capacity / (timestep_seconds / 3600.0)

    # Limited by max charge power
    return min(self.max_charge_power, capacity_limited_power)

def idle(self) -> None:
    """Set battery to idle state (no charging or discharging)."""
    self.current_power = 0.0

def get_energy_stored(self) -> float:
    """
    Get current energy stored in the battery.

    Returns:
        Energy stored in kWh
    """
    return self.current_soc * self.capacity

def __repr__(self) -> str:

```



```
return (
    f"BESS(name='{self.name}', capacity={self.capacity}kWh, "
    f"SOC={self.current_soc*100:.1f}%, "
    f"power={self.current_power:+.2f}kW, "
    f"strategy={self.control_strategy.value})"
)
```

# models/boat.py

---

```
"""Boat model for the electric recreational port simulator."""

from dataclasses import dataclass, field
from enum import Enum

HULL_FACTOR = 1

class BoatState(Enum):
    """Possible states for a boat."""

    IDLE = "idle"
    CHARGING = "charging"
    SAILING = "sailing"

@dataclass
class Boat:
    """
    Represents an electric boat in the port.

    Attributes:
        name: Name/identifier of the boat
        motor_power: Motor power in kW
        weight: Boat weight in kg
        length: Boat length in meters
        battery_capacity: Battery capacity in kWh
        range_speed: Range speed in knots
        soc: State of charge (0-1, where 1 = 100%)
        _state: Current state of the boat
    """

    motor_power: int # kW
    weight: float # kg
    length: float # m
    battery_capacity: float # kWh
    range_speed: float # knots
    soc: float = 1.0 # Default fully charged (0-1)
    name: str = "" # Will be set in __post_init__ if empty
    _state: BoatState = field(default=BoatState.IDLE, init=False)

    # Class variable to track boat count for default naming
    _boat_count: int = field(default=0, init=False, repr=False)

    def __post_init__(self):
        """Validate boat attributes and set default name if needed."""
        # Set default name if not provided
        if not self.name:
            Boat._boat_count = getattr(Boat, "_boat_count", 0) + 1
            self.name = f"Boat_{Boat._boat_count}"

        # Validation
        if self.motor_power <= 0:
            raise ValueError("Motor power must be positive")
        if self.weight <= 0:
            raise ValueError("Weight must be positive")
        if self.length <= 0:
            raise ValueError("Length must be positive")
        if self.battery_capacity <= 0:
            raise ValueError("Battery capacity must be positive")
        if self.range_speed <= 0:
```

```

        raise ValueError("Range speed must be positive")
    if not 0 <= self.soc <= 1:
        raise ValueError("SOC must be between 0 and 1")

@property
def k(self) -> float:
    """
    Calculate k-factor for cube law: motor_power / range_speed^3.

    Returns:
        K-factor for power consumption calculations
    """
    return self.motor_power / (self.range_speed**3) * HULL_FACTOR

@property
def state(self) -> BoatState:
    """Get the current state of the boat."""
    return self._state

@state.setter
def state(self, new_state: BoatState):
    """
    Set the state of the boat.

    Args:
        new_state: New state for the boat
    """
    if not isinstance(new_state, BoatState):
        raise ValueError(f"State must be a BoatState enum, got {type(new_state)}")
    self._state = new_state

def __repr__(self) -> str:
    return (
        f"Boat(name='{self.name}', motor_power={self.motor_power}kW, "
        f"battery={self.battery_capacity}kWh, soc={self.soc:.1%}, "
        f"state={self.state.value})"
    )

```

# models/charger.py

---

```
"""Charger model for the electric recreational port simulator."""

from dataclasses import dataclass, field
from enum import Enum
from typing import Optional

class ChargerState(Enum):
    """Possible states for a charger."""

    IDLE = "idle"
    CHARGING = "charging"

@dataclass
class Charger:
    """
    Represents a charging station in the port.

    Attributes:
        name: Name/identifier of the charger
        max_power: Maximum power output in kW
        efficiency: Charging efficiency (0-1, where 1 = 100%)
        power: Current power output in kW
        _state: Current state of the charger
        connected_boat: Reference to connected boat (if any)
    """

    max_power: int # kW
    efficiency: float = 0.95 # Default 95% efficiency (0-1)
    power: float = 0.0 # Current power output in kW
    name: str = "" # Will be set in __post_init__ if empty
    _state: ChargerState = field(default=ChargerState.IDLE, init=False)
    connected_boat: Optional[str] = field(
        default=None, init=False
    ) # Boat name/id if connected

    # Class variable to track charger count for default naming
    _charger_count: int = field(default=0, init=False, repr=False)

    def __post_init__(self):
        """Validate charger attributes and set default name if needed."""
        # Set default name if not provided
        if not self.name:
            Charger._charger_count = getattr(Charger, "_charger_count", 0) + 1
            self.name = f"Charger_{Charger._charger_count}"

        # Validation
        if self.max_power <= 0:
            raise ValueError("Max power must be positive")
        if not 0 < self.efficiency <= 1:
            raise ValueError("Efficiency must be between 0 and 1")
        if self.power < 0:
            raise ValueError("Power cannot be negative")
        if self.power > self.max_power:
            raise ValueError("Power cannot exceed max_power")

    @property
    def state(self) -> ChargerState:
        """Get the current state of the charger."""
        return self._state
    @state.setter
```

```

def state(self, new_state: ChargerState):
    """
    Set the state of the charger.

    Args:
        new_state: New state for the charger
    """
    if not isinstance(new_state, ChargerState):
        raise ValueError(
            f"State must be a ChargerState enum, got {type(new_state)}"
        )
    self._state = new_state

    # Reset power to 0 when going idle
    if new_state == ChargerState.IDLE:
        self.power = 0.0
        self.connected_boat = None

@property
def effective_power(self) -> float:
    """
    Calculate the effective power delivered to the battery after efficiency losses.

    Returns:
        Effective power in kW
    """
    return self.power * self.efficiency

def __repr__(self) -> str:
    boat_info = f", connected to '{self.connected_boat}'" if self.connected_boat else ""
    return (
        f"Charger(name='{self.name}', max_power={self.max_power}kW, "
        f"efficiency={self.efficiency:.1%}, power={self.power}kW, "
        f"state={self.state.value}{boat_info})"
    )

```

# models/port.py

---

```
"""Port model for the electric recreational port simulator."""

import json
from pathlib import Path
from datetime import datetime
from dataclasses import dataclass, field
from typing import List, Dict, Optional, TYPE_CHECKING

if TYPE_CHECKING:
    from .boat import Boat
    from .charger import Charger
    from .pv import PV
    from .bess import BESS

@dataclass
class Port:
    """
    Represents an electric recreational port.

    Attributes:
        name: Name of the port
        contracted_power: Maximum contracted power in kW
        lat: Latitude coordinate
        lon: Longitude coordinate
        boats: List of boats at this port
        chargers: List of chargers at this port
        pv_systems: List of PV systems at this port
        bess_systems: List of BESS (battery storage) systems at this port
    """

    name: str
    contracted_power: int # in kW
    lat: float
    lon: float
    boats: List["Boat"] = field(default_factory=list)
    chargers: List["Charger"] = field(default_factory=list)
    pv_systems: List["PV"] = field(default_factory=list)
    bess_systems: List["BESS"] = field(default_factory=list)
    tariff_path: Optional[str] = None # Path to tariff JSON file
    _tariff_data: Optional[Dict] = field(default=None, init=False, repr=False)

    def __post_init__(self):
        """Validate port attributes."""
        if self.contracted_power <= 0:
            raise ValueError("Contracted power must be positive")
        if not -90 <= self.lat <= 90:
            raise ValueError("Latitude must be between -90 and 90")
        if not -180 <= self.lon <= 180:
            raise ValueError("Longitude must be between -180 and 180")

        # Load tariff if path is provided
        if self.tariff_path:
            self._load_tariff()
        else:
            # Try default tariff path
            default_path = Path(__file__).parent.parent / "assets" / "tariff" / "default_tariff.json"
            if default_path.exists():
                self.tariff_path = str(default_path)
                self._load_tariff()

    def _load_tariff(self) -> None:
```

```

        """Load tariff data from JSON file."""
        if not self.tariff_path:
            return

        tariff_file = Path(self.tariff_path)
        if not tariff_file.exists():
            raise FileNotFoundError(f"Tariff file not found: {self.tariff_path}")

        try:
            with open(tariff_file, 'r') as f:
                self._tariff_data = json.load(f)
        except json.JSONDecodeError as e:
            raise ValueError(f"Invalid JSON in tariff file: {e}")

    @property
    def tariff(self) -> Optional[Dict]:
        """Get tariff data."""
        return self._tariff_data

    def get_tariff_price(self, timestamp: datetime) -> float:
        """
        Get electricity price for a specific timestamp.

        Args:
            timestamp: Datetime to get price for

        Returns:
            Price per kWh in the tariff currency (default: 0.0 if no tariff)
        """
        if not self._tariff_data or "tariff" not in self._tariff_data:
            return 0.0

        # Get day of week (0=Monday, 6=Sunday)
        weekday = timestamp.weekday()
        day_key = str(weekday)

        # Get pricing for this day
        if day_key not in self._tariff_data["tariff"]:
            return 0.0

        day_pricing = self._tariff_data["tariff"][day_key]["pricing"]

        # Format time as HH:MM (15-minute intervals)
        time_str = timestamp.strftime("%H:%M")

        # Round to nearest 15 minutes
        minute = timestamp.minute
        rounded_minute = (minute // 15) * 15
        time_str = timestamp.strftime(f"%H:{rounded_minute:02d}")

        # Get price for this time slot
        return day_pricing.get(time_str, 0.0)

    def add_boat(self, boat: "Boat") -> None:
        """Add a boat to the port."""
        self.boats.append(boat)

    def add_charger(self, charger: "Charger") -> None:
        """Add a charger to the port."""
        self.chargers.append(charger)

    def add_pv(self, pv: "PV") -> None:
        """Add a PV system to the port."""
        self.pv_systems.append(pv)

    def add_bess(self, bess: "BESS") -> None:
        """Add a BESS to the port."""
        self.bess_systems.append(bess)

```

```
def __repr__(self) -> str:
    return (
        f"Port(name='{self.name}', contracted_power={self.contracted_power}kW, "
        f"coordinates=({self.lat}, {self.lon}), "
        f"boats={len(self.boats)}, chargers={len(self.chargers)}, "
        f"pv={len(self.pv_systems)}, bess={len(self.bess_systems)})"
    )
```



# models/pv.py

---

```
from dataclasses import dataclass
import pvlib

# Constants for DC power model
REFERENCE_IRRADIANCE = 1000.0 # W/m² STC (standard test conditions)
REFERENCE_TEMP = 25.0 # °C STC (standard test conditions)
TEMP_COEFF_PDC = -0.004 # 1/°C, typical for crystalline silicon (PVWatts)

# Sandia temperature model parameters for open rack glass/polymer (PVWatts)
SAPM_TEMP_PARAMS = pvlib.temperature.TEMPERATURE_MODEL_PARAMETERS["sapm"][
    "open_rack_glass_polymer"
]

@dataclass
class PV:
    name: str
    capacity: float # kW DC at STC (pdc0)
    tilt: float = 30.0
    azimuth: float = 180.0
    latitude: float = 0.0
    longitude: float = 0.0
    current_production: float = 0.0

    def __post_init__(self):
        if self.capacity <= 0:
            raise ValueError("Capacity must be positive")
        if not 0 <= self.tilt <= 90:
            raise ValueError("Tilt must be 0-90°")
        if not 0 <= self.azimuth < 360:
            raise ValueError("Azimuth must be 0-360°")
        if not -90 <= self.latitude <= 90:
            raise ValueError("Latitude must be -90-90")
        if not -180 <= self.longitude <= 180:
            raise ValueError("Longitude must be -180-180")

    def calculate_production(
        self,
        ghi: float,
        dni: float,
        dhi: float,
        temperature: float,
        timestamp,
        wind_speed: float = 1.0, # m/s (assumed to be 1 m/s)
    ) -> float:

        solpos = pvlib.solarposition.get_solarposition(
            timestamp, self.latitude, self.longitude
        )

        if solpos["apparent_elevation"].iloc[0] <= 0:
            self.current_production = 0.0
            return 0.0

        poa = pvlib.irradiance.get_total_irradiance(
            surface_tilt=self.tilt,
            surface_azimuth=self.azimuth,
            solar_zenith=solpos["zenith"].iloc[0],
            solar_azimuth=solpos["azimuth"].iloc[0],
            dni=dni,
            ghi=ghi,
            dhi=dhi,
```

```

        albedo=0.2,
    )["poa_global"]

    # Sandia cell temperature model
    cell_temperature = pvlib.temperature.sapm_cell(
        poa_global=poa,
        temp_air=temperature,
        wind_speed=wind_speed,
        **SAPM_TEMP_PARAMS,
    )

    pdc = pvlib.pvsystem.pvwatts_dc(
        effective_irradiance=poa,
        temp_cell=cell_temperature,
        pdc0=self.capacity * REFERENCE_IRRADIANCE,
        gamma_pdc=TEMP_COEFF_PDC,
        temp_ref=REFERENCE_TEMP,
    )

    self.current_production = max(0.0, float(pdc) / 1000.0)
    return self.current_production

def __repr__(self) -> str:
    return (
        f"PV(name='{self.name}', capacity={self.capacity}kW, "
        f"tilt={self.tilt}°, azimuth={self.azimuth}°, "
        f"production={self.current_production:.2f}kW)"
    )

```

# models/trip.py

---

```
"""Trip model for boat routes."""

import csv
from dataclasses import dataclass
from datetime import datetime, timedelta
from pathlib import Path
from typing import List, Optional

@dataclass
class TripPoint:
    """A single point in a trip route."""

    timestamp: datetime
    point_type: str # Static, Dock, Terrestrial, Interpolated
    speed: float # knots
    heading: float # degrees
    latitude: float
    longitude: float

class Trip:
    """
    Represents a boat trip with waypoints.

    Attributes:
        route_name: Name of the route (e.g., 'route_1')
        points: List of trip waypoints
        duration: Total trip duration in seconds
    """

    def __init__(self, csv_path: str):
        """
        Load a trip from a CSV file.

        Args:
            csv_path: Path to the CSV file containing trip data
        """
        self.route_name = Path(csv_path).stem
        self.points: List[TripPoint] = []
        self._load_from_csv(csv_path)

        if self.points:
            # Calculate trip duration
            first_time = self.points[0].timestamp
            last_time = self.points[-1].timestamp
            self.duration = (last_time - first_time).total_seconds()
        else:
            self.duration = 0

    def _load_from_csv(self, csv_path: str):
        """Load trip points from CSV file."""
        with open(csv_path, "r") as f:
            reader = csv.DictReader(f)
            for row_num, row in enumerate(reader, start=2): # Start at 2 (header is line 1)
                # Skip completely empty rows
                if not any(row.values()):
                    continue

                # Skip rows without timestamp
                if not row.get("timestamp") or not row.get("timestamp").strip():
                    continue
```

```

try:
    # Parse timestamp - handle both microseconds and nanoseconds
    timestamp_str = row["timestamp"].strip()
    try:
        # Try with microseconds first (standard Python format)
        timestamp = datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S.%f")
    except ValueError:
        # Handle nanoseconds by truncating to microseconds
        if "." in timestamp_str:
            base, frac = timestamp_str.split(".")
            # Truncate to 6 digits (microseconds)
            frac = frac[:6].ljust(6, "0")
            timestamp_str = f"{base}.{frac}"
            timestamp = datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S.%f")
        else:
            timestamp = datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S")

    # Skip rows with empty data fields
    if not row.get("speed") or not row.get("speed").strip():
        continue
    if not row.get("latitude") or not row.get("latitude").strip():
        continue

    point = TripPoint(
        timestamp=timestamp,
        point_type=row.get("type", ""),
        speed=float(row["speed"]),
        heading=float(row["heading"]) if row.get("heading") else 0.0,
        latitude=float(row["latitude"]),
        longitude=float(row["longitude"]),
    )
    self.points.append(point)
except (ValueError, KeyError) as e:
    # Skip problematic rows silently
    pass

def get_point_at_elapsed_time(self, elapsed_seconds: float) -> Optional[TripPoint]:
    """
    Get the trip point at a given elapsed time from trip start.

    Args:
        elapsed_seconds: Seconds elapsed since trip start

    Returns:
        The closest trip point, or None if past the end
    """
    if not self.points or elapsed_seconds > self.duration:
        return None

    # Find the closest point by time
    target_time = self.points[0].timestamp + timedelta(seconds=elapsed_seconds)

    # Binary search would be more efficient, but linear is fine for our data size
    closest_point = self.points[0]
    min_diff = abs((target_time - closest_point.timestamp).total_seconds())

    for point in self.points[1:]:
        diff = abs((target_time - point.timestamp).total_seconds())
        if diff < min_diff:
            min_diff = diff
            closest_point = point
        else:
            # Points are sorted, so we can stop once diff starts increasing
            break

    return closest_point

def estimate_energy_required(self, boat_k_factor: float) -> float:

```

```

"""
Estimate total energy required for this trip.

Args:
    boat_k_factor: The boat's k factor (motor_power / range_speed^3)

Returns:
    Estimated energy in kWh
"""
if not self.points:
    return 0.0

# Sum up energy for each segment based on speed
total_energy = 0.0
for i in range(len(self.points) - 1):
    point = self.points[i]
    next_point = self.points[i + 1]

    # Time for this segment
    segment_duration = (
        next_point.timestamp - point.timestamp
    ).total_seconds() # seconds

    # Power consumption at this speed
    speed_knots = point.speed
    power_kw = boat_k_factor * (speed_knots**3)

    # Energy for this segment
    energy_kwh = (power_kw * segment_duration) / 3600

    total_energy += energy_kwh

return total_energy

def get_energy_between(
    self, start_elapsed_seconds: float, end_elapsed_seconds: float, boat_k_factor: float
) -> float:
    """
    Energy consumed in the time window [start_elapsed, end_elapsed] using the
    same segment-based integration as estimate_energy_required. Use this in
    the simulation so in-trip discharge matches the predicted requirement.

    Args:
        start_elapsed_seconds: Start of window (s from trip start)
        end_elapsed_seconds: End of window (s from trip start)
        boat_k_factor: Boat k factor (motor_power / range_speed^3)

    Returns:
        Energy in kWh for that window
    """
    if not self.points or start_elapsed_seconds >= end_elapsed_seconds:
        return 0.0

    t0 = self.points[0].timestamp
    total_energy = 0.0
    for i in range(len(self.points) - 1):
        point = self.points[i]
        next_point = self.points[i + 1]
        seg_start = (point.timestamp - t0).total_seconds()
        seg_end = (next_point.timestamp - t0).total_seconds()

        overlap_start = max(seg_start, start_elapsed_seconds)
        overlap_end = min(seg_end, end_elapsed_seconds)
        duration = max(0.0, overlap_end - overlap_start)
        if duration <= 0:
            continue

        power_kw = boat_k_factor * (point.speed**3)

```

```
        total_energy += (power_kw * duration) / 3600

    return total_energy

def __repr__(self) -> str:
    return (
        f"Trip(route={self.route_name}, points={len(self.points)}, "
        f"duration={self.duration/3600:.2f}h)"
    )
```

## optimization/\_\_init\_\_.py

---

```
"""Optimization module for port energy management."""  
  
from .base_optimizer import BaseOptimizer, BaseOptimizationResult  
  
__all__ = ["BaseOptimizer", "BaseOptimizationResult"]
```

# optimization/base\_optimizer.py

---

```
"""Base optimizer: minimize cost subject to import <= contracted_power.

Supports PV production (free solar) and BESS (battery storage) to reduce
grid import costs. The power balance is:

    grid_import + pv_production +  $\Sigma$  bess_discharge
    =  $\Sigma$  charger_power +  $\Sigma$  bess_charge + pv_export

Performance-tuned for 10+ vessel fleets:
- Tight big-M bounds (energy_req, not battery_capacity)
- SOS1 sets on departure slots for better branching
- Strong symmetry breaking across all delay slots
- MIP warm-start with greedy on-time schedule
- Solver parameter tuning (time limit, gap tolerance, emphasis)
"""

from collections import defaultdict
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Dict, List, Optional, Tuple

from pycscpt import Model, quicksum

from database import DatabaseManager
from forecasting import EnergyForecast
from models import Port

@dataclass
class BaseOptimizationResult:
    status: str
    charger_schedules: Dict[str, List[Tuple[datetime, float]]]
    boat_schedules: Dict[str, List[Tuple[datetime, float]]]
    peak_power_kw: float
    total_energy_kwh: float
    total_cost: float

class BaseOptimizer:
    """Minimize cost. Single constraint: grid import <= contracted_power."""

    def __init__(
        self,
        port: Port,
        db_manager: DatabaseManager,
        timestep_seconds: int = 900,
        boat_charger_assignments: Dict[str, int] = None,
        trip_schedule: tuple = ((9, 0), (14, 1)),
        max_slack_timesteps: int = 8,
        deadline_decay_factor: float = 0.5,
        bess_end_soc_penalty: float = 0.5,
        time_limit_seconds: int = 120,
        mip_gap: float = 0.01,
    ):
        self.port = port
        self.db_manager = db_manager
        self.timestep_seconds = timestep_seconds
        self.timestep_hours = timestep_seconds / 3600.0
        self.boat_charger_assignments = boat_charger_assignments or {}
        self.trip_schedule = trip_schedule
        self.max_slack_timesteps = max_slack_timesteps
        self.deadline_decay_factor = deadline_decay_factor
```



```

self.bess_end_soc_penalty = bess_end_soc_penalty
self.time_limit_seconds = time_limit_seconds
self.mip_gap = mip_gap

self._boat_battery_cap = {b.name: b.battery_capacity for b in self.port.boats}

# ----- #
# Public entry point                                     #
# ----- #

def optimize_daily_schedule(
    self,
    forecast_date: datetime,
    energy_forecasts: List[EnergyForecast],
) -> BaseOptimizationResult:

    T = len(energy_forecasts)
    timesteps = list(range(T))

    model = Model("base_optimizer")
    model.hideOutput()

    # --- Solver tuning for large fleets ---
    model.setParam("limits/time", self.time_limit_seconds)
    model.setParam("limits/gap", self.mip_gap)
    # Emphasise finding feasible solutions fast
    # model.setParam("heuristics/emphasis", "aggressive")
    # model.setParam("separating/maxrounds", 5) # Limit cut rounds at root

    trip_events = self._extract_trip_events(energy_forecasts)
    print("TRIP EVENTS", trip_events)

    # Lookups
    boat_objects = {b.name: b for b in self.port.boats}
    charger_objects = {c.name: c for c in self.port.chargers}
    charger_to_boat = {
        self.port.chargers[idx].name: boat
        for boat, idx in self.boat_charger_assignments.items()
    }
    boat_list = list(self.boat_charger_assignments.keys())

    # --- Decision variables ---
    grid_import = self._add_grid_import_vars(model, timesteps)
    charger_power = self._add_charger_power_vars(model, timesteps)

    # --- PV (parameter) + BESS (variables) + export (slack) ---
    pv_forecast = self._get_pv_forecast(energy_forecasts, timesteps)
    pv_export = self._add_export_vars(model, timesteps)
    bess_charge, bess_discharge, bess_soc = self._add_bess_variables(
        model, timesteps
    )

    # --- Power balance (equality with export slack – tighter LP) ---
    self._add_power_balance_constraints(
        model,
        timesteps,
        grid_import,
        charger_power,
        pv_forecast,
        bess_charge,
        bess_discharge,
        pv_export,
    )

    # --- BESS SOC dynamics ---
    self._add_bess_soc_dynamics(
        model, timesteps, bess_charge, bess_discharge, bess_soc
    )

```

```

# --- Departure / at-sea variables ---
departure_ctx = self._add_departure_variables(
    model, T, timesteps, trip_events, boat_list, forecast_date
)

# --- Aggregate boat_away (compact no-charge linking) ---
boat_away = self._add_boat_away_variables(
    model, timesteps, trip_events, departure_ctx["at_sea"]
)

# --- Return-drain (explicit vars for tighter LP bounds) ---
drain_at = self._add_drain_variables(model, T, trip_events, departure_ctx)

# --- SOC dynamics ---
soc = self._add_soc_variables_and_dynamics(
    model,
    timesteps,
    trip_events,
    charger_to_boat,
    boat_objects,
    charger_objects,
    charger_power,
    boat_away,
    drain_at,
)

# --- SOC >= energy_req at departure (tight big-M) ---
self._add_departure_soc_constraints(model, T, trip_events, soc, departure_ctx)

# --- Symmetry breaking (strengthened) ---
self._add_symmetry_breaking(model, trip_events, boat_list, departure_ctx)

# --- Tariffs & objective ---
tariff_price = self._compute_tariff_prices(forecast_date, timesteps)

self._set_objective(
    model,
    timesteps,
    grid_import,
    tariff_price,
    boat_list,
    trip_events,
    departure_ctx,
    bess_soc,
)

# --- MIP warm-start: greedy on-time solution ---
# self._add_warm_start(model, trip_events, departure_ctx)

# --- Solve ---
model.optimize()
status = model.getStatus()
if status not in ("optimal", "bestsollimit", "timelimit"):
    # Accept any status that produced at least one feasible solution
    if model.getNSols() == 0:
        raise RuntimeError(f"SCIP failed ({status}), no feasible solution")
    print(
        f"    Warning: solver status '{status}' but using best solution found"
    )

# --- Post-processing ---
self._log_departure_decisions(model, forecast_date, trip_events, departure_ctx)
self._log_bess_summary(model, timesteps, bess_charge, bess_discharge, bess_soc)
self._log_pv_summary(model, timesteps, pv_forecast, pv_export)

result = self._extract_results(
    model,
    forecast_date,

```

```

        timesteps,
        charger_power,
        grid_import,
        tariff_price,
        status,
    )

    self._save_bess_schedules_to_db(
        model, forecast_date, timesteps, bess_charge, bess_discharge
    )

    return result

# ----- #
# Variable & constraint builders                                     #
# ----- #

def _add_grid_import_vars(self, model, timesteps):
    return {
        t: model.addVar(lb=0, ub=self.port.contracted_power, name=f"grid_{t}")
        for t in timesteps
    }

def _add_charger_power_vars(self, model, timesteps):
    charger_power = {}
    for charger in self.port.chargers:
        charger_power[charger.name] = {
            t: model.addVar(
                lb=0, ub=charger.max_power, name=f"p_{charger.name}_{t}"
            )
            for t in timesteps
        }
    return charger_power

# ----- #
# PV, BESS, and export variables                                     #
# ----- #

def _get_pv_forecast(self, energy_forecasts, timesteps):
    return {t: energy_forecasts[t].power_active_production_kw for t in timesteps}

def _add_export_vars(self, model, timesteps):
    """Slack variable for excess power (PV curtailment / grid export).

    Keeping this as an explicit variable gives an equality power-balance
    constraint, which produces a tighter LP relaxation than an inequality.
    """
    return {t: model.addVar(lb=0, name=f"export_{t}") for t in timesteps}

def _add_bess_variables(self, model, timesteps):
    bess_charge = {}
    bess_discharge = {}
    bess_soc = {}

    for bess in self.port.bess_systems:
        bess_charge[bess.name] = {
            t: model.addVar(
                lb=0,
                ub=bess.max_charge_power,
                name=f"bess_chg_{bess.name}_{t}",
            )
            for t in timesteps
        }
        bess_discharge[bess.name] = {
            t: model.addVar(
                lb=0,
                ub=bess.max_discharge_power,
                name=f"bess_dis_{bess.name}_{t}",
            )

```

```

        )
        for t in timesteps
    }
    bess_soc[bess.name] = {
        t: model.addVar(
            lb=bess.soc_min * bess.capacity,
            ub=bess.soc_max * bess.capacity,
            name=f"bess_soc_{bess.name}_{t}",
        )
        for t in timesteps
    }

    return bess_charge, bess_discharge, bess_soc

def _add_bess_soc_dynamics(
    self, model, timesteps, bess_charge, bess_discharge, bess_soc
):
    for bess in self.port.bess_systems:
        initial_soc_kwh = bess.current_soc * bess.capacity
        for t in timesteps:
            soc_prev = bess_soc[bess.name][t - 1] if t > 0 else initial_soc_kwh
            energy_in = (
                bess_charge[bess.name][t] * bess.efficiency * self.timestep_hours
            )
            energy_out = (
                bess_discharge[bess.name][t]
                * (1.0 / bess.efficiency)
                * self.timestep_hours
            )
            model.addCons(
                bess_soc[bess.name][t] == soc_prev + energy_in - energy_out,
                name=f"bess_soc_dyn_{bess.name}_{t}",
            )

# ----- #
# Power balance (equality – tighter LP than inequality) #
# ----- #

def _add_power_balance_constraints(
    self,
    model,
    timesteps,
    grid_import,
    charger_power,
    pv_forecast,
    bess_charge,
    bess_discharge,
    pv_export,
):
    """
    Power balance at each timestep (equality for tight LP relaxation):

    grid_import + pv_production +  $\Sigma$  bess_discharge
    =  $\Sigma$  charger_power +  $\Sigma$  bess_charge + pv_export
    """
    for t in timesteps:
        supply = grid_import[t] + pv_forecast[t]
        if self.port.bess_systems:
            supply += quicksum(
                bess_discharge[b.name][t] for b in self.port.bess_systems
            )

        demand = quicksum(charger_power[c.name][t] for c in self.port.chargers)
        if self.port.bess_systems:
            demand += quicksum(
                bess_charge[b.name][t] for b in self.port.bess_systems
            )
        demand += pv_export[t]

```

```

        model.addCons(supply == demand, name=f"power_balance_{t}")

# ----- #
# Departure / at-sea with SOS1 + branching priorities #
# ----- #

def _add_departure_variables(
    self,
    model,
    T,
    timesteps,
    trip_events,
    boat_list,
    forecast_date,    # <-- REQUIRED for correct cutoff
):
    """
    Create depart_at, did_depart, fully_ready, at_sea variables.

    Fixes:
    ■ Departure cutoff enforced at 18:00 RELATIVE TO forecast_date
    ■ Prevents optimizer from abandoning chargeable trips
    ■ Keeps SOS1 + strong branching
    """

    depart_at = {}
    depart_slots = {}
    at_sea = {}
    fully_ready = {}
    did_depart = {}

    # -----
    # Hard operational cutoff: 18:00 relative to forecast_date
    # -----
    cutoff_time = forecast_date.replace(hour=18, minute=0, second=0)
    cutoff_t = int(
        (cutoff_time - forecast_date).total_seconds()
        / self.timestep_seconds
    )

    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])

        for i, (t_deadline, energy_req, dur) in enumerate(boat_trips):
            t_orig_depart = t_deadline + 1

            # --- compute maximum allowed delay ---
            if i + 1 < len(boat_trips):
                next_deadline = boat_trips[i + 1][0]
                max_late = next_deadline - dur - t_orig_depart
            else:
                max_late = T - dur - t_orig_depart

            max_late = max(0, min(max_late, self.max_slack_timesteps))

            # --- filter delay slots by cutoff ---
            slots = []
            for s in range(max_late + 1):
                dep_t = t_orig_depart + s
                if dep_t <= cutoff_t:
                    slots.append(s)

            depart_slots[(boat, i)] = slots

            slot_vars = []
            for s in slots:
                v = model.addVar(
                    vtype="B",
                    name=f"depart_{boat}_trip{i+1}_s{s}",

```

```

    )
    depart_at[(boat, i, s)] = v
    slot_vars.append(v)

    # High priority: branch on departures early
    model.chgVarBranchPriority(v, 100)

# At most one departure slot
if slot_vars:
    model.addCons(
        quicksum(slot_vars) <= 1,
        name=f"one_depart_{boat}_trip{i+1}",
    )

    # SOS1 for strong branching
    if len(slot_vars) > 1:
        model.addConsSOS1(
            slot_vars,
            weights=[float(s) for s in slots],
            name=f"sos1_depart_{boat}_trip{i+1}",
        )

# did_depart indicator
did_depart[(boat, i)] = model.addVar(
    vtype="B",
    name=f"did_depart_{boat}_trip{i+1}",
)

if slot_vars:
    model.addCons(
        did_depart[(boat, i)] == quicksum(slot_vars),
        name=f"did_depart_link_{boat}_trip{i+1}",
    )
else:
    # No feasible slot → forced skip
    model.addCons(
        did_depart[(boat, i)] == 0,
        name=f"did_depart_forced_skip_{boat}_trip{i+1}",
    )

# fully_ready = on-time departure (slot 0)
if 0 in slots:
    fully_ready[(boat, i)] = depart_at[(boat, i, 0)]

# --- at-sea variables ---
all_sea_times = set()
for s in slots:
    dep_t = t_orig_depart + s
    for dt in range(dur):
        t_sea = dep_t + dt
        if t_sea < T:
            all_sea_times.add(t_sea)

for t in all_sea_times:
    at_sea[(boat, i, t)] = model.addVar(
        lb=0,
        ub=1,
        vtype="C",
        name=f"sea_{boat}_trip{i+1}_t{t}",
    )

for t in all_sea_times:
    contributing_slots = [
        s
        for s in slots
        if t_orig_depart + s <= t < t_orig_depart + s + dur
    ]
    model.addCons(

```

```

        at_sea[(boat, i, t)]
        == quicksum(depart_at[(boat, i, s)] for s in contributing_slots),
        name=f"sea_link_{boat}_trip{i+1}_t{t}",
    )

    return {
        "depart_at": depart_at,
        "depart_slots": depart_slots,
        "at_sea": at_sea,
        "fully_ready": fully_ready,
        "did_depart": did_depart,
    }

def _add_boat_away_variables(self, model, timesteps, trip_events, at_sea):
    """Aggregate at_sea across trips -> single boat_away per (boat, t).

    One compact no-charge constraint per timestep (not per slot).
    """
    boat_away = {}
    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])
        for t in timesteps:
            sea_vars = [
                at_sea[(boat, i, t)]
                for i in range(len(boat_trips))
                if (boat, i, t) in at_sea
            ]
            if sea_vars:
                boat_away[(boat, t)] = model.addVar(
                    lb=0, ub=1, vtype="C", name=f"away_{boat}_t{t}"
                )
                model.addCons(
                    boat_away[(boat, t)] == quicksum(sea_vars),
                    name=f"away_link_{boat}_t{t}",
                )
    return boat_away

def _add_drain_variables(self, model, T, trip_events, departure_ctx):
    """Return-drain with explicit continuous variables.

    Keeping drain_at as explicit variables (rather than inlining
    energy_req * depart_at) gives the LP relaxation tighter bounds
    and avoids pseudo-bilinear structure in SOC dynamics.
    """
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]
    drain_at = {}

    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])
        for i, (t_deadline, energy_req, dur) in enumerate(boat_trips):
            t_orig_depart = t_deadline + 1
            slots = depart_slots[(boat, i)]

            for s in slots:
                t_return = t_orig_depart + s + dur
                if t_return >= T:
                    continue

                if (boat, i, t_return) not in drain_at:
                    drain_at[(boat, i, t_return)] = model.addVar(
                        lb=0,
                        ub=energy_req,
                        name=f"drain_{boat}_trip{i+1}_t{t_return}",
                    )

            d = drain_at[(boat, i, t_return)]

```

```

        da = depart_at[(boat, i, s)]

        model.addCons(
            d <= energy_req * da,
            name=f"drain_ub_{boat}_trip{i+1}_s{s}",
        )
        model.addCons(
            d >= energy_req * da,
            name=f"drain_lb_{boat}_trip{i+1}_s{s}",
        )

    return drain_at

def _add_soc_variables_and_dynamics(
    self,
    model,
    timesteps,
    trip_events,
    charger_to_boat,
    boat_objects,
    charger_objects,
    charger_power,
    boat_away,
    drain_at,
):
    """SOC tracking per boat with charging, draining, and no-charge-while-away."""
    soc = {}

    for charger in self.port.chargers:
        charger_name = charger.name
        boat = charger_to_boat[charger_name]
        battery_cap = boat_objects[boat].battery_capacity
        initial_soc_kwh = boat_objects[boat].soc * battery_cap
        charger_eff = charger_objects[charger_name].efficiency

        soc[boat] = {
            t: model.addVar(
                lb=0,
                ub=battery_cap * 0.99,
                name=f"soc_{boat}_{t}",
            )
            for t in timesteps
        }

        for t in timesteps:
            # One no-charge constraint per timestep (via aggregated boat_away)
            if (boat, t) in boat_away:
                model.addCons(
                    charger_power[charger_name][t]
                    <= charger.max_power * (1 - boat_away[(boat, t)]),
                    name=f"no_charge_away_{boat}_t{t}",
                )

            soc_prev = soc[boat][t - 1] if t > 0 else initial_soc_kwh
            charge_energy = (
                charger_power[charger_name][t] * charger_eff * self.timestep_hours
            )

            boat_trips = trip_events.get(boat, [])
            drain_terms = [
                drain_at[(boat, i, t)]
                for i in range(len(boat_trips))
                if (boat, i, t) in drain_at
            ]
            total_drain = quicksum(drain_terms) if drain_terms else 0.0

            model.addCons(
                soc[boat][t] == soc_prev + charge_energy - total_drain,

```



```

        name=f"soc_dyn_{boat}_{t}",
    )

    return soc

# ----- #
# Departure SOC constraints (tight big-M) #
# ----- #

def _add_departure_soc_constraints(self, model, T, trip_events, soc, departure_ctx):
    """SOC >= energy_req at actual departure.

    Uses tight Big-M = energy_req (not battery_capacity).
    When depart_at=0 the constraint relaxes to soc >= 0, which is
    always satisfied by the variable bound. This is the tightest
    possible big-M for this formulation.
    """
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]

    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])

        for i, (t_deadline, energy_req, dur) in enumerate(boat_trips):
            t_orig_depart = t_deadline + 1
            slots = depart_slots[(boat, i)]
            # Tight big-M: when depart_at=0, RHS becomes
            # energy_req - energy_req = 0, and soc >= 0 is the var bound.
            big_m = energy_req

            for s in slots:
                t_dl = t_orig_depart + s - 1
                if t_dl < 0 or t_dl >= T:
                    continue
                model.addCons(
                    soc[boat][t_dl]
                    >= energy_req - big_m * (1 - depart_at[(boat, i, s)]),
                    name=f"soc_req_{boat}_trip{i+1}_s{s}",
                )

# ----- #
# Symmetry breaking (strengthened) #
# ----- #

def _add_symmetry_breaking(self, model, trip_events, boat_list, departure_ctx):
    """Strong symmetry breaking across identical boats.

    For each trip index, boats are ordered: if boat k hasn't departed
    by slot s, neither can boat k+1. This covers all delay slots,
    not just on-time (slot 0).

    Cumulative form: sum(depart_at[curr, 0..s]) <= sum(depart_at[prev, 0..s])
    """
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]

    max_trips_per_boat = max(
        (len(trip_events.get(b, [])) for b in boat_list), default=0
    )

    for i in range(max_trips_per_boat):
        for k in range(1, len(boat_list)):
            b_prev = boat_list[k - 1]
            b_curr = boat_list[k]

            if (b_prev, i) not in depart_slots or (b_curr, i) not in depart_slots:
                continue
            slots_prev = depart_slots[(b_prev, i)]

```

```

        slots_curr = depart_slots[(b_curr, i)]

        common_slots = min(len(slots_prev), len(slots_curr))
        for s in range(common_slots):
            cum_prev = quicksum(
                depart_at[(b_prev, i, ss)] for ss in slots_prev[: s + 1]
            )
            cum_curr = quicksum(
                depart_at[(b_curr, i, ss)] for ss in slots_curr[: s + 1]
            )
            model.addCons(
                cum_curr <= cum_prev,
                name=f"sym_{b_curr}_trip{i+1}_s{s}",
            )

# ----- #
# MIP warm-start #
# ----- #

def _add_warm_start(self, model, trip_events, departure_ctx):
    """Provide a greedy initial solution: all boats depart on time (slot 0).

    This gives the solver a feasible starting point so it can prune
    the search tree immediately rather than spending time finding
    the first feasible solution.
    """
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]
    did_depart = departure_ctx["did_depart"]

    sol = model.createSol()

    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])
        for i in range(len(boat_trips)):
            slots = depart_slots.get((boat, i), [])
            for s in slots:
                model.setSolVal(
                    sol, depart_at[(boat, i, s)], 1.0 if s == 0 else 0.0
                )
            if (boat, i) in did_depart:
                model.setSolVal(sol, did_depart[(boat, i)], 1.0)

    # trySol lets SCIP complete continuous variables and check feasibility
    accepted = model.trySol(sol)
    if accepted:
        print("    Warm-start: on-time solution accepted")
    else:
        print(
            "    Warm-start: on-time solution infeasible, "
            "solver starts from scratch"
        )

# ----- #
# Tariffs & objective #
# ----- #

def _compute_tariff_prices(self, forecast_date, timesteps):
    return {
        t: self.port.get_tariff_price(
            forecast_date + timedelta(seconds=t * self.timestep_seconds)
        )
        for t in timesteps
    }

def _set_objective(
    self,
    model,

```

```

timesteps,
grid_import,
tariff_price,
boat_list,
trip_events,
departure_ctx,
bess_soc,
):
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]
    did_depart = departure_ctx["did_depart"]

    # ----- Energy cost (only grid import carries a tariff) -----
    energy_cost = quicksum(
        grid_import[t] * tariff_price[t] * self.timestep_hours for t in timesteps
    )

    # ----- Decaying reward for on-time departure -----
    n_boats = len(boat_list)
    base_reward = 1000.0
    ready_reward_terms = []

    for boat_idx, boat in enumerate(boat_list):
        boat_trips = trip_events.get(boat, [])
        tie_break = (n_boats - boat_idx) * 0.1

        for i in range(len(boat_trips)):
            slots = depart_slots.get((boat, i), [])
            for s in slots:
                reward_at_s = (
                    base_reward * (self.deadline_decay_factor**s) + tie_break
                )
                ready_reward_terms.append(reward_at_s * depart_at[(boat, i, s)])

    ready_reward = quicksum(ready_reward_terms) if ready_reward_terms else 0

    # ----- Missed-trip penalty -----
    missed_penalty_terms = []
    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])
        for i in range(len(boat_trips)):
            if (boat, i) in did_depart:
                missed_penalty_terms.append(2000 * (1 - did_depart[(boat, i)]))

    missed_trip_penalty = (
        quicksum(missed_penalty_terms) if missed_penalty_terms else 0
    )

    # ----- BESS end-of-day SOC penalty -----
    bess_end_penalty = 0
    if self.port.bess_systems and self.bess_end_soc_penalty > 0:
        T_last = timesteps[-1]
        bess_shortfall_terms = []
        for bess in self.port.bess_systems:
            target_kwh = bess.current_soc * bess.capacity
            shortfall = model.addVar(lb=0, name=f"bess_end_short_{bess.name}")
            model.addCons(
                shortfall >= target_kwh - bess_soc[bess.name][T_last],
                name=f"bess_end_target_{bess.name}",
            )
            bess_shortfall_terms.append(self.bess_end_soc_penalty * shortfall)
        bess_end_penalty = quicksum(bess_shortfall_terms)

    model.setObjective(
        -ready_reward + missed_trip_penalty + energy_cost + bess_end_penalty,
        "minimize",
    )
# ----- #

```

```

# Post-solve logging #
# ----- #

def _log_departure_decisions(
    self, model, forecast_date, trip_events, departure_ctx
):
    depart_at = departure_ctx["depart_at"]
    depart_slots = departure_ctx["depart_slots"]

    for boat in self.boat_charger_assignments:
        boat_trips = trip_events.get(boat, [])
        for i, (t_deadline, energy_req, dur) in enumerate(boat_trips):
            slots = depart_slots.get((boat, i), [])
            departed = False
            for s in slots:
                if model.getVal(depart_at[(boat, i, s)]) > 0.5:
                    t_actual = t_deadline + 1 + s
                    ts = forecast_date + timedelta(
                        seconds=t_actual * self.timestep_seconds
                    )
                    if s == 0:
                        print(f"    {boat} trip {i+1}: ON TIME at {ts}")
                    else:
                        print(
                            f"    {boat} trip {i+1}: DELAYED +{s} steps "
                            f"({s * self.timestep_seconds / 60:.0f} min), "
                            f"departs {ts}"
                        )
                    departed = True
                    break
            if not departed:
                print(f"    {boat} trip {i+1}: SKIPPED (infeasible)")

def _log_bess_summary(
    self, model, timesteps, bess_charge, bess_discharge, bess_soc
):
    if not self.port.bess_systems:
        return

    for bess in self.port.bess_systems:
        total_charged = sum(
            model.getVal(bess_charge[bess.name][t]) * self.timestep_hours
            for t in timesteps
        )
        total_discharged = sum(
            model.getVal(bess_discharge[bess.name][t]) * self.timestep_hours
            for t in timesteps
        )
        soc_start = bess.current_soc * bess.capacity
        soc_end = model.getVal(bess_soc[bess.name][timesteps[-1]])
        peak_discharge = max(
            model.getVal(bess_discharge[bess.name][t]) for t in timesteps
        )

        print(f"    ■ {bess.name}:")
        print(
            f"        Charged:    {total_charged:7.2f} kWh    "
            f"Discharged: {total_discharged:7.2f} kWh"
        )
        print(
            f"        SOC start:  {soc_start:7.2f} kWh    "
            f"SOC end:    {soc_end:7.2f} kWh"
        )
        print(f"        Peak discharge power: {peak_discharge:.2f} kW")

def _log_pv_summary(self, model, timesteps, pv_forecast, pv_export):
    if not self.port.pv_systems:
        return

```

```

total_pv = sum(pv_forecast[t] * self.timestep_hours for t in timesteps)
total_export = sum(
    model.getVal(pv_export[t]) * self.timestep_hours for t in timesteps
)
total_used = total_pv - total_export
pct_used = (total_used / total_pv * 100) if total_pv > 0 else 0

print(f"    ■■ PV:")
print(
    f"        Produced: {total_pv:7.2f} kWh    "
    f"Used: {total_used:7.2f} kWh ({pct_used:.1f}%)"
)
print(f"        Exported / curtailed: {total_export:7.2f} kWh")

# ----- #
# Post-solve extraction                                     #
# ----- #

def _extract_results(
    self,
    model,
    forecast_date,
    timesteps,
    charger_power,
    grid_import,
    tariff_price,
    status,
):
    charger_schedules = {c.name: [] for c in self.port.chargers}
    peak_power = 0.0
    total_energy = 0.0
    total_cost_val = 0.0

    for t in timesteps:
        ts = forecast_date + timedelta(seconds=t * self.timestep_seconds)
        power_t = 0.0
        for c in self.port.chargers:
            p = max(0, model.getVal(charger_power[c.name][t]))
            charger_schedules[c.name].append((ts, p))
            power_t += p
        peak_power = max(peak_power, power_t)
        total_energy += power_t * self.timestep_hours
        total_cost_val += (
            model.getVal(grid_import[t]) * tariff_price[t] * self.timestep_hours
        )

    boat_schedules = {}
    for boat, idx in self.boat_charger_assignments.items():
        charger_name = self.port.chargers[idx].name
        boat_schedules[boat] = list(charger_schedules[charger_name])

    return BaseOptimizationResult(
        status=status,
        charger_schedules=charger_schedules,
        boat_schedules=boat_schedules,
        peak_power_kw=peak_power,
        total_energy_kwh=total_energy,
        total_cost_val=total_cost_val,
    )

# ----- #
# Trip extraction                                           #
# ----- #

def _extract_trip_events(self, energy_forecasts):
    trips = defaultdict(list)
    T = len(energy_forecasts)
    for boat in self.port.boats:

```

```

t = 0
while t < T:
    prev_avail = (
        energy_forecasts[t - 1].boat_available.get(boat.name, 1)
        if t > 0
        else 1
    )
    curr_avail = energy_forecasts[t].boat_available.get(boat.name, 1)

    if prev_avail == 1 and curr_avail == 0:
        start_t = t
        dur = 0
        while (
            t < T
            and energy_forecasts[t].boat_available.get(boat.name, 1) == 0
        ):
            dur += 1
            t += 1

        deadline_t = start_t - 1
        energy_req = energy_forecasts[
            deadline_t
        ].boat_required_energy_kwh.get(boat.name, 0.0)

        trips[boat.name].append((deadline_t, energy_req, dur))
    else:
        t += 1

return trips

# ----- #
# Database persistence #
# ----- #

def save_schedules_to_db(self, result: BaseOptimizationResult) -> None:
    schedules = []
    power_setpoint_met = self.db_manager.get_metric_id("power_setpoint")

    for charger_name, schedule in result.charger_schedules.items():
        charger_src = self.db_manager.get_or_create_source(charger_name, "charger")
        for timestamp, power in schedule:
            ts_str = timestamp.strftime("%Y-%m-%d %H:%M:%S")
            schedules.append((ts_str, charger_src, power_setpoint_met, str(power)))

    if schedules:
        self.db_manager.save_records_batch("scheduling", schedules)
        print(f"    Saved {len(schedules)} charger schedule entries")

def _save_bess_schedules_to_db(
    self, model, forecast_date, timesteps, bess_charge, bess_discharge
):
    if not self.port.bess_systems:
        return

    schedules = []
    power_setpoint_met = self.db_manager.get_metric_id("power_setpoint")

    for bess in self.port.bess_systems:
        bess_src = self.db_manager.get_or_create_source(bess.name, "bess")
        for t in timesteps:
            ts = forecast_date + timedelta(seconds=t * self.timestep_seconds)
            ts_str = ts.strftime("%Y-%m-%d %H:%M:%S")
            chg = model.getVal(bess_charge[bess.name][t])
            dis = model.getVal(bess_discharge[bess.name][t])
            net_power = dis - chg
            schedules.append((ts_str, bess_src, power_setpoint_met, str(net_power)))

    if schedules:

```

```
self.db_manager.save_records_batch("scheduling", schedules)
print(f"    Saved {len(schedules)} BESS schedule entries")
```

## simulation/\_\_init\_\_.py

---

```
"""Simulation engine for the electric port simulator."""  
from .engine import SimulationEngine  
__all__ = ["SimulationEngine"]
```



# simulation/engine.py

---

```
"""Core simulation engine for the electric port simulator."""

import time
from datetime import datetime, timedelta, timezone
from typing import Dict, List, Optional, Union

from config import Settings, SimulationMode
from database import DatabaseManager
from forecasting import PortForecaster
from models import BoatState, ChargerState, Port, Trip
from optimization import BaseOptimizer
from simulation.trip_manager import TripManager
from weather import OpenMeteoClient

class SimulationEngine:
    """Main simulation engine for the electric port."""

    def __init__(
        self,
        port: Port,
        settings: Settings,
        db_manager: DatabaseManager,
        start_date: Optional[Union[datetime, str]] = None,
        days: int = 1,
        trips_directory: str = "assets/trips",
    ):
        """
        Initialize the simulation engine.

        Args:
            port: Port instance (contains boats and chargers)
            settings: Simulation settings
            db_manager: Database manager
            start_date: Simulation start date as datetime or string "YYYY-MM-DD" (default: today at midnight)
            days: Number of days to simulate (max 7)
            trips_directory: Directory containing trip CSV files
        """
        self.port = port
        self.settings = settings
        self.db_manager = db_manager

        # Set start date to today at midnight UTC if not provided
        if start_date is None:
            now = datetime.now(timezone.utc)
            self.start_date = datetime(now.year, now.month, now.day, 0, 0, 0)
        elif isinstance(start_date, str):
            # Handle string dates (e.g., "2025-09-01")
            self.start_date = datetime.strptime(start_date, "%Y-%m-%d")
        else:
            self.start_date = start_date

        # Cap days at 7
        self.days = min(days, 7)

        # Calculate total simulation time in seconds
        self.total_duration = self.days * 24 * 3600

        # Current simulation datetime
        self.current_datetime = self.start_date

        # Track boat-charger assignments
```

```

self.boat_charger_map = {} # {boat_name: charger_name}

# Trip schedule from settings: (start_hour, slot_number)
self.trip_schedule = self.settings.trip_schedule

# Initialize trip manager
print(f"\nLoading trips from {trips_directory}...")
self.trip_manager = TripManager(trips_directory)

# Track active trips: {boat_name: (trip, start_datetime, elapsed_seconds)}
self.active_trips: Dict[str, tuple[Trip, datetime, float]] = {}

# Track delayed trips (boats waiting for sufficient SOC): {boat_name: trip}
self.delayed_trips: Dict[str, Trip] = {}

# Track last date we assigned trips (to assign at midnight)
self.last_assignment_date: Optional[str] = None

# Track last date we fetched weather forecast
self.last_weather_fetch_date: Optional[str] = None

# Track last date we generated energy forecast
self.last_energy_forecast_date: Optional[str] = None

# Initialize forecaster
self.forecaster = PortForecaster(
    port, db_manager, settings.timestep, settings.trip_schedule
)

# Initialize optimizer (if enabled)
self.optimizer = None
self.use_optimizer = settings.use_optimizer
# Pre-assign boats to chargers 1:1 (required when optimizer is enabled)
self.boat_charger_assignments = {} # {boat_name: charger_index}
if self.use_optimizer:
    num_boats = len(self.port.boats)
    num_chargers = len(self.port.chargers)
    if num_boats != num_chargers:
        raise ValueError(
            f"Optimizer requires n_boats == n_chargers, "
            f"got {num_boats} boats and {num_chargers} chargers"
        )
    # Fixed 1:1 mapping: boat i -> charger i
    for i, boat in enumerate(self.port.boats):
        self.boat_charger_assignments[boat.name] = i
    self.optimizer = BaseOptimizer(
        port,
        db_manager,
        settings.timestep,
        self.boat_charger_assignments,
    )

    print(
        f"    Pre-assignments: {'', '.join(f'{b} → {self.port.chargers[c].name}' for b, c in self.boa"
    )

# Store latest forecasts for optimizer
self.latest_energy_forecasts = []

# Track boats with energy shortfalls (for priority charging)
self.boats_with_shortfalls = set() # {boat_name}

# Initialize weather client and fetch forecast if PV systems present
self.weather_client = None
self.weather_forecast = {} # {timestamp_str: {metric: value}}
self.forecast_loaded = False

if self.port.pv_systems:

```

```

        print(
            f"\nInitializing weather data for {len(self.port.pv_systems)} PV system(s)..."
        )
        self.weather_client = OpenMeteoClient(self.port.lat, self.port.lon)
        self._load_weather_forecast()
        # Mark initial fetch date
        self.last_weather_fetch_date = self.start_date.strftime("%Y-%m-%d")

def run(self):
    """Run the simulation based on the configured mode."""
    print("\n" + "=" * 60)
    print("Starting Simulation")
    print("=" * 60)
    print(f"Port: {self.port.name}")
    print(f"Start: {self.start_date.strftime('%Y-%m-%d %H:%M:%S')} UTC")
    print(f"Duration: {self.days} day(s)")
    print(f"Timestep: {self.settings.timestep}s")
    print(f"Mode: {self.settings.mode.value}")
    print(f"Boats: {len(self.port.boats)}")
    print(f"Chargers: {len(self.port.chargers)}")
    print("=" * 60 + "\n")

    if self.settings.mode == SimulationMode.BATCH:
        self._run_batch()
    else:
        self._run_realtime()

def _run_batch(self):
    """Run simulation in batch mode (all timesteps at once)."""
    timestep_count = int(self.total_duration / self.settings.timestep)

    print(f"Running {timestep_count} timesteps...\n")

    for step in range(timestep_count):
        self._simulate_timestep()

        # Progress indicator every 100 steps or at midnight
        if step % 100 == 0 or self.current_datetime.hour == 0:
            print(
                f"[{step}/{timestep_count}] {self.current_datetime.strftime('%Y-%m-%d %H:%M:%S')} UTC"
            )

        # Advance time
        self.current_datetime += timedelta(seconds=self.settings.timestep)

    print(f"\n✓ Simulation completed: {timestep_count} timesteps")

def _run_realtime(self):
    """Run simulation in real-time mode."""
    print("Running in real-time mode...\n")

    timestep_count = int(self.total_duration / self.settings.timestep)
    start_real_time = time.time()

    for step in range(timestep_count):
        step_start = time.time()

        self._simulate_timestep()

        # Print status
        if step % 10 == 0:
            print(
                f"[{step}/{timestep_count}] {self.current_datetime.strftime('%Y-%m-%d %H:%M:%S')} UTC"
            )

        # Advance time
        self.current_datetime += timedelta(seconds=self.settings.timestep)
        # Wait for the timestep duration

```

```

        elapsed = time.time() - step_start
        sleep_time = self.settings.timestep - elapsed
        if sleep_time > 0:
            time.sleep(sleep_time)

    total_real_time = time.time() - start_real_time
    print(
        f"\n✓ Simulation completed: {timestep_count} timesteps in {total_real_time:.1f}s"
    )

def _simulate_timestep(self):
    """Simulate a single timestep."""
    # Check if it's midnight (00:00)
    is_midnight = (
        self.current_datetime.hour == 0 and self.current_datetime.minute == 0
    )

    if is_midnight:
        # 0. Fetch weather forecast daily at midnight (00:00)
        self._fetch_weather()
        # 1. Assign daily trips at midnight (00:00):
        self._assign_daily_trips()
        # 2. Generate energy forecast at midnight (00:00):
        self._generate_energy_forecast()

    # 3. Check and handle trip schedules
    self._handle_trips()

    # 4. Update PV production based on weather
    self._update_pv_production()

    # 5. Check which boats need charging and assign to chargers
    self._assign_boats_to_chargers()

    # 6. Update BESS (after charger assignment to see correct load)
    self._update_bess()

    # 7. Update charging for boats
    self._update_charging()

    # 8. Save measurements to database
    self._save_measurements()

def _fetch_weather(self):
    """Fetch weather forecast daily at midnight (00:00)."""
    if not self.weather_client:
        return

    current_date_str = self.current_datetime.strftime("%Y-%m-%d")

    # Check if we haven't fetched for this date yet
    if self.last_weather_fetch_date != current_date_str:
        print(f"\n ■■ Fetching weather forecast for {current_date_str}")

        # Calculate remaining days in simulation from current date
        days_remaining = (
            self.start_date + timedelta(days=self.days) - self.current_datetime
        ).days
        days_to_fetch = min(days_remaining, 7) # OpenMeteo provides up to 7 days

        if days_to_fetch > 0:
            # Fetch forecast from current date
            self._load_weather_forecast(
                start_from=self.current_datetime, days=days_to_fetch
            )
            self.last_weather_fetch_date = current_date_str
            print(f" ✓ Weather data updated for next {days_to_fetch} day(s)")
        print()

```

```

def _generate_energy_forecast(self):
    """Generate energy consumption/production forecast at midnight (00:00)."""
    current_date_str = self.current_datetime.strftime("%Y-%m-%d")

    # Check if we haven't forecasted for this date yet
    if self.last_energy_forecast_date != current_date_str:
        print(f" ■ Generating energy forecast for {current_date_str}")

        # Get trip assignments for today
        trip_assignments = {}
        for boat in self.port.boats:
            trips = self.trip_manager.get_trips_for_date(
                boat.name, self.current_datetime
            )
            trip_assignments[boat.name] = trips

        # Generate 24-hour forecast
        forecasts = self.forecaster.generate_daily_forecast(
            self.current_datetime, trip_assignments
        )

        # Store forecasts for optimizer
        self.latest_energy_forecasts = forecasts

        # Save to database
        self.forecaster.save_forecasts_to_db(forecasts, forecast_type="port_energy")

        # Print summary
        self.forecaster.print_forecast_summary(forecasts)

        # Run optimization if enabled
        if self.use_optimizer and self.optimizer:
            self._run_optimization()

        self.last_energy_forecast_date = current_date_str
        print()

def _run_optimization(self):
    """
    Run optimization to create optimal schedules for chargers and BESS.
    """
    print(f" ■ Running optimization...")

    # Run optimization
    result = self.optimizer.optimize_daily_schedule(
        self.current_datetime, self.latest_energy_forecasts
    )

    # Save schedules to database
    self.optimizer.save_schedules_to_db(result)

    print(f" ✓ Schedules saved to database")

def _handle_energy_shortfalls(self, result, trip_assignments: Dict[str, List]):
    """
    Handle cases where optimizer cannot meet energy requirements (graceful degradation).

    When energy shortfalls are detected, this function:
    1. Logs warnings about which boats are affected
    2. Attempts to maximize charging for affected boats
    3. Updates schedules to use maximum available power

    Args:
        result: OptimizationResult with energy_shortfalls
        trip_assignments: Trip assignments per boat
    """
    if not result.energy_shortfalls:
        return

```

```

print(
    f"      ■■ Handling energy shortfalls for {len(result.energy_shortfalls)} boat(s)..."
)

# Update tracking of boats with shortfalls
self.boats_with_shortfalls = set(result.energy_shortfalls.keys())

# For each boat with shortfall, try to maximize charging
for boat_name, shortfall_kwh in result.energy_shortfalls.items():
    boat = next(b for b in self.port.boats if b.name == boat_name)
    shortfall_pct = (shortfall_kwh / boat.battery_capacity) * 100

    print(
        f"      {boat_name}: {shortfall_kwh:.2f} kWh shortfall ({shortfall_pct:.1f}% of battery)"
    )

# Calculate remaining trips and energy needed
trips = trip_assignments.get(boat_name, [])
if trips:
    total_trip_energy = sum(
        trip.estimate_energy_required(boat.k) for trip in trips
    )
    current_energy = boat.soc * boat.battery_capacity
    energy_needed = total_trip_energy + (
        boat.battery_capacity - current_energy
    )

    print(f"      Current SOC: {boat.soc:.1f}%")
    print(f"      Energy needed: {energy_needed:.2f} kWh")
    print(f"      Energy available: {current_energy:.2f} kWh")
    print(f"      Shortfall: {shortfall_kwh:.2f} kWh")

# Override schedules to use maximum power for this boat when available
# This maximizes charging priority for boats with shortfalls
self._override_schedules_for_shortfall_boat(boat_name, result)

if trips:
    print(
        f"      ■■ Warning: {len(trips)} trip(s) may be delayed or cancelled"
    )
    print(
        f"      Action: Maximizing charging priority for {boat_name}"
    )

# Note: The charger assignment logic will prioritize boats with lower SOC,
# and boats with shortfalls are now marked for maximum power charging.

def _assign_daily_trips(self):
    """Assign trips for all boats at midnight (00:00)."""
    current_date_str = self.current_datetime.strftime("%Y-%m-%d")

    # Check if we haven't assigned for this date yet
    if self.last_assignment_date != current_date_str:
        # Clear any delayed trips from previous day
        if self.delayed_trips:
            print(
                f"      ■■ Clearing {len(self.delayed_trips)} delayed trip(s) from previous day"
            )
            self.delayed_trips.clear()

        print(f"      ■ Assigning trips for {current_date_str}")
        weekday_name = self.current_datetime.strftime("%A")
        print(f"      Day: {weekday_name}")

        for boat in self.port.boats:
            trips = self.trip_manager.assign_daily_trips(
                boat.name, self.current_datetime
            )

```

```

        if trips:
            trip_names = [t.route_name for t in trips]
            print(f"        {boat.name}: {len(trips)} trip(s) - {trip_names}")
        else:
            print(f"        {boat.name}: No trips (rest day)")

    self.last_assignment_date = current_date_str
    print()

def _handle_trips(self):
    """Handle boat trips based on schedule."""
    current_hour = self.current_datetime.hour
    current_minute = self.current_datetime.minute

    for boat in self.port.boats:
        boat_name = boat.name

        # Check if boat is currently on a trip
        if boat_name in self.active_trips:
            trip, start_time, elapsed = self.active_trips[boat_name]

            # Update elapsed time
            elapsed += self.settings.timestep

            # Check if trip is complete
            if elapsed >= trip.duration:
                # Discharge for final partial timestep [elapsed - timestep, duration]
                self._discharge_boat_on_trip(boat, trip, elapsed)
                # Trip completed, return to port
                boat.state = BoatState.IDLE
                print(
                    f"    ← {boat.name} returned from {trip.route_name} at {self.current_datetime.strftime('%H:%M')}
                )
                del self.active_trips[boat_name]

            else:
                # Still on trip, discharge battery (same segment-based energy as requirement)
                self._discharge_boat_on_trip(boat, trip, elapsed)
                self.active_trips[boat_name] = (trip, start_time, elapsed)
                continue

        # Check for delayed trips (boats waiting for sufficient charge)
        if boat_name in self.delayed_trips:
            trip = self.delayed_trips[boat_name]

            # Don't start trips after 6 PM (18:00)
            if current_hour >= 18:
                # Cancel delayed trip if it's past 6 PM
                if current_hour == 18 and current_minute < (
                    self.settings.timestep / 60
                ):
                    print(
                        f"    ■ {boat.name} cancelled DELAYED {trip.route_name} - too late (after 6 PM)"
                    )
                    del self.delayed_trips[boat_name]
                    continue

            estimated_energy = trip.estimate_energy_required(boat.k)
            required_soc = estimated_energy / boat.battery_capacity

            # Check if boat now has enough charge (tolerance for float precision)
            if boat.soc >= (int(required_soc * 100) / 100):
                # Disconnect from charger if connected
                if boat_name in self.boat_charger_map:
                    charger_name = self.boat_charger_map[boat_name]
                    charger = next(
                        c for c in self.port.chargers if c.name == charger_name
                    )

```

```

        charger.state = ChargerState.IDLE
        charger.power = 0.0
        charger.connected_boat = None
        del self.boat_charger_map[boat_name]

    # Start delayed trip
    self.active_trips[boat_name] = (
        trip,
        self.current_datetime,
        0.0,
    )
    boat.state = BoatState.SAILING
    print(
        f" → {boat.name} starting DELAYED {trip.route_name} at {self.current_datetime.strftime('%H:%M')} "
        f"SOC={boat.soc:.1%} (needed {required_soc:.1%})"
    )
    del self.delayed_trips[boat_name]
    continue

# Check if it's time to start a new trip
for start_hour, slot in self.trip_schedule:
    # Start trip at the scheduled hour (check within the timestep window)
    if current_hour == start_hour and current_minute < (
        self.settings.timestep / 60
    ):
        # Get assigned trip for this slot
        trip = self.trip_manager.get_trip_for_slot(
            boat_name, self.current_datetime, slot
        )

        if trip is None:
            continue # No trip assigned for this slot

        # Estimate energy required for trip
        estimated_energy = trip.estimate_energy_required(boat.k)
        required_soc = estimated_energy / boat.battery_capacity

        # Check if boat has enough charge (tolerance for float precision)
        if boat.soc >= (int(required_soc * 100) / 100):
            # Disconnect from charger if connected
            if boat_name in self.boat_charger_map:
                charger_name = self.boat_charger_map[boat_name]
                charger = next(
                    c for c in self.port.chargers if c.name == charger_name
                )
                charger.state = ChargerState.IDLE
                charger.power = 0.0
                charger.connected_boat = None
                del self.boat_charger_map[boat_name]

            # Start trip
            self.active_trips[boat_name] = (
                trip,
                self.current_datetime,
                0.0,
            )
            boat.state = BoatState.SAILING
            print(
                f" → {boat.name} starting {trip.route_name} at {self.current_datetime.strftime('%H:%M')} "
                f"SOC={boat.soc:.1%} (need {required_soc:.1%})"
            )
        else:
            break
    # Not enough charge - delay the trip
    self.delayed_trips[boat_name] = trip
    print(
        f" ■■ {boat.name} delaying {trip.route_name} - insufficient charge "

```



```

        f"(has {boat.soc:.1%}, needs {required_soc:.1%}). Will depart when ready."
    )
    break

def _discharge_boat_on_trip(self, boat, trip: Trip, elapsed_seconds: float):
    """Discharge boat battery using the same segment-based energy as the requirement estimate."""
    # Energy for the period that just passed: [elapsed - timestep, elapsed], capped at trip end
    start_elapsed = max(0.0, elapsed_seconds - self.settings.timestep)
    end_elapsed = min(elapsed_seconds, trip.duration)
    if start_elapsed >= end_elapsed:
        return

    energy_consumed = trip.get_energy_between(start_elapsed, end_elapsed, boat.k)
    soc_decrease = energy_consumed / boat.battery_capacity
    boat.soc = max(0, boat.soc - soc_decrease)

def _assign_boats_to_chargers(self):
    """Assign boats that need charging to available chargers."""
    # Check if we're using optimizer schedules
    if self.use_optimizer:
        self._assign_boats_to_chargers_with_schedule()
    elif (
        hasattr(self.settings, "power_limit_mode")
        and self.settings.power_limit_mode
    ):
        self._assign_boats_to_chargers_power_limited()
    else:
        self._assign_boats_to_chargers_default()

def _get_current_power_usage(self) -> float:
    """Calculate current total power being used by chargers."""
    return sum(
        c.power for c in self.port.chargers if c.state == ChargerState.CHARGING
    )

def _get_available_power(self):
    used_power = self._get_current_power_usage()
    pv_production = sum(pv.current_production for pv in self.port.pv_systems)

    if self.use_optimizer:
        bess_available = 0.0
    else:
        bess_available = sum(
            bess.get_max_discharge_power_available(self.settings.timestep)
            for bess in self.port.bess_systems
        )

    return self.port.contracted_power + pv_production + bess_available - used_power

def _assign_boats_to_chargers_with_schedule(self):
    """
    Assign boats to chargers following the optimizer's CHARGER power schedules.

    Uses the pre-assigned boat→charger mapping to determine each boat's charging power.
    Each boat can only connect to ONE charger at a time (physical constraint).

    IMPORTANT: Updates self.boat_charger_map which _update_charging() uses.
    """

    now = self.current_datetime
    ts_str = now.strftime("%Y-%m-%d %H:%M:%S")

    # -----
    # Load scheduled CHARGER powers for THIS timestep
    # -----
    scheduled_charger_power = {} # charger_name -> power_kw
    power_setpoint_met = self.db_manager.get_metric_id("power_setpoint")
    for charger in self.port.chargers:

```

```

src = self.db_manager.get_or_create_source(charger.name, "charger")
rows = self.db_manager.get_records(
    table="scheduling",
    source_id=src,
    metric_id=power_setpoint_met,
    start_time=ts_str,
    end_time=ts_str,
)
scheduled_charger_power[charger.name] = (
    float(rows[0]["value"]) if rows else 0.0
)

# -----
# Derive BOAT powers from charger schedules using pre-assignments
# boat_charger_assignments: {boat_name: charger_index}
# -----
scheduled_boat_power = {}
for boat_name, charger_idx in self.boat_charger_assignments.items():
    charger_name = self.port.chargers[charger_idx].name
    scheduled_boat_power[boat_name] = scheduled_charger_power.get(
        charger_name, 0.0
    )

# DEBUG: Print scheduled powers
if now.hour < 18 and now.minute % 15 == 0:
    print(
        f"\n [DEBUG {ts_str}] Scheduled charger powers: {scheduled_charger_power}"
    )
    print(f" [DEBUG {ts_str}] Derived boat powers: {scheduled_boat_power}")

# -----
# Build current charger ↔ boat mapping from charger state (one-to-one)
# -----
charger_to_boat = {}
boat_to_charger = {}

for charger in self.port.chargers:
    if charger.connected_boat:
        charger_to_boat[charger.name] = charger.connected_boat
        boat_to_charger[charger.connected_boat] = charger.name

# DEBUG: Print current assignments
if now.hour < 18 and now.minute % 15 == 0:
    currently_charging = [
        (
            boat_name,
            charger_name,
            next(b for b in self.port.boats if b.name == boat_name).soc,
        )
        for boat_name, charger_name in boat_to_charger.items()
    ]
    print(f" [DEBUG] Currently charging: {currently_charging}")

# -----
# Force disconnect boats that are already fully charged
# -----
for boat_name, charger_name in list(boat_to_charger.items()):
    boat = next(b for b in self.port.boats if b.name == boat_name)

    if boat.soc >= 0.99:
        charger = next(c for c in self.port.chargers if c.name == charger_name)

        if now.hour < 18 and now.minute % 15 == 0:
            print(f" [DEBUG] Disconnecting {boat_name} - fully charged")

        charger.state = ChargerState.IDLE
        charger.power = 0.0
        charger.connected_boat = None

```

```

        boat.state = BoatState.IDLE

        boat_to_charger.pop(boat_name, None)
        charger_to_boat.pop(charger_name, None)
        self.boat_charger_map.pop(boat_name, None)

# -----
# Process each boat according to its scheduled power
# -----
releases_made = []
assignments_made = []

for boat in self.port.boats:
    scheduled_power = scheduled_boat_power.get(boat.name, 0.0)

    # Skip boats that are sailing
    if boat.state == BoatState.SAILING:
        continue

    # Skip fully charged boats
    if boat.soc >= 0.99:
        continue

# -----
# Case 1: Boat should NOT be charging (scheduled power = 0)
# -----
    if scheduled_power <= 0.01:
        if boat.name in boat_to_charger:
            charger_name = boat_to_charger[boat.name]
            charger = next(
                c for c in self.port.chargers if c.name == charger_name
            )

            releases_made.append((boat.name, charger_name))

            charger.state = ChargerState.IDLE
            charger.power = 0.0
            charger.connected_boat = None
            boat.state = BoatState.IDLE

            boat_to_charger.pop(boat.name, None)
            charger_to_boat.pop(charger_name, None)
            self.boat_charger_map.pop(boat.name, None)
            continue

# -----
# Case 2: Boat SHOULD be charging (scheduled power > 0)
# -----
    if boat.name in boat_to_charger:
        # Already connected - just update power
        charger_name = boat_to_charger[boat.name]
        charger = next(c for c in self.port.chargers if c.name == charger_name)

        old_power = charger.power
        new_power = min(scheduled_power, charger.max_power)
        charger.power = new_power
        charger.state = ChargerState.CHARGING
        boat.state = BoatState.CHARGING

        if (
            now.hour < 18
            and now.minute % 15 == 0
            and abs(old_power - new_power) > 0.1
        ):
            print(
                f" [DEBUG] Updated {boat.name} @ {charger.name}: {old_power:.1f}→{new_power:.1f} k
            )
    else:

```

```

        # Use pre-assigned charger (1:1 mapping)
        assigned_idx = self.boat_charger_assignments.get(boat.name)
        if assigned_idx is not None:
            assigned_charger = self.port.chargers[assigned_idx]

            # Connect boat to its pre-assigned charger
            new_power = min(scheduled_power, assigned_charger.max_power)

            assigned_charger.connected_boat = boat.name
            assigned_charger.state = ChargerState.CHARGING
            assigned_charger.power = new_power

            boat.state = BoatState.CHARGING

            charger_to_boat[assigned_charger.name] = boat.name
            boat_to_charger[boat.name] = assigned_charger.name
            self.boat_charger_map[boat.name] = assigned_charger.name

            assignments_made.append(
                (boat.name, assigned_charger.name, new_power)
            )
        else:
            if now.hour < 18 and now.minute % 15 == 0:
                print(
                    f" [DEBUG] WARNING: No pre-assignment for {boat.name} (needs {scheduled_power:.1f} kW)"
                )

    # DEBUG: Print releases and new assignments
    if now.hour < 18 and now.minute % 15 == 0:
        if releases_made:
            print(f" [DEBUG] Released: {releases_made}")
        if assignments_made:
            print(f" [DEBUG] New assignments: {assignments_made}")

    # DEBUG: Print final state
    if now.hour < 18 and now.minute % 15 == 0:
        total_power = sum(
            c.power for c in self.port.chargers if c.state == ChargerState.CHARGING
        )
        active_chargers = [
            (c.name, c.power, c.connected_boat)
            for c in self.port.chargers
            if c.state == ChargerState.CHARGING
        ]
        print(f" [DEBUG] Active chargers: {active_chargers}")
        print(
            f" [DEBUG] Total power: {total_power:.1f} kW / {self.port.contracted_power} kW contracted"
        )

def _override_schedules_for_shortfall_boat(self, boat_name: str, result):
    """
    Override schedules to maximize charging for a boat with energy shortfall.

    This updates the schedules in the database to use maximum power
    when the boat is available (not sailing).

    Args:
        boat_name: Name of boat with shortfall
        result: OptimizationResult with schedules
    """
    # Find which charger is assigned to this boat (if any)
    assigned_charger = None
    if boat_name in self.boat_charger_map:
        charger_name = self.boat_charger_map[boat_name]
        assigned_charger = next(
            c for c in self.port.chargers if c.name == charger_name
        )
    # Update schedules to use maximum power for this boat's charger

```

```

# when boat is available
if assigned_charger and assigned_charger.name in result.charger_schedules:
    updated_schedules = []
    charger_schedule = result.charger_schedules[assigned_charger.name]

    for timestamp, power in charger_schedule:
        # Check if boat is available at this timestamp
        # Find corresponding forecast
        forecast = next(
            (
                f
                for f in self.latest_energy_forecasts
                if f.timestamp == timestamp
            ),
            None,
        )

        if forecast and forecast.boat_available.get(boat_name, 0) == 1:
            # Boat is available - use maximum power
            updated_schedules.append((timestamp, assigned_charger.max_power))
        else:
            # Boat is sailing or forecast not found - keep original schedule
            updated_schedules.append((timestamp, power))

    # Update the schedule in result
    result.charger_schedules[assigned_charger.name] = updated_schedules

def _assign_boats_to_chargers_default(self):
    """
    Assign boats to chargers using first-come-first-served (FCFS) logic.

    Default behavior without optimizer:
    - First come, first served: boats with delayed trips get priority, then
      boats are served in arrival order (list order)
    - Use max power available on each charger
    - NO contracted power limit - boats charge freely at max charger power
    - This allows measuring the impact of unlimited charging on results
    """
    # Get boats that need charging (not sailing, not fully charged)
    boats_needing_charge = [
        b
        for b in self.port.boats
        if b.state != BoatState.SAILING
        and b.soc < 0.99
        and b.name not in self.boat_charger_map
    ]

    # FCFS ordering: prioritize boats with delayed trips, then maintain list order
    # (no SOC-based sorting - pure first-come-first-served)
    boats_needing_charge.sort(
        key=lambda b: (0 if b.name in self.delayed_trips else 1)
    )

    # Get available chargers
    available_chargers = [
        c for c in self.port.chargers if c.state == ChargerState.IDLE
    ]

    # Assign boats to chargers (FCFS with max power, no contracted power limit)
    for boat in boats_needing_charge:
        if not available_chargers:
            # No chargers available - boat idles waiting
            break

        charger = available_chargers.pop(0)

        # Assign boat to charger at max power (no power limit check)
        self.boat_charger_map[boat.name] = charger.name

```

```

        boat.state = BoatState.CHARGING
        charger.state = ChargerState.CHARGING
        charger.power = charger.max_power
        charger.connected_boat = boat.name

    # Log charging start
    if self.current_datetime.minute % 15 == 0:
        priority_note = (
            " (priority - delayed trip)"
            if boat.name in self.delayed_trips
            else ""
        )
        print(
            f" ■ {boat.name} started charging at {charger.name}, SOC={boat.soc:.1%}"
        )

def _assign_boats_to_chargers_power_limited(self):
    """
    Assign boats to chargers with power limiting to respect contracted power.

    Baseline behavior with power limit enforcement:
    - First come, first served: boats with delayed trips get priority, then
      boats are served in arrival order (list order)
    - Enforces contracted power limit by capping total charging power
    - Distributes available power proportionally when limit is hit
    - This provides a baseline to compare against optimizer performance
    """
    # Get boats that need charging (not sailing, not fully charged)
    boats_needing_charge = [
        b
        for b in self.port.boats
        if b.state != BoatState.SAILING
        and b.soc < 0.99
        and b.name not in self.boat_charger_map
    ]

    # FCFS ordering: prioritize boats with delayed trips, then maintain list order
    boats_needing_charge.sort(
        key=lambda b: (0 if b.name in self.delayed_trips else 1)
    )

    # Get available chargers
    available_chargers = [
        c for c in self.port.chargers if c.state == ChargerState.IDLE
    ]

    # Calculate available power (contracted power - current usage)
    current_power_used = self._get_current_power_usage()
    pv_production = sum(pv.current_production for pv in self.port.pv_systems)
    available_power = (
        self.port.contracted_power + pv_production - current_power_used
    )

    # Assign boats to chargers (FCFS with power limit enforcement)
    for boat in boats_needing_charge:
        if not available_chargers:
            # No chargers available - boat idles waiting
            break

        if available_power <= 0.1:
            # No power available - cannot charge more boats
            break

        charger = available_chargers.pop(0)

        # Assign boat to charger, respecting power limit
        # Use minimum of charger max power and available power
        charge_power = min(charger.max_power, available_power)

```

```

self.boat_charger_map[boat.name] = charger.name
boat.state = BoatState.CHARGING
charger.state = ChargerState.CHARGING
charger.power = charge_power
charger.connected_boat = boat.name

# Update available power
available_power -= charge_power

# Log charging start
if self.current_datetime.minute % 15 == 0:
    priority_note = (
        " (priority - delayed trip)"
        if boat.name in self.delayed_trips
        else ""
    )
    power_note = (
        f" [{charge_power:.1f}kW]"
        if charge_power < charger.max_power
        else ""
    )
    print(
        f" ■ {boat.name} started charging at {charger.name}{priority_note}{power_note}, SOC={b
    )

# If power limit is hit, proportionally reduce charging power for already-charging boats
# This ensures we don't exceed the limit
total_power_used = self._get_current_power_usage()
max_total_power = self.port.contracted_power + pv_production

if (
    total_power_used > max_total_power + 0.1
): # Small tolerance for floating point
    # Need to reduce power proportionally
    scale_factor = max_total_power / total_power_used

    for boat_name, charger_name in self.boat_charger_map.items():
        charger = next(c for c in self.port.chargers if c.name == charger_name)
        if charger.state == ChargerState.CHARGING:
            charger.power = charger.power * scale_factor

def _update_charging(self):
    """Update battery charge for boats that are charging."""
    for boat_name, charger_name in list(self.boat_charger_map.items()):
        boat = next(b for b in self.port.boats if b.name == boat_name)
        charger = next(c for c in self.port.chargers if c.name == charger_name)

        # Calculate energy delivered to battery in this timestep
        effective_power = charger.effective_power # kW
        energy_delivered = (effective_power * self.settings.timestep) / 3600 # kWh

        # Update boat SOC
        soc_increase = energy_delivered / boat.battery_capacity
        boat.soc = min(1.0, boat.soc + soc_increase)

        # If boat is fully charged, disconnect
        if boat.soc >= 0.99:
            print(
                f" ✓ {boat.name} fully charged at {self.current_datetime.strftime('%H:%M')}"
            )
            boat.state = BoatState.IDLE
            charger.state = ChargerState.IDLE
            charger.power = 0.0
            charger.connected_boat = None
            del self.boat_charger_map[boat_name]

def _load_weather_forecast(
    self, start_from: Optional[datetime] = None, days: Optional[int] = None

```

```

):
    """Load weather forecast from Open-Meteo and save to database.

    Args:
        start_from: Starting date for forecast (default: simulation start_date)
        days: Number of days to fetch (default: simulation days, max 7)
    """
    if not self.weather_client:
        return

    # Use provided values or defaults
    fetch_start = start_from if start_from else self.start_date
    fetch_days = min(days if days else self.days, 7) # Cap at 7 days

    print(" Fetching weather forecast from Open-Meteo...")
    forecast_data = self.weather_client.fetch_forecast(fetch_start, fetch_days)

    if not forecast_data or "timestamps" not in forecast_data:
        print(" ■ Failed to fetch weather forecast")
        return

    timestamps = forecast_data["timestamps"]
    print(f" ✓ Received {len(timestamps)} hours of forecast data")

    # Save to database
    forecasts = []
    openmeteo_src = self.db_manager.get_or_create_source("openmeteo", "weather")

    for i, ts in enumerate(timestamps):
        ts_str = ts.strftime("%Y-%m-%d %H:%M:%S")

        # Save each metric
        for metric, values in forecast_data.items():
            if metric == "timestamps":
                continue

            if i < len(values) and values[i] is not None:
                metric_id = self.db_manager.get_metric_id(metric)
                forecasts.append(
                    (ts_str, openmeteo_src, metric_id, str(float(values[i])))
                )

        # Store in memory for quick access
        if i < len(timestamps):
            self.weather_forecast[ts_str] = {}
            for metric, metric_values in forecast_data.items():
                if (
                    metric != "timestamps"
                    and i < len(metric_values)
                    and metric_values[i] is not None
                ):
                    self.weather_forecast[ts_str][metric] = float(metric_values[i])

    # Save to database
    if forecasts:
        self.db_manager.save_records_batch("forecast", forecasts)
        print(f" ✓ Saved {len(forecasts)} forecast values to database")

    self.forecast_loaded = True

def _get_weather_conditions(self, timestamp: datetime) -> Dict:
    """
    Get weather conditions for a specific timestamp.

    Args:
        timestamp: Datetime to get conditions for

    Returns:

```



```

        Dictionary with weather metrics
    """
    # Round to nearest hour for lookup
    rounded = timestamp.replace(minute=0, second=0, microsecond=0)
    ts_str = rounded.strftime("%Y-%m-%d %H:%M:%S")

    if ts_str in self.weather_forecast:
        return self.weather_forecast[ts_str]

    # Return default values if not found
    return {
        "ghi": 0.0,
        "dni": 0.0,
        "dhi": 0.0,
        "temperature": 20.0,
    }

def _update_pv_production(self):
    """Update PV production for all PV systems."""
    if not self.port.pv_systems:
        return

    # Get current weather conditions
    conditions = self._get_weather_conditions(self.current_datetime)

    # Update each PV system
    for pv in self.port.pv_systems:
        pv.calculate_production(
            ghi=conditions.get("ghi", 0.0),
            dni=conditions.get("dni", 0.0),
            dhi=conditions.get("dhi", 0.0),
            temperature=conditions.get("temperature", 20.0),
            timestamp=self.current_datetime,
        )

def _update_bess(self):
    """Update BESS charge/discharge."""
    if not self.port.bess_systems:
        return

    # Check if we're using optimizer schedules
    if self.use_optimizer:
        self._update_bess_with_schedule()
    else:
        self._update_bess_default()

def _update_bess_with_schedule(self):
    """
    Update BESS using optimized schedules from database.

    When schedule says idle or no schedule exists, opportunistically charge
    from excess PV production. Any remaining excess PV will be exported to grid.
    """
    timestamp_str = self.current_datetime.strftime("%Y-%m-%d %H:%M:%S")
    power_setpoint_met = self.db_manager.get_metric_id("power_setpoint")

    # Calculate current power flows for opportunistic charging
    pv_production = sum(pv.current_production for pv in self.port.pv_systems)
    charger_load = sum(
        c.power for c in self.port.chargers if c.state == ChargerState.CHARGING
    )
    # Excess PV = PV production not being used by chargers
    excess_pv = max(0.0, pv_production - charger_load)

    for bess in self.port.bess_systems:
        bess_src = self.db_manager.get_or_create_source(bess.name, "bess")
        schedule = self.db_manager.get_records(
            "scheduling",

```

```

        source_id=bess_src,
        metric_id=power_setpoint_met,
        start_time=timestamp_str,
        end_time=timestamp_str,
    )

    if schedule:
        # Use scheduled power (positive = discharge, negative = charge)
        scheduled_power = float(schedule[0]["value"])

        if scheduled_power > 0.1:
            # Discharge as scheduled
            bess.discharge(scheduled_power, self.settings.timestep)
        elif scheduled_power < -0.1:
            # Charge as scheduled
            bess.charge(abs(scheduled_power), self.settings.timestep)
        else:
            # Schedule says idle - opportunistically charge from excess PV
            if excess_pv > 0.1:
                max_charge = bess.get_max_charge_power_available(
                    self.settings.timestep
                )
                charge_power = min(excess_pv, max_charge)
                if charge_power > 0.1:
                    bess.charge(charge_power, self.settings.timestep)
                    excess_pv -= charge_power
                else:
                    bess.idle()
            else:
                bess.idle()
    else:
        # No schedule - opportunistically charge from excess PV
        if excess_pv > 0.1:
            max_charge = bess.get_max_charge_power_available(
                self.settings.timestep
            )
            charge_power = min(excess_pv, max_charge)
            if charge_power > 0.1:
                bess.charge(charge_power, self.settings.timestep)
                excess_pv -= charge_power
            else:
                bess.idle()
        else:
            bess.idle()

    # Note: Any remaining excess_pv after BESS charging will be tracked as
    # power_active_export in _save_measurements()

def _update_bess_default(self):
    """
    Update BESS using default control strategy.

    Behavior depends on whether PV/renewables are present:

    With PV systems:
    - Charge when PV generation exceeds load (store excess solar)
    - Discharge when load exceeds generation or for peak shaving

    Without PV systems:
    - Charge when electricity is cheaper AND boats are not charging
    - Discharge when load exceeds contracted power (peak shaving)
    """
    # Calculate current power flows
    pv_production = sum(pv.current_production for pv in self.port.pv_systems)
    charger_load = sum(
        c.power for c in self.port.chargers if c.state == ChargerState.CHARGING
    )
    has_pv = len(self.port.pv_systems) > 0
    if has_pv:

```

```

        self._update_bess_with_pv(pv_production, charger_load)
    else:
        self._update_bess_without_pv(charger_load)

def _update_bess_with_pv(self, pv_production: float, charger_load: float):
    """
    Update BESS when paired with PV/renewables (default mode, no optimizer).

    - Charge when PV generation exceeds load (store excess solar)
    - NO peak shaving (no contracted power limit in default mode)
    - Grid is assumed unlimited, so no discharge needed for peak shaving
    """
    # PV surplus = excess PV not used by chargers
    pv_surplus = pv_production - charger_load

    for bess in self.port.bess_systems:
        if pv_surplus > 0:
            # Excess PV available - charge BESS with the surplus
            max_charge = bess.get_max_charge_power_available(self.settings.timestep)
            charge_power = min(pv_surplus, max_charge)

            if charge_power > 0.1:
                bess.charge(charge_power, self.settings.timestep)
                pv_surplus -= charge_power
            else:
                bess.idle()
        else:
            # No surplus - idle (no peak shaving in default mode)
            bess.idle()

def _update_bess_without_pv(self, charger_load: float):
    """
    Update BESS when NOT paired with PV/renewables (default mode, no optimizer).

    - Charge when electricity is cheaper AND boats are not charging
    - NO peak shaving (no contracted power limit in default mode)
    - Grid is assumed unlimited for charging
    """
    # Get current tariff price
    current_price = self.port.get_tariff_price(self.current_datetime)

    # Determine if current price is "cheap":
    # - If tariff is available, use price threshold (below 0.10 €/kWh is cheap)
    # - If no tariff (price=0), use time-based heuristic (22:00 - 06:00 off-peak)
    current_hour = self.current_datetime.hour
    if current_price > 0:
        # Use actual tariff - consider cheap if below threshold
        # Typical off-peak prices are around 0.05-0.10 €/kWh
        is_cheap_period = current_price < 0.10
    else:
        # No tariff loaded - use time-based heuristic
        is_cheap_period = current_hour >= 22 or current_hour < 6

    # Check if any boats are currently charging
    boats_charging = charger_load > 0.1

    for bess in self.port.bess_systems:
        if is_cheap_period and not boats_charging:
            # Cheap electricity and no boats charging - charge BESS from grid
            # No grid capacity limit in default mode - use max charge rate
            max_charge = bess.get_max_charge_power_available(self.settings.timestep)

            if max_charge > 0.1:
                bess.charge(max_charge, self.settings.timestep)
            else:
                bess.idle()
        else:
            # Expensive period or boats charging - idle

```

```

        bess.idle()

def _save_measurements(self):
    """Save current state to database."""
    measurements = []

    # Convert current datetime to ISO format string (UTC)
    timestamp_str = self.current_datetime.strftime("%Y-%m-%d %H:%M:%S")

    # Calculate PV production (renewables)
    total_pv_production = sum(pv.current_production for pv in self.port.pv_systems)

    # Calculate BESS current state
    total_bess_power = sum(bess.current_power for bess in self.port.bess_systems)
    # Positive = charging (consuming power), Negative = discharging (providing power)
    bess_discharge = -total_bess_power if total_bess_power < 0 else 0
    bess_charge = total_bess_power if total_bess_power > 0 else 0

    # Calculate BESS usable capacity (what it COULD provide if needed)
    if self.use_optimizer:
        bess_usable_capacity = 0.0
    else:
        bess_usable_capacity = sum(
            bess.get_max_discharge_power_available(self.settings.timestep)
            for bess in self.port.bess_systems
        )

    # Calculate port metrics
    total_power_used = sum(
        c.power for c in self.port.chargers if c.state == ChargerState.CHARGING
    )

    # Available power = contracted_power + renewables + usable_bess - used_power
    # This represents the maximum power the port could still draw/use
    available_power = (
        self.port.contracted_power
        + total_pv_production
        + bess_usable_capacity
        - total_power_used
    )

    # Calculate grid import/export
    # Grid balance = (consumption + BESS charging) - (PV production + BESS discharging)
    # Positive = importing from grid, Negative = exporting to grid
    grid_balance = (
        total_power_used + bess_charge - total_pv_production - bess_discharge
    )
    power_active_import = max(0.0, grid_balance) # Power drawn from grid
    power_active_export = max(0.0, -grid_balance) # Power exported to grid

    # Get source and metric IDs
    port_src = self.db_manager.get_or_create_source(self.port.name, "port")
    power_active_consumption_met = self.db_manager.get_metric_id(
        "power_active_consumption"
    )
    power_active_production_met = self.db_manager.get_metric_id(
        "power_active_production"
    )
    power_active_import_met = self.db_manager.get_metric_id("power_active_import")
    power_active_export_met = self.db_manager.get_metric_id("power_active_export")
    bess_discharge_met = self.db_manager.get_metric_id("bess_discharge")
    bess_charge_met = self.db_manager.get_metric_id("bess_charge")
    available_power_met = self.db_manager.get_metric_id("available_power")
    contracted_power_met = self.db_manager.get_metric_id("contracted_power")
    soc_met = self.db_manager.get_metric_id("soc")
    state_met = self.db_manager.get_metric_id("state")
    power_active_met = self.db_manager.get_metric_id("power_active")
    energy_stored_met = self.db_manager.get_metric_id("energy_stored")

```

```

# Port measurements
measurements.append(
    (
        timestamp_str,
        port_src,
        power_active_consumption_met,
        str(total_power_used),
    )
)
measurements.append(
    (
        timestamp_str,
        port_src,
        power_active_production_met,
        str(total_pv_production),
    )
)
measurements.append(
    (timestamp_str, port_src, bess_discharge_met, str(bess_discharge))
)
measurements.append(
    (timestamp_str, port_src, bess_charge_met, str(bess_charge))
)
measurements.append(
    (timestamp_str, port_src, power_active_import_met, str(power_active_import))
)
measurements.append(
    (timestamp_str, port_src, power_active_export_met, str(power_active_export))
)
measurements.append(
    (timestamp_str, port_src, available_power_met, str(available_power))
)
measurements.append(
    (
        timestamp_str,
        port_src,
        contracted_power_met,
        str(self.port.contracted_power),
    )
)

# Boat measurements
for boat in self.port.boats:
    boat_src = self.db_manager.get_or_create_source(boat.name, "boat")
    measurements.append((timestamp_str, boat_src, soc_met, str(boat.soc * 100)))
    measurements.append(
        (
            timestamp_str,
            boat_src,
            state_met,
            str(float(boat.state.value == "sailing")),
        )
    )

# Calculate current motor power (only when sailing)
if boat.state == BoatState.SAILING and boat.name in self.active_trips:
    trip, _, elapsed = self.active_trips[boat.name]
    point = trip.get_point_at_elapsed_time(elapsed)
    if point:
        motor_power = boat.k * (point.speed**3)
    else:
        motor_power = 0.0
else:
    # Motor is off when charging or idle
    motor_power = 0.0

measurements.append(
    (timestamp_str, boat_src, power_active_met, str(motor_power))
)

```

```

    )

# Charger measurements
for charger in self.port.chargers:
    charger_src = self.db_manager.get_or_create_source(charger.name, "charger")
    measurements.append(
        (timestamp_str, charger_src, power_active_met, str(charger.power))
    )
    measurements.append(
        (
            timestamp_str,
            charger_src,
            state_met,
            str(float(charger.state.value == "charging")),
        )
    )
)

# PV system measurements
for pv in self.port.pv_systems:
    pv_src = self.db_manager.get_or_create_source(pv.name, "pv")
    measurements.append(
        (
            timestamp_str,
            pv_src,
            power_active_production_met,
            str(pv.current_production),
        )
    )
)

# BESS measurements
for bess in self.port.bess_systems:
    bess_src = self.db_manager.get_or_create_source(bess.name, "bess")
    measurements.append(
        (timestamp_str, bess_src, soc_met, str(bess.current_soc * 100))
    )
    measurements.append(
        (timestamp_str, bess_src, power_active_met, str(bess.current_power))
    )
    measurements.append(
        (
            timestamp_str,
            bess_src,
            energy_stored_met,
            str(bess.get_energy_stored()),
        )
    )
)

# Save to database
self.db_manager.save_records_batch("measurements", measurements)

```

# simulation/trip\_manager.py

---

```
"""Trip management for the simulation."""

import random
from pathlib import Path
from typing import Dict, List, Optional
from datetime import datetime

from models.trip import Trip

class TripManager:
    """Manages trip assignments and loading for boats."""

    def __init__(self, trips_directory: str = "assets/trips"):
        """
        Initialize the trip manager.

        Args:
            trips_directory: Directory containing trip CSV files
        """
        self.trips_directory = Path(trips_directory)
        self.available_trips: List[Trip] = []
        self._load_trips()

        # Track assigned trips per boat per day
        # {boat_name: {date_str: [trip1, trip2, ...]}}
        self.daily_assignments: Dict[str, Dict[str, List[Trip]]] = {}

    def _load_trips(self):
        """Load all available trip CSV files."""
        if not self.trips_directory.exists():
            print(f"Warning: Trips directory not found: {self.trips_directory}")
            return

        csv_files = list(self.trips_directory.glob("route_*.csv"))
        for csv_file in sorted(csv_files):
            try:
                trip = Trip(str(csv_file))
                if trip.points: # Only add trips with valid data
                    self.available_trips.append(trip)
                    print(f"  Loaded {trip.route_name}: {len(trip.points)} points, {trip.duration/3600:.2f}h")
            except Exception as e:
                print(f"  Warning: Failed to load {csv_file}: {e}")

        if self.available_trips:
            print(f"  Total trips loaded: {len(self.available_trips)}")
        else:
            print("  Warning: No trips loaded!")

    def assign_daily_trips(self, boat_name: str, current_date: datetime) -> List[Trip]:
        """
        Assign trips for a boat for the given day.

        Args:
            boat_name: Name of the boat
            current_date: Current simulation date

        Returns:
            List of trips assigned for this day
        """
        if not self.available_trips:
            return []
```

```

date_str = current_date.strftime("%Y-%m-%d")
weekday = current_date.weekday() # 0=Monday, 6=Sunday

# Check if already assigned for this day
if boat_name in self.daily_assignments:
    if date_str in self.daily_assignments[boat_name]:
        return self.daily_assignments[boat_name][date_str]

# Determine number of trips based on day of week
if weekday < 5: # Monday-Friday
    num_trips = 2
elif weekday == 5: # Saturday
    num_trips = 1
else: # Sunday
    num_trips = 0

# Randomly select trips from available trips
assigned_trips = []
if num_trips > 0:
    # Allow repetition if we have fewer routes than needed trips
    if num_trips <= len(self.available_trips):
        assigned_trips = random.sample(self.available_trips, num_trips)
    else:
        assigned_trips = random.choices(self.available_trips, k=num_trips)

# Store assignment
if boat_name not in self.daily_assignments:
    self.daily_assignments[boat_name] = {}
self.daily_assignments[boat_name][date_str] = assigned_trips

return assigned_trips

def get_trip_for_slot(
    self, boat_name: str, current_date: datetime, slot: int
) -> Optional[Trip]:
    """
    Get the assigned trip for a specific time slot.

    Args:
        boat_name: Name of the boat
        current_date: Current simulation date
        slot: Time slot (0 = morning 9AM, 1 = afternoon 2PM)

    Returns:
        The assigned trip, or None if no trip for this slot
    """
    date_str = current_date.strftime("%Y-%m-%d")

    if boat_name not in self.daily_assignments:
        return None

    if date_str not in self.daily_assignments[boat_name]:
        return None

    trips = self.daily_assignments[boat_name][date_str]
    if slot < len(trips):
        return trips[slot]

    return None

def get_trips_for_date(
    self, boat_name: str, current_date: datetime
) -> List[Trip]:
    """
    Get all assigned trips for a boat on a specific date.

    Args:
        boat_name: Name of the boat

```



```
        current_date: Date to get trips for

Returns:
    List of trips assigned for this date (may be empty)
    """
    date_str = current_date.strftime("%Y-%m-%d")

    if boat_name not in self.daily_assignments:
        return []

    if date_str not in self.daily_assignments[boat_name]:
        return []

    return self.daily_assignments[boat_name][date_str]
```

# streamlit\_app.py

---

```
from datetime import datetime
from pathlib import Path
import sqlite3

import pandas as pd
import plotly.graph_objects as go
import streamlit as st

# -----
# Config
# -----

DATA_TABLES = ("measurements", "forecast", "scheduling")

st.set_page_config(
    page_title="Port Simulation Viewer",
    page_icon="■",
    layout="wide",
)

# -----
# Data helpers
# -----

def connect(db_path: str):
    return sqlite3.connect(db_path)

@st.cache_data
def get_db_files():
    base = Path(__file__).parent
    return sorted([str(p.name) for p in base.glob("*.db")])

@st.cache_data(ttl=5)
def get_sources(db_path: str) -> pd.DataFrame:
    with connect(db_path) as conn:
        return pd.read_sql_query(
            "SELECT source_id, source_name FROM source ORDER BY source_name",
            conn,
        )

@st.cache_data(ttl=5)
def get_metrics(db_path: str) -> pd.DataFrame:
    with connect(db_path) as conn:
        return pd.read_sql_query(
            "SELECT metric_id, metric_name, unit FROM metric ORDER BY metric_name",
            conn,
        )

@st.cache_data(ttl=5)
def load_data(
    db_path: str,
    table: str,
    source_ids: list[int] | None,
    metric_ids: list[int] | None,
    start_time: str | None,
    end_time: str | None,
) -> pd.DataFrame:
```

```

with connect(db_path) as conn:
    q = f"""
        SELECT
            d.timestamp,
            s.source_name,
            m.metric_name,
            m.unit,
            d.value
        FROM {table} d
        JOIN source s ON d.source_id = s.source_id
        JOIN metric m ON d.metric_id = m.metric_id
        WHERE 1=1
    """
    params: list = []

    if source_ids:
        q += f" AND d.source_id IN ({','.join('? ' * len(source_ids))})"
        params.extend(source_ids)

    if metric_ids:
        q += f" AND d.metric_id IN ({','.join('? ' * len(metric_ids))})"
        params.extend(metric_ids)

    if start_time:
        q += " AND d.timestamp >= ?"
        params.append(start_time)

    if end_time:
        q += " AND d.timestamp <= ?"
        params.append(end_time)

    q += " ORDER BY d.timestamp"

    return pd.read_sql_query(q, conn, params=params)

# -----
# Plot
# -----

def make_plot(df: pd.DataFrame, show_legend: bool) -> go.Figure | None:
    if df.empty:
        return None

    df = df.copy()
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df["value"] = pd.to_numeric(df["value"], errors="coerce")

    fig = go.Figure()

    for (table, source, metric), g in df.groupby(
        ["table", "source_name", "metric_name"]
    ):
        unit = g["unit"].iloc[0]
        label = f"{table} • {source} • {metric}"
        if unit:
            label += f" ({unit})"

        fig.add_trace(
            go.Scatter(
                x=g["timestamp"],
                y=g["value"],
                mode="lines",
                name=label,
            )
        )
    fig.update_layout(

```

```

        xaxis_title="Time",
        yaxis_title="Value",
        hovermode="x unified",
        height=500,
        showlegend=show_legend, # ■ toggle applied here
        legend=dict(orientation="h", y=1.02),
        margin=dict(l=10, r=10, t=40, b=10),
    )

    return fig

# -----
# App
# -----

def main():
    st.title("■ Port Simulation Viewer")

    # -----
    # Session state
    # -----

    if "filters" not in st.session_state:
        st.session_state.filters = {
            "tables": [DATA_TABLES[0]],
            "sources": [],
            "metrics": [],
            "start_date": None,
            "end_date": None,
        }

    # -----
    # Sidebar
    # -----

    db_files = get_db_files()
    if not db_files:
        st.error("No .db files found")
        return

    selected_db = st.sidebar.selectbox("Database", db_files)
    page = st.sidebar.radio("Page", ["Explore", "Raw data"])

    sources = get_sources(selected_db)
    metrics = get_metrics(selected_db)

    # -----
    # EXPLORE PAGE
    # -----

    if page == "Explore":
        st.subheader("Explore")

        # ---- Controls card ----
        with st.container(border=True):
            st.markdown("### Filters")

            col1, col2, col3 = st.columns(3)

            with col1:
                selected_tables = st.multiselect(
                    "Tables",
                    DATA_TABLES,
                    default=st.session_state.filters["tables"],
                )
                st.session_state.filters["tables"] = selected_tables

```

```

with col2:
    selected_sources = st.multiselect(
        "Sources",
        options=sources["source_name"].tolist(),
        default=st.session_state.filters["sources"],
    )
    st.session_state.filters["sources"] = selected_sources

with col3:
    metric_labels = [
        f"{r.metric_name} ({r.unit})" if r.unit else r.metric_name
        for r in metrics.itertuples()
    ]
    label_to_name = dict(zip(metric_labels, metrics["metric_name"]))

    selected_metric_labels = st.multiselect(
        "Metrics",
        options=metric_labels,
        default=[
            l
            for l in metric_labels
            if label_to_name[l] in st.session_state.filters["metrics"]
        ],
    )

    selected_metrics = [label_to_name[l] for l in selected_metric_labels]
    st.session_state.filters["metrics"] = selected_metrics

col4, col5 = st.columns(2)
with col4:
    start_date = st.date_input(
        "Start date",
        value=st.session_state.filters["start_date"],
    )
    st.session_state.filters["start_date"] = start_date

with col5:
    end_date = st.date_input(
        "End date",
        value=st.session_state.filters["end_date"],
    )
    st.session_state.filters["end_date"] = end_date

# ---- Plot card ----
with st.container(border=True):
    st.markdown("### Time series")

    # ■ Legend toggle
    show_legend = st.toggle("Show legend", value=True)

    if not selected_sources and not selected_metrics:
        st.info("Select a source or a metric to begin")
        return

    source_ids = (
        sources.loc[
            sources["source_name"].isin(selected_sources), "source_id"
        ].tolist()
        if selected_sources
        else None
    )

    metric_ids = (
        metrics.loc[
            metrics["metric_name"].isin(selected_metrics), "metric_id"
        ].tolist()
        if selected_metrics
        else None
    )

```

```

    )

    start_time = (
        datetime.combine(start_date, datetime.min.time()).isoformat()
        if start_date
        else None
    )
    end_time = (
        datetime.combine(end_date, datetime.max.time()).isoformat()
        if end_date
        else None
    )

    dfs: list[pd.DataFrame] = []

    for table in selected_tables:
        df = load_data(
            selected_db,
            table,
            source_ids,
            metric_ids,
            start_time,
            end_time,
        )
        if not df.empty:
            df["table"] = table
            dfs.append(df)

    df = pd.concat(dfs, ignore_index=True) if dfs else pd.DataFrame()

    fig = make_plot(df, show_legend)

    if fig:
        st.plotly_chart(fig, use_container_width=True)

        tables_label = ", ".join(sorted(df["table"].unique()))
        sources_label = ", ".join(sorted(df["source_name"].unique()))
        metrics_label = ", ".join(sorted(df["metric_name"].unique()))

        st.caption(f"***{tables_label} – {sources_label} – {metrics_label}***")
    else:
        st.warning("No data found for the selected filters")

# -----
# RAW DATA PAGE
# -----

else:
    st.subheader("Raw data")

    with st.container(border=True):
        table = st.selectbox("Table", DATA_TABLES, format_func=str.capitalize)
        limit = st.slider("Rows", 100, 5000, 1000, step=100)

    with connect(selected_db) as conn:
        df = pd.read_sql_query(
            f"""
            SELECT
                d.timestamp,
                s.source_name,
                m.metric_name,
                m.unit,
                d.value
            FROM {table} d
            JOIN source s ON d.source_id = s.source_id
            JOIN metric m ON d.metric_id = m.metric_id
            ORDER BY d.timestamp DESC
            LIMIT ?
        """

```

```

        """
        conn,
        params=[limit],
    )

    st.dataframe(df, use_container_width=True, hide_index=True)

    st.download_button(
        "Download CSV",
        df.to_csv(index=False),
        file_name=f"{table}_raw.csv",
        mime="text/csv",
    )

if __name__ == "__main__":
    main()

```

## tests/\_\_init\_\_.py

---

```
"""Tests for the electric port simulator."""
```



# tests/port\_eletrification\_studies/10\_vessels\_no\_opt\_no\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)

```

```

port.add_boat(boat9)
port.add_boat(boat10)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.name}, {boat8.name}, {boat9.name}, {boat10.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {charger6.name}, {charger7.name}, {charger8.name}, {charger9.name}, {charger10.name}"
)
# print(f"PV: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="10_vessels_no_opt_no_der.db", # Change database file path if needed
    use_optimizer=False, # Set to False to disable optimization (SCIP)
    power_limit_mode=True,
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/10\_vessels\_opt\_d er.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=30.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=50.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)

```

```

port.add_boat(boat9)
port.add_boat(boat10)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)

port.add_pv(pv_system)
port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.name}
")
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {charger6.name}, {charger7.name}, {charger8.name}, {charger9.name}, {charger10.name}
")
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="10_vessels_opt_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```



# tests/port\_eletrification\_studies/10\_vessels\_opt\_n o\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)

```

```

port.add_boat(boat9)
port.add_boat(boat10)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.name}, {boat8.name}, {boat9.name}, {boat10.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {charger6.name}, {charger7.name}, {charger8.name}, {charger9.name}, {charger10.name}"
)
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="10_vessels_opt_no_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/20\_vessels\_no\_opt\_no\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat11 = Boat(
        name="SeaBreeze_11",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat12 = Boat(
        name="SeaBreeze_12",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat13 = Boat(
        name="SeaBreeze_13",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat14 = Boat(
        name="SeaBreeze_14",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat15 = Boat(
        name="SeaBreeze_15",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat16 = Boat(
        name="SeaBreeze_16",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat17 = Boat(
        name="SeaBreeze_17",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)

```

```

        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

boat18 = Boat(
    name="SeaBreeze_18",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat19 = Boat(
    name="SeaBreeze_19",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat20 = Boat(
    name="SeaBreeze_20",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger11 = Charger(
    name="FastCharger_K", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

```



```

charger12 = Charger(
    name="FastCharger_L", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger13 = Charger(
    name="FastCharger_M", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger14 = Charger(
    name="FastCharger_N", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger15 = Charger(
    name="FastCharger_O", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger16 = Charger(
    name="FastCharger_P", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger17 = Charger(
    name="FastCharger_Q", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger18 = Charger(
    name="FastCharger_R", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger19 = Charger(
    name="FastCharger_S", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger20 = Charger(
    name="FastCharger_T", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)
port.add_boat(boat9)
port.add_boat(boat10)
port.add_boat(boat11)
port.add_boat(boat12)
port.add_boat(boat13)
port.add_boat(boat14)

```

```

port.add_boat(boat15)
port.add_boat(boat16)
port.add_boat(boat17)
port.add_boat(boat18)
port.add_boat(boat19)
port.add_boat(boat20)
port.add_charger(charger11)
port.add_charger(charger12)
port.add_charger(charger13)
port.add_charger(charger14)
port.add_charger(charger15)
port.add_charger(charger16)
port.add_charger(charger17)
port.add_charger(charger18)
port.add_charger(charger19)
port.add_charger(charger20)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {charger6.name}"
)
# print(f"PV: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="20_vessels_no_opt_no_der.db", # Change database file path if needed
    use_optimizer=False, # Set to False to disable optimization (SCIP)
    power_limit_mode=True,
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)

```

```
print(f"✓ Results saved to: {settings.db_path}")  
print("=" * 60)
```

```
if __name__ == "__main__":  
    main()
```

# tests/port\_eletrification\_studies/20\_vessels\_opt\_d er.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat11 = Boat(
        name="SeaBreeze_11",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat12 = Boat(
        name="SeaBreeze_12",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat13 = Boat(
        name="SeaBreeze_13",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat14 = Boat(
        name="SeaBreeze_14",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat15 = Boat(
        name="SeaBreeze_15",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat16 = Boat(
        name="SeaBreeze_16",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat17 = Boat(
        name="SeaBreeze_17",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)

```

```

        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

boat18 = Boat(
    name="SeaBreeze_18",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat19 = Boat(
    name="SeaBreeze_19",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat20 = Boat(
    name="SeaBreeze_20",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger11 = Charger(
    name="FastCharger_K", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

```

```

charger12 = Charger(
    name="FastCharger_L", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger13 = Charger(
    name="FastCharger_M", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger14 = Charger(
    name="FastCharger_N", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger15 = Charger(
    name="FastCharger_O", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger16 = Charger(
    name="FastCharger_P", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger17 = Charger(
    name="FastCharger_Q", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger18 = Charger(
    name="FastCharger_R", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger19 = Charger(
    name="FastCharger_S", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger20 = Charger(
    name="FastCharger_T", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=60.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)
port.add_boat(boat9)
port.add_boat(boat10)
port.add_boat(boat11)
port.add_boat(boat12)
port.add_boat(boat13)
port.add_boat(boat14)

```



```

port.add_boat(boat15)
port.add_boat(boat16)
port.add_boat(boat17)
port.add_boat(boat18)
port.add_boat(boat19)
port.add_boat(boat20)
port.add_charger(charger11)
port.add_charger(charger12)
port.add_charger(charger13)
port.add_charger(charger14)
port.add_charger(charger15)
port.add_charger(charger16)
port.add_charger(charger17)
port.add_charger(charger18)
port.add_charger(charger19)
port.add_charger(charger20)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)

port.add_pv(pv_system)
port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.n
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {ch
)
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="20_vessels_opt_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current ti
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")

```

```
print("=" * 60)
```

```
if __name__ == "__main__":  
    main()
```

# tests/port\_eletrification\_studies/20\_vessels\_opt\_n o\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.30, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat6 = Boat(
    name="SeaBreeze_6",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat7 = Boat(
    name="SeaBreeze_7",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat8 = Boat(
    name="SeaBreeze_8",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat9 = Boat(
    name="SeaBreeze_9",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat10 = Boat(
    name="SeaBreeze_10",
    motor_power=100, # Adjust motor power (kw)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
)

```

```

        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat11 = Boat(
        name="SeaBreeze_11",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat12 = Boat(
        name="SeaBreeze_12",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat13 = Boat(
        name="SeaBreeze_13",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat14 = Boat(
        name="SeaBreeze_14",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat15 = Boat(
        name="SeaBreeze_15",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat16 = Boat(
        name="SeaBreeze_16",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat17 = Boat(
        name="SeaBreeze_17",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)

```

```

        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

boat18 = Boat(
    name="SeaBreeze_18",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat19 = Boat(
    name="SeaBreeze_19",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat20 = Boat(
    name="SeaBreeze_20",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger6 = Charger(
    name="FastCharger_F", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger7 = Charger(
    name="FastCharger_G", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger8 = Charger(
    name="FastCharger_H", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger9 = Charger(
    name="FastCharger_I", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger10 = Charger(
    name="FastCharger_J", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger11 = Charger(
    name="FastCharger_K", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

```

```

charger12 = Charger(
    name="FastCharger_L", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger13 = Charger(
    name="FastCharger_M", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger14 = Charger(
    name="FastCharger_N", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger15 = Charger(
    name="FastCharger_O", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger16 = Charger(
    name="FastCharger_P", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger17 = Charger(
    name="FastCharger_Q", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger18 = Charger(
    name="FastCharger_R", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger19 = Charger(
    name="FastCharger_S", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger20 = Charger(
    name="FastCharger_T", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)
port.add_boat(boat6)
port.add_boat(boat7)
port.add_boat(boat8)
port.add_boat(boat9)
port.add_boat(boat10)
port.add_boat(boat11)
port.add_boat(boat12)
port.add_boat(boat13)
port.add_boat(boat14)

```

```

port.add_boat(boat15)
port.add_boat(boat16)
port.add_boat(boat17)
port.add_boat(boat18)
port.add_boat(boat19)
port.add_boat(boat20)
port.add_charger(charger11)
port.add_charger(charger12)
port.add_charger(charger13)
port.add_charger(charger14)
port.add_charger(charger15)
port.add_charger(charger16)
port.add_charger(charger17)
port.add_charger(charger18)
port.add_charger(charger19)
port.add_charger(charger20)
port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
port.add_charger(charger6)
port.add_charger(charger7)
port.add_charger(charger8)
port.add_charger(charger9)
port.add_charger(charger10)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name} {boat6.name}, {boat7.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}, {charger6.name}"
)
# print(f"PV: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="20_vessels_opt_no_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")

```



```
print("=" * 60)
```

```
if __name__ == "__main__":  
    main()
```

# tests/port\_eletrification\_studies/5\_vessels\_no\_opt\_no\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)

```

```

port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}"
)
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="5_vessels_no_opt_no_der.db", # Change database file path if needed
    use_optimizer=False, # Set to False to disable optimization (SCIP)
    power_limit_mode=True,
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/5\_vessels\_opt\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```

```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=15.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=25.0, # Adjust BESS capacity (kWh)
    max_charge_power=12.0, # Adjust max charge power (kW)
    max_discharge_power=12.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)

```

```

port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)

port.add_pv(pv_system)
port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}"
)
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="5_vessels_opt_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/5\_vessels\_opt\_no\_der.py

---

```
"""Main entry point for the electric port simulator."""

from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
from config import Settings, SimulationMode
from database import DatabaseManager
from simulation import SimulationEngine

def main():
    """Main function to run the simulator."""
    print("=" * 60)
    print("Electric Port Simulator")
    print("=" * 60)

    # =====
    # CONFIGURATION SECTION - Modify these values to customize your simulation
    # =====

    # Port configuration
    # Change port name, contracted power, and location as needed
    port = Port(
        name="Funchal",
        contracted_power=80, # Adjust contracted power limit (kW)
        lat=32.64542, # Set port latitude
        lon=-16.90841, # Set port longitude
        tariff_path="assets/tariff/default_tariff.json",
    )

    # Boat configuration
    # Add/remove boats, modify boat parameters (motor_power, weight, length,
    # battery_capacity, range_speed, soc) as needed
    boat1 = Boat(
        name="SeaBreeze_1",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat2 = Boat(
        name="SeaBreeze_2",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )

    boat3 = Boat(
        name="SeaBreeze_3",
        motor_power=100, # Adjust motor power (kW)
        weight=2500, # Adjust weight (kg)
        length=8.5, # Adjust length (m)
        battery_capacity=100, # Adjust battery capacity (kWh)
        range_speed=16.0, # Adjust range speed (knots)
        soc=0.50, # Adjust initial state of charge (0.0-1.0)
    )
```



```

boat4 = Boat(
    name="SeaBreeze_4",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

boat5 = Boat(
    # Charger configuration
    name="SeaBreeze_5",
    motor_power=100, # Adjust motor power (kW)
    weight=2500, # Adjust weight (kg)
    length=8.5, # Adjust length (m)
    battery_capacity=100, # Adjust battery capacity (kWh)
    range_speed=16.0, # Adjust range speed (knots)
    soc=0.50, # Adjust initial state of charge (0.0-1.0)
)

charger1 = Charger(
    name="FastCharger_A", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger2 = Charger(
    name="FastCharger_B", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger3 = Charger(
    name="FastCharger_C", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger4 = Charger(
    name="FastCharger_D", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency
charger5 = Charger(
    name="FastCharger_E", max_power=22, efficiency=0.95
) # Adjust max_power (kW) and efficiency

# PV system configuration
# Modify PV capacity, tilt, azimuth, efficiency, or remove PV system entirely
pv_system = PV(
    name="Solar_Array_1",
    capacity=10.0, # Adjust PV capacity (kW DC)
    tilt=30.0, # Adjust panel tilt angle (degrees)
    azimuth=180.0, # Adjust panel azimuth (degrees, 180 = South-facing)
    latitude=port.lat,
    longitude=port.lon,
)

# BESS (Battery Energy Storage System) configuration
# Modify BESS parameters or remove BESS entirely
bess = BESS(
    name="Battery_Storage_1",
    capacity=100.0, # Adjust BESS capacity (kWh)
    max_charge_power=25.0, # Adjust max charge power (kW)
    max_discharge_power=25.0, # Adjust max discharge power (kW)
    efficiency=0.90, # Adjust round-trip efficiency (0.0-1.0)
    soc_min=0.10, # Adjust minimum SOC (0.0-1.0)
    soc_max=0.90, # Adjust maximum SOC (0.0-1.0)
    initial_soc=0.50, # Adjust initial SOC (0.0-1.0)
    control_strategy=BESSControlStrategy.DEFAULT, # Change control strategy if needed
)

# Add components to port
port.add_boat(boat1)
port.add_boat(boat2)
port.add_boat(boat3)
port.add_boat(boat4)
port.add_boat(boat5)

```

```

port.add_charger(charger1)
port.add_charger(charger2)
port.add_charger(charger3)
port.add_charger(charger4)
port.add_charger(charger5)
# No PV system or BESS
# port.add_pv(pv_system)
# port.add_bess(bess)

print(f"\nPort: {port}")
print(
    f"Boats: {boat1.name}, {boat2.name}, {boat3.name}, {boat4.name}, {boat5.name}"
)
print(
    f"Chargers: {charger1.name}, {charger2.name}, {charger3.name}, {charger4.name}, {charger5.name}"
)
# print(f"Pv: {pv_system}")
# print(f"BESS: {bess}")

# Simulation settings
# Modify simulation parameters (timestep, mode, db_path, use_optimizer, start_date, days)
settings = Settings(
    timestep=900, # Adjust timestep duration (seconds, 900 = 15 minutes)
    mode=SimulationMode.BATCH, # Change to SimulationMode.REALTIME for real-time simulation
    db_path="5_vessels_opt_no_der.db", # Change database file path if needed
    use_optimizer=True, # Set to False to disable optimization
)

# Initialize database
db_manager = DatabaseManager(settings.db_path)
db_manager.initialize_schema()
db_manager.initialize_default_metrics()

# Create and run simulation
# Modify start_date (None = current time) and days (simulation duration)
sim = SimulationEngine(
    port=port,
    settings=settings,
    db_manager=db_manager,
    start_date="2025-09-01", # Set start date as "YYYY-MM-DD" string, datetime, or None for current time
    days=1, # Adjust simulation duration (days)
)

sim.run()

print("\n" + "=" * 60)
print(f"✓ Results saved to: {settings.db_path}")
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/calculate\_kpis.py

---

```
"""
Calculate KPIs for Port Electrification Study

KPI 1 - Total Cost of Operation (€)
KPI 2 - Cost per Successful Trip (€/trip)
KPI 3 - Peak Power Demand (kW)
KPI 4 - Self-Consumption Rate (% of PV consumed locally)
KPI 5 - Self-Sufficiency Rate (% of demand from PV)
KPI 6 - Trip Reliability (% of trips completed)
"""

import sqlite3
import pandas as pd
import os
from pathlib import Path
from tabulate import tabulate

# =====
# CONFIGURATION
# =====

# Electricity tariff (€/kWh) - simplified time-of-use
TARIFF_PEAK = 0.25 # Peak hours (8:00-20:00)
TARIFF_OFF_PEAK = 0.12 # Off-peak hours (20:00-8:00)

# Contracted power for baseline comparison
CONTRACTED_POWER = 80 # kW

# Scenarios
SCENARIOS = [
    ("no_opt_no_der", "No Optimization, No DER", False),
    ("opt_no_der", "Optimization, No DER", False),
    ("opt_der", "Optimization + DER", True),
]

VESSEL_COUNTS = [5, 10, 20]

def get_db_path(vessels: int, scenario: str) -> str:
    """Get database path for a scenario."""
    project_root = Path(__file__).parent.parent.parent
    return str(project_root / f"{vessels}_vessels_{scenario}.db")

def get_electricity_cost(timestamp_str: str) -> float:
    """Get electricity cost based on time of day."""
    hour = int(timestamp_str.split(" ")[1].split(":")[0])
    if 8 <= hour < 20:
        return TARIFF_PEAK
    return TARIFF_OFF_PEAK

def analyze_trip_reliability(db_path: str, num_vessels: int) -> dict:
    """Analyze trip reliability from boat state data."""
    total_trips = num_vessels * 2 # 2 trips per vessel per day

    if not os.path.exists(db_path):
        return {'on_time': 0, 'delayed': 0, 'cancelled': total_trips, 'total': total_trips}

    conn = sqlite3.connect(db_path)

    # Get state metric
```

```

state_result = pd.read_sql_query(
    "SELECT metric_id FROM metric WHERE metric_name = 'state'", conn
)
if state_result.empty:
    conn.close()
    return {'on_time': 0, 'delayed': 0, 'cancelled': total_trips, 'total': total_trips}

state_id = state_result.iloc[0, 0]

# Get boat sources
boats = pd.read_sql_query(
    "SELECT source_id, source_name FROM source WHERE source_type = 'boat'", conn
)

on_time = 0
delayed = 0
cancelled = 0

for _, boat in boats.iterrows():
    query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as state
        FROM measurements
        WHERE source_id = {boat['source_id']} AND metric_id = {state_id}
        ORDER BY timestamp
    """
    states = pd.read_sql_query(query, conn)

    if states.empty:
        cancelled += 2
        continue

    states['timestamp'] = pd.to_datetime(states['timestamp'])

    # Find trip starts (state changes to 1.0 = sailing)
    trip_starts = []
    prev_state = 0
    for _, row in states.iterrows():
        if row['state'] == 1.0 and prev_state == 0.0:
            trip_starts.append(row['timestamp'])
        prev_state = row['state']

    # Analyze each trip slot
    morning_found = False
    afternoon_found = False

    for ts in trip_starts:
        hour = ts.hour
        minute = ts.minute

        if 9 <= hour < 12:
            if hour == 9 and minute < 15:
                on_time += 1
            else:
                delayed += 1
                morning_found = True
        elif 14 <= hour < 18:
            if hour == 14 and minute < 15:
                on_time += 1
            else:
                delayed += 1
                afternoon_found = True

    if not morning_found:
        cancelled += 1
    if not afternoon_found:
        cancelled += 1

conn.close()

```

```

return {
    'on_time': on_time,
    'delayed': delayed,
    'cancelled': cancelled,
    'total': total_trips,
    'reliability': ((on_time + delayed) / total_trips * 100) if total_trips > 0 else 0
}

def calculate_kpis(db_path: str, has_der: bool = False) -> dict:
    """Calculate all KPIs for a scenario."""
    if not os.path.exists(db_path):
        return None

    conn = sqlite3.connect(db_path)

    # Get port source
    port_source = pd.read_sql_query(
        "SELECT source_id FROM source WHERE source_type = 'port'", conn
    )
    if port_source.empty:
        conn.close()
        return None
    port_id = port_source.iloc[0, 0]

    # Get metrics
    metrics = pd.read_sql_query("SELECT metric_id, metric_name FROM metric", conn)
    metric_map = dict(zip(metrics['metric_name'], metrics['metric_id']))

    timestep_hours = 0.25 # 15 minutes

    # =====
    # Get consumption and import data
    # =====

    consumption_query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as power_kw
        FROM measurements
        WHERE source_id = {port_id}
        AND metric_id = {metric_map.get('power_active_consumption', -1)}
        ORDER BY timestamp
    """
    consumption_df = pd.read_sql_query(consumption_query, conn)

    if consumption_df.empty:
        conn.close()
        return None

    consumption_df['energy_kwh'] = consumption_df['power_kw'] * timestep_hours
    total_consumption_kwh = consumption_df['energy_kwh'].sum()
    peak_consumption = consumption_df['power_kw'].max()

    # Grid import
    import_query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as power_kw
        FROM measurements
        WHERE source_id = {port_id}
        AND metric_id = {metric_map.get('power_active_import', -1)}
        ORDER BY timestamp
    """
    import_df = pd.read_sql_query(import_query, conn)

    if not import_df.empty:
        import_df['energy_kwh'] = import_df['power_kw'] * timestep_hours
        import_df['cost_eur'] = import_df.apply(
            lambda row: row['energy_kwh'] * get_electricity_cost(row['timestamp']),
            axis=1
        )

```

```

        total_grid_import_kwh = import_df['energy_kwh'].sum()
        total_cost_eur = import_df['cost_eur'].sum()
        peak_import = import_df['power_kw'].max()
    else:
        total_grid_import_kwh = total_consumption_kwh
        consumption_df['cost_eur'] = consumption_df.apply(
            lambda row: row['energy_kwh'] * get_electricity_cost(row['timestamp']),
            axis=1
        )
        total_cost_eur = consumption_df['cost_eur'].sum()
        peak_import = peak_consumption

# =====
# PV data
# =====

pv_generated_kwh = 0
pv_consumed_kwh = 0
self_consumption_rate = 0
self_sufficiency_rate = 0

if has_der:
    # Get PV production from port metrics
    pv_query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as power_kw
        FROM measurements
        WHERE source_id = {port_id}
        AND metric_id = {metric_map.get('power_active_production', -1)}
        ORDER BY timestamp
    """
    pv_df = pd.read_sql_query(pv_query, conn)

    if not pv_df.empty:
        pv_df['energy_kwh'] = pv_df['power_kw'] * timestep_hours
        pv_generated_kwh = pv_df['energy_kwh'].sum()

    # Self-consumption = consumption - grid import (what came from PV)
    pv_consumed_kwh = max(0, total_consumption_kwh - total_grid_import_kwh)

    if pv_generated_kwh > 0:
        self_consumption_rate = min(100, (pv_consumed_kwh / pv_generated_kwh) * 100)

    if total_consumption_kwh > 0:
        self_sufficiency_rate = (pv_consumed_kwh / total_consumption_kwh) * 100

conn.close()

return {
    'total_consumption_kwh': total_consumption_kwh,
    'total_grid_import_kwh': total_grid_import_kwh,
    'total_cost_eur': total_cost_eur,
    'peak_consumption_kw': peak_consumption,
    'peak_import_kw': peak_import,
    'pv_generated_kwh': pv_generated_kwh,
    'pv_consumed_kwh': pv_consumed_kwh,
    'self_consumption_rate': self_consumption_rate,
    'self_sufficiency_rate': self_sufficiency_rate,
}

def main():
    print("=" * 100)
    print("PORT ELECTRIFICATION STUDY - KPI ANALYSIS")
    print("=" * 100)

    results = []

    for vessels in VESSEL_COUNTS:

```

```

for scenario_key, scenario_name, has_der in SCENARIOS:
    db_path = get_db_path(vessels, scenario_key)

    kpis = calculate_kpis(db_path, has_der)
    reliability = analyze_trip_reliability(db_path, vessels)

    if kpis is None:
        print(f"Warning: Could not load {vessels} vessels, {scenario_name}")
        continue

    # Cost per successful trip
    successful_trips = reliability['on_time'] + reliability['delayed']
    cost_per_trip = kpis['total_cost_eur'] / successful_trips if successful_trips > 0 else float('inf')

    results.append({
        'Vessels': vessels,
        'Scenario': scenario_name,
        'On-Time': reliability['on_time'],
        'Delayed': reliability['delayed'],
        'Cancelled': reliability['cancelled'],
        'KPI 6: Reliability (%)': round(reliability['reliability'], 1),
        'Energy (kWh)': round(kpis['total_consumption_kwh'], 1),
        'KPI 1: Cost (€)': round(kpis['total_cost_eur'], 2),
        'KPI 2: €/Trip': round(cost_per_trip, 2),
        'KPI 3: Peak (kW)': round(kpis['peak_import_kw'], 1),
        'PV (kWh)': round(kpis['pv_generated_kwh'], 1) if has_der else '-',
        'KPI 4: Self-Cons (%)': round(kpis['self_consumption_rate'], 1) if has_der else '-',
        'KPI 5: Self-Suff (%)': round(kpis['self_sufficiency_rate'], 1) if has_der else '-',
    })

df = pd.DataFrame(results)

# =====
# TABLE 1: RELIABILITY METRICS
# =====
print("\n" + "=" * 100)
print("TABLE 1: TRIP RELIABILITY PERFORMANCE")
print("=" * 100)
cols1 = ['Vessels', 'Scenario', 'On-Time', 'Delayed', 'Cancelled', 'KPI 6: Reliability (%)']
print(tabulate(df[cols1], headers='keys', tablefmt='grid', showindex=False))

# =====
# TABLE 2: ECONOMIC METRICS
# =====
print("\n" + "=" * 100)
print("TABLE 2: ECONOMIC PERFORMANCE")
print("=" * 100)
cols2 = ['Vessels', 'Scenario', 'Energy (kWh)', 'KPI 1: Cost (€)', 'KPI 2: €/Trip', 'KPI 3: Peak (kW)']
print(tabulate(df[cols2], headers='keys', tablefmt='grid', showindex=False))

# =====
# TABLE 3: RENEWABLE ENERGY METRICS (DER only)
# =====
print("\n" + "=" * 100)
print("TABLE 3: RENEWABLE ENERGY UTILIZATION (DER Scenarios)")
print("=" * 100)
der_df = df[df['Scenario'].str.contains('DER') & ~df['Scenario'].str.contains('No DER')]
cols3 = ['Vessels', 'Scenario', 'PV (kWh)', 'KPI 4: Self-Cons (%)', 'KPI 5: Self-Suff (%)']
print(tabulate(der_df[cols3], headers='keys', tablefmt='grid', showindex=False))

# Save full table
output_path = Path(__file__).parent / "comparison_results" / "kpi_analysis.csv"
df.to_csv(output_path, index=False)
print(f"\n✓ Full KPI table saved to: {output_path}")

# =====
# LATEX TABLE
# =====

```

```

print("\n" + "=" * 100)
print("LATEX TABLE FOR THESIS")
print("=" * 100)

latex = r"""
\begin{table}[htbp]
\centering
\caption{Key Performance Indicators for Port Electrification Scenarios}
\label{tab:kpis}
\small
\begin{tabular}{|c|l|c|c|c|c|c|}
\hline
\textbf{Vessels} & \textbf{Scenario} & \textbf{Reliability} & \textbf{Cost} & \textbf{€/Trip} & \textbf{Peak Power} & \textbf{Self-Sufficiency} \\
& & (\%) & (€) & (kW) & (\%) & (\%) \\
\hline
"""

for _, row in df.iterrows():
    sc = row['KPI 4: Self-Cons (%)'] if row['KPI 4: Self-Cons (%)'] != '-' else '-'
    ss = row['KPI 5: Self-Suff (%)'] if row['KPI 5: Self-Suff (%)'] != '-' else '-'
    latex += f"{row['Vessels']} & {row['Scenario']} & {row['KPI 6: Reliability (%)']} & {row['KPI 1: Cost (€)']} & {row['KPI 2: €/Trip']} & {row['KPI 3: Peak Power (kW)']} & {row['KPI 5: Self-Sufficiency (%)]}' \\
    latex += r"""\hline
\end{tabular}
\end{table}
"""

print(latex)

latex_path = Path(__file__).parent / "comparison_results" / "kpi_table.tex"
with open(latex_path, 'w') as f:
    f.write(latex)
print(f"✓ LaTeX table saved to: {latex_path}")

# =====
# KEY FINDINGS
# =====

print("\n" + "=" * 100)
print("KEY FINDINGS FOR THESIS")
print("=" * 100)

for vessels in VESSEL_COUNTS:
    v_df = df[df['Vessels'] == vessels]
    baseline = v_df[v_df['Scenario'] == 'No Optimization, No DER'].iloc[0]
    opt_der = v_df[v_df['Scenario'] == 'Optimization + DER'].iloc[0]

    reliability_improvement = opt_der['KPI 6: Reliability (%)'] - baseline['KPI 6: Reliability (%)']
    cost_per_trip_change = ((opt_der['KPI 2: €/Trip'] / baseline['KPI 2: €/Trip']) - 1) * 100

    print(f"\n{vessels} Vessels:")
    print(f"Baseline (No Opt, No DER):")
    print(f"- Reliability: {baseline['KPI 6: Reliability (%)']}%")
    print(f"- Cost: €{baseline['KPI 1: Cost (€)]} (€{baseline['KPI 2: €/Trip']}/trip)")
    print(f"- Peak Power: {baseline['KPI 3: Peak (kW)']} kW")
    print(f"With Optimization + DER:")
    print(f"- Reliability: {opt_der['KPI 6: Reliability (%)']}% ({reliability_improvement:+.1f}pp im)")
    print(f"- Cost: €{opt_der['KPI 1: Cost (€)]} (€{opt_der['KPI 2: €/Trip']}/trip)")
    print(f"- Cost per trip change: {cost_per_trip_change:+.1f}%")
    if opt_der['KPI 5: Self-Suff (%)'] != '-':
        print(f"- Self-sufficiency: {opt_der['KPI 5: Self-Suff (%)']}%")

if __name__ == "__main__":
    main()

```



# tests/port\_eletrification\_studies/compare\_scenario s.py

---

```
"""
Comparison script for port electrification studies.

This script reads the simulation databases and generates comparison plots for:
1. Individual SOC of boats (with departure times as vertical lines)
2. Contracted power vs charger usage
3. Summary of reliability (delayed/canceled trips)
"""

import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
from pathlib import Path
from datetime import datetime, timedelta
import os

# Configuration
SCENARIOS = {
    "no_opt_no_der": "No Optimization, No DER",
    "opt_no_der": "Optimization, No DER",
    "opt_der": "Optimization + DER",
}

VESSEL_COUNTS = [5, 10, 20]

# Trip schedule times
TRIP_DEPARTURE_HOURS = [9, 14] # 9:00 AM and 2:00 PM

# Colors for scenarios
COLORS = {
    "no_opt_no_der": "#e74c3c", # Red
    "opt_no_der": "#3498db", # Blue
    "opt_der": "#2ecc71", # Green
}

def get_db_path(vessels: int, scenario: str) -> str:
    """Get the database path for a specific scenario."""
    # Databases are in the project root
    project_root = Path(__file__).parent.parent.parent
    return str(project_root / f"{vessels}_vessels_{scenario}.db")

def load_boat_soc_data(db_path: str) -> pd.DataFrame:
    """Load boat SOC data from the database."""
    if not os.path.exists(db_path):
        print(f"Warning: Database not found: {db_path}")
        return pd.DataFrame()

    conn = sqlite3.connect(db_path)

    # Get SOC metric ID
    soc_query = "SELECT metric_id FROM metric WHERE metric_name = 'soc'"
    soc_id = pd.read_sql_query(soc_query, conn).iloc[0, 0]

    # Get all boat sources
    boat_sources = pd.read_sql_query(
        "SELECT source_id, source_name FROM source WHERE source_type = 'boat'",
        conn
```

```

)

# Get SOC measurements for all boats
all_data = []
for _, row in boat_sources.iterrows():
    query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as soc, '{row['source_name']}' as boat
        FROM measurements
        WHERE source_id = {row['source_id']} AND metric_id = {soc_id}
        ORDER BY timestamp
    """
    df = pd.read_sql_query(query, conn)
    all_data.append(df)

conn.close()

if all_data:
    result = pd.concat(all_data, ignore_index=True)
    result['timestamp'] = pd.to_datetime(result['timestamp'])
    return result
return pd.DataFrame()

def load_power_data(db_path: str) -> pd.DataFrame:
    """Load power consumption and contracted power data from the database."""
    if not os.path.exists(db_path):
        print(f"Warning: Database not found: {db_path}")
        return pd.DataFrame()

    conn = sqlite3.connect(db_path)

    # Get metric IDs
    consumption_id = pd.read_sql_query(
        "SELECT metric_id FROM metric WHERE metric_name = 'power_active_consumption'",
        conn
    ).iloc[0, 0]

    contracted_id = pd.read_sql_query(
        "SELECT metric_id FROM metric WHERE metric_name = 'contracted_power'",
        conn
    ).iloc[0, 0]

    # Get port source ID
    port_source = pd.read_sql_query(
        "SELECT source_id FROM source WHERE source_type = 'port'",
        conn
    ).iloc[0, 0]

    # Get power measurements
    query = f"""
        SELECT
            m1.timestamp,
            CAST(m1.value AS FLOAT) as consumption,
            CAST(m2.value AS FLOAT) as contracted_power
        FROM measurements m1
        JOIN measurements m2 ON m1.timestamp = m2.timestamp AND m2.source_id = {port_source} AND m2.metric_id = {contracted_id}
        WHERE m1.source_id = {port_source} AND m1.metric_id = {consumption_id}
        ORDER BY m1.timestamp
    """

    df = pd.read_sql_query(query, conn)
    conn.close()

    if not df.empty:
        df['timestamp'] = pd.to_datetime(df['timestamp'])
    return df

def load_charger_power_data(db_path: str) -> pd.DataFrame:
    """Load individual charger power data from the database."""

```

```

if not os.path.exists(db_path):
    print(f"Warning: Database not found: {db_path}")
    return pd.DataFrame()

conn = sqlite3.connect(db_path)

# Get power_active metric ID
power_id = pd.read_sql_query(
    "SELECT metric_id FROM metric WHERE metric_name = 'power_active'",
    conn
).iloc[0, 0]

# Get all charger sources
charger_sources = pd.read_sql_query(
    "SELECT source_id, source_name FROM source WHERE source_type = 'charger'",
    conn
)

# Get power measurements for all chargers
all_data = []
for _, row in charger_sources.iterrows():
    query = f"""
        SELECT timestamp, CAST(value AS FLOAT) as power, '{row['source_name']}' as charger
        FROM measurements
        WHERE source_id = {row['source_id']} AND metric_id = {power_id}
        ORDER BY timestamp
    """
    df = pd.read_sql_query(query, conn)
    all_data.append(df)

conn.close()

if all_data:
    result = pd.concat(all_data, ignore_index=True)
    result['timestamp'] = pd.to_datetime(result['timestamp'])
    return result
return pd.DataFrame()

def load_boat_state_data(db_path: str) -> pd.DataFrame:
    """Load boat state data (sailing = 1.0, not sailing = 0.0) from the database."""
    if not os.path.exists(db_path):
        return pd.DataFrame()

    conn = sqlite3.connect(db_path)

    # Get state metric ID
    state_query = "SELECT metric_id FROM metric WHERE metric_name = 'state'"
    state_result = pd.read_sql_query(state_query, conn)
    if state_result.empty:
        conn.close()
        return pd.DataFrame()
    state_id = state_result.iloc[0, 0]

    # Get all boat sources
    boat_sources = pd.read_sql_query(
        "SELECT source_id, source_name FROM source WHERE source_type = 'boat'",
        conn
    )

    # Get state measurements for all boats
    all_data = []
    for _, row in boat_sources.iterrows():
        query = f"""
            SELECT timestamp, CAST(value AS FLOAT) as state, '{row['source_name']}' as boat
            FROM measurements
            WHERE source_id = {row['source_id']} AND metric_id = {state_id}
            ORDER BY timestamp
        """

```

```

        """
        df = pd.read_sql_query(query, conn)
        all_data.append(df)

    conn.close()

    if all_data:
        result = pd.concat(all_data, ignore_index=True)
        result['timestamp'] = pd.to_datetime(result['timestamp'])
        return result
    return pd.DataFrame()

def analyze_reliability(db_path: str, num_vessels: int) -> dict:
    """
    Analyze trip reliability by examining boat state patterns.

    A trip is detected when boat state changes to sailing (1.0).
    A trip is on-time if it starts at the scheduled hour (9:00 or 14:00).
    A trip is delayed if it starts later.
    A trip is cancelled if it never starts (second trip slot not detected).

    Returns dict with:
    - total_trips: Total scheduled trips
    - on_time_trips: Trips that started on time
    - delayed_trips: Trips that were delayed
    - cancelled_trips: Trips that were cancelled
    """
    # Total trips = 2 trips per vessel per day (Monday schedule)
    total_trips = num_vessels * 2

    if not os.path.exists(db_path):
        return {
            "total_trips": total_trips,
            "on_time_trips": 0,
            "delayed_trips": 0,
            "cancelled_trips": total_trips,
            "delay_rate": 100.0,
            "cancel_rate": 100.0,
        }

    # Load boat state data
    state_data = load_boat_state_data(db_path)

    if state_data.empty:
        return {
            "total_trips": total_trips,
            "on_time_trips": 0,
            "delayed_trips": 0,
            "cancelled_trips": total_trips,
            "delay_rate": 100.0,
            "cancel_rate": 100.0,
        }

    # Get unique boats
    boats = state_data['boat'].unique()

    on_time_trips = 0
    delayed_trips = 0
    cancelled_trips = 0

    for boat in boats:
        boat_data = state_data[state_data['boat'] == boat].sort_values('timestamp')

        # Find all trip starts (when state changes from 0 to 1)
        state_values = boat_data['state'].values
        timestamps = boat_data['timestamp'].values
        trip_starts = []

```

```

for i in range(1, len(state_values)):
    if state_values[i] == 1.0 and state_values[i-1] == 0.0:
        trip_starts.append(pd.to_datetime(timestamps[i]))

# Analyze each trip slot
morning_trip_found = False
afternoon_trip_found = False

for trip_start in trip_starts:
    hour = trip_start.hour
    minute = trip_start.minute

    if hour == 9 and minute < 15:
        # Morning trip started on time
        on_time_trips += 1
        morning_trip_found = True
    elif 9 <= hour < 14:
        # Morning trip delayed
        if not morning_trip_found:
            delayed_trips += 1
            morning_trip_found = True
    elif hour == 14 and minute < 15:
        # Afternoon trip started on time
        on_time_trips += 1
        afternoon_trip_found = True
    elif 14 <= hour < 18:
        # Afternoon trip delayed but still started
        if not afternoon_trip_found:
            delayed_trips += 1
            afternoon_trip_found = True
    elif hour >= 18:
        # Late trip - could be delayed morning or afternoon
        if not morning_trip_found:
            delayed_trips += 1
            morning_trip_found = True
        elif not afternoon_trip_found:
            delayed_trips += 1
            afternoon_trip_found = True

# Count cancelled trips
if not morning_trip_found:
    cancelled_trips += 1
if not afternoon_trip_found:
    cancelled_trips += 1

# Calculate rates
delay_rate = (delayed_trips / total_trips) * 100 if total_trips > 0 else 0
cancel_rate = (cancelled_trips / total_trips) * 100 if total_trips > 0 else 0

return {
    "total_trips": total_trips,
    "on_time_trips": on_time_trips,
    "delayed_trips": delayed_trips,
    "cancelled_trips": cancelled_trips,
    "delay_rate": delay_rate,
    "cancel_rate": cancel_rate,
}

}

def plot_soc_comparison(vessels: int, output_dir: str):
    """Create SOC comparison plot for a given number of vessels."""
    fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharey=True)

    for idx, (scenario, label) in enumerate(SCENARIOS.items()):
        ax = axes[idx]
        db_path = get_db_path(vessels, scenario)
        soc_data = load_boat_soc_data(db_path)
        if soc_data.empty:

```

```

        ax.text(0.5, 0.5, f"No data for {label}", ha='center', va='center', transform=ax.transAxes)
        ax.set_title(label)
        continue

    # Plot SOC for each boat
    boats = sorted(soc_data['boat'].unique())
    colors_boats = plt.cm.tab20(np.linspace(0, 1, len(boats)))

    for i, boat in enumerate(boats):
        boat_data = soc_data[soc_data['boat'] == boat].sort_values('timestamp')
        ax.plot(boat_data['timestamp'], boat_data['soc'],
                color=colors_boats[i], alpha=0.7, linewidth=1.5, label=boat)

    # Add vertical lines for departure times
    if not soc_data.empty:
        base_date = soc_data['timestamp'].min().date()
        for hour in TRIP_DEPARTURE_HOURS:
            departure_time = datetime.combine(base_date, datetime.min.time().replace(hour=hour))
            ax.axvline(x=departure_time, color='red', linestyle='--', alpha=0.7, linewidth=2)
            ax.text(departure_time, 105, f"{hour}:00", ha='center', va='bottom',
                    color='red', fontsize=9, fontweight='bold')

    # Add minimum SOC requirement line
    ax.axhline(y=62.6, color='orange', linestyle=':', alpha=0.8, linewidth=1.5, label='Min SOC for Trip')

    ax.set_xlabel('Time', fontsize=11)
    ax.set_ylabel('SOC (%)', fontsize=11) if idx == 0 else None
    ax.set_title(f'{label}', fontsize=12, fontweight='bold')
    ax.set_ylim(-5, 110)
    ax.grid(True, alpha=0.3)

    # Format x-axis
    ax.tick_params(axis='x', rotation=45)

    # Add legend
    handles = [mpatches.Patch(color='red', label='Departure Time'),
               mpatches.Patch(color='orange', label='Min SOC (62.6%)')]
    fig.legend(handles=handles, loc='upper right', bbox_to_anchor=(0.99, 0.99))

    plt.suptitle(f'Boat SOC Comparison - {vessels} Vessels', fontsize=14, fontweight='bold', y=1.02)
    plt.tight_layout()

    output_path = os.path.join(output_dir, f'soc_comparison_{vessels}_vessels.png')
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f" Saved: {output_path}")

def plot_power_comparison(vessels: int, output_dir: str):
    """Create power usage comparison plot for a given number of vessels."""
    fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

    for idx, (scenario, label) in enumerate(SCENARIOS.items()):
        ax = axes[idx]
        db_path = get_db_path(vessels, scenario)
        power_data = load_power_data(db_path)

        if power_data.empty:
            ax.text(0.5, 0.5, f"No data for {label}", ha='center', va='center', transform=ax.transAxes)
            ax.set_title(label)
            continue

        # Plot consumption
        ax.fill_between(power_data['timestamp'], 0, power_data['consumption'],
                        alpha=0.6, color=COLORS[scenario], label='Charger Usage')
        ax.plot(power_data['timestamp'], power_data['consumption'],
                color=COLORS[scenario], linewidth=1.5)

        # Plot contracted power limit

```

```

contracted = power_data['contracted_power'].iloc[0]
ax.axhline(y=contracted, color='black', linestyle='--',
           linewidth=2, label=f'Contracted Power ({contracted} kW)')

# Add departure time markers
if not power_data.empty:
    base_date = power_data['timestamp'].min().date()
    for hour in TRIP_DEPARTURE_HOURS:
        departure_time = datetime.combine(base_date, datetime.min.time().replace(hour=hour))
        ax.axvline(x=departure_time, color='red', linestyle=':', alpha=0.5, linewidth=1.5)

ax.set_xlabel('Time', fontsize=11)
ax.set_ylabel('Power (kW)', fontsize=11) if idx == 0 else None
ax.set_title(f'{label}', fontsize=12, fontweight='bold')
ax.grid(True, alpha=0.3)
ax.legend(loc='upper right', fontsize=9)

# Format x-axis
ax.tick_params(axis='x', rotation=45)

plt.suptitle(f'Power Usage vs Contracted Power - {vessels} Vessels', fontsize=14, fontweight='bold', y=1)
plt.tight_layout()

output_path = os.path.join(output_dir, f'power_comparison_{vessels}_vessels.png')
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
print(f" Saved: {output_path}")

def plot_reliability_summary(output_dir: str):
    """Create reliability summary plot comparing all scenarios."""
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))

    for idx, vessels in enumerate(VESSEL_COUNTS):
        ax = axes[idx]

        scenario_names = []
        on_time = []
        delayed = []
        cancelled = []

        for scenario, label in SCENARIOS.items():
            db_path = get_db_path(vessels, scenario)
            reliability = analyze_reliability(db_path, vessels)

            scenario_names.append(label.replace(', ', '\n'))
            on_time.append(reliability['on_time_trips'])
            delayed.append(reliability['delayed_trips'])
            cancelled.append(reliability['cancelled_trips'])

        x = np.arange(len(scenario_names))
        width = 0.25

        bars1 = ax.bar(x - width, on_time, width, label='On-Time', color='#2ecc71')
        bars2 = ax.bar(x, delayed, width, label='Delayed', color='#f39c12')
        bars3 = ax.bar(x + width, cancelled, width, label='Cancelled', color='#e74c3c')

        ax.set_xlabel('Scenario', fontsize=11)
        ax.set_ylabel('Number of Trips', fontsize=11) if idx == 0 else None
        ax.set_title(f'{vessels} Vessels', fontsize=12, fontweight='bold')
        ax.set_xticks(x)
        ax.set_xticklabels(scenario_names, fontsize=9)
        ax.legend(fontsize=9)
        ax.grid(True, alpha=0.3, axis='y')

    # Add value labels on bars
    for bars in [bars1, bars2, bars3]:
        for bar in bars:

```

```

        height = bar.get_height()
        if height > 0:
            ax.annotate(f'{int(height)}',
                        xy=(bar.get_x() + bar.get_width() / 2, height),
                        xytext=(0, 3),
                        textcoords="offset points",
                        ha='center', va='bottom', fontsize=8)

plt.suptitle('Trip Reliability Summary by Scenario', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()

output_path = os.path.join(output_dir, 'reliability_summary.png')
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
print(f" Saved: {output_path}")

def create_summary_table(output_dir: str):
    """Create a summary table with key metrics."""
    summary_data = []

    for vessels in VESSEL_COUNTS:
        for scenario, label in SCENARIOS.items():
            db_path = get_db_path(vessels, scenario)
            reliability = analyze_reliability(db_path, vessels)

            # Load power data for peak analysis
            power_data = load_power_data(db_path)
            peak_power = power_data['consumption'].max() if not power_data.empty else 0
            avg_power = power_data['consumption'].mean() if not power_data.empty else 0
            contracted = power_data['contracted_power'].iloc[0] if not power_data.empty else 80

            summary_data.append({
                'Vessels': vessels,
                'Scenario': label,
                'Total Trips': reliability['total_trips'],
                'On-Time': reliability['on_time_trips'],
                'Delayed': reliability['delayed_trips'],
                'Cancelled': reliability['cancelled_trips'],
                'Delay Rate (%)': f"{reliability['delay_rate']:.1f}",
                'Cancel Rate (%)': f"{reliability['cancel_rate']:.1f}",
                'Peak Power (kW)': f"{peak_power:.1f}",
                'Avg Power (kW)': f"{avg_power:.1f}",
                'Contracted (kW)': f"{contracted:.1f}",
            })

    df = pd.DataFrame(summary_data)

    # Save to CSV
    csv_path = os.path.join(output_dir, 'summary_table.csv')
    df.to_csv(csv_path, index=False)
    print(f" Saved: {csv_path}")

    # Print summary
    print("\n" + "=" * 80)
    print("SUMMARY TABLE")
    print("=" * 80)
    print(df.to_string(index=False))
    print("=" * 80)

    return df

def plot_combined_comparison(output_dir: str):
    """Create a combined comparison plot showing key metrics across all scenarios."""
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # 1. Cancel rate comparison

```



```

ax1 = axes[0, 0]
x = np.arange(len(VESSEL_COUNTS))
width = 0.25

for i, (scenario, label) in enumerate(SCENARIOS.items()):
    cancel_rates = []
    for vessels in VESSEL_COUNTS:
        db_path = get_db_path(vessels, scenario)
        reliability = analyze_reliability(db_path, vessels)
        cancel_rates.append(reliability['cancel_rate'])

    ax1.bar(x + i * width, cancel_rates, width, label=label, color=COLORS[scenario])

ax1.set_xlabel('Number of Vessels', fontsize=11)
ax1.set_ylabel('Cancellation Rate (%)', fontsize=11)
ax1.set_title('Trip Cancellation Rate by Scenario', fontsize=12, fontweight='bold')
ax1.set_xticks(x + width)
ax1.set_xticklabels(VESSEL_COUNTS)
ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3, axis='y')

# 2. Peak power comparison
ax2 = axes[0, 1]
for i, (scenario, label) in enumerate(SCENARIOS.items()):
    peak_powers = []
    for vessels in VESSEL_COUNTS:
        db_path = get_db_path(vessels, scenario)
        power_data = load_power_data(db_path)
        peak_powers.append(power_data['consumption'].max() if not power_data.empty else 0)

    ax2.bar(x + i * width, peak_powers, width, label=label, color=COLORS[scenario])

# Add contracted power reference line
ax2.axhline(y=80, color='black', linestyle='--', linewidth=2, label='Contracted Power')

ax2.set_xlabel('Number of Vessels', fontsize=11)
ax2.set_ylabel('Peak Power (kW)', fontsize=11)
ax2.set_title('Peak Power Consumption by Scenario', fontsize=12, fontweight='bold')
ax2.set_xticks(x + width)
ax2.set_xticklabels(VESSEL_COUNTS)
ax2.legend(fontsize=9)
ax2.grid(True, alpha=0.3, axis='y')

# 3. Successful trips comparison
ax3 = axes[1, 0]
for i, (scenario, label) in enumerate(SCENARIOS.items()):
    success_rates = []
    for vessels in VESSEL_COUNTS:
        db_path = get_db_path(vessels, scenario)
        reliability = analyze_reliability(db_path, vessels)
        success_rate = ((reliability['on_time_trips'] + reliability['delayed_trips']) / reliability['total_trips'])
        success_rates.append(success_rate)

    ax3.bar(x + i * width, success_rates, width, label=label, color=COLORS[scenario])

ax3.set_xlabel('Number of Vessels', fontsize=11)
ax3.set_ylabel('Success Rate (%)', fontsize=11)
ax3.set_title('Trip Success Rate (On-Time + Delayed)', fontsize=12, fontweight='bold')
ax3.set_xticks(x + width)
ax3.set_xticklabels(VESSEL_COUNTS)
ax3.legend(fontsize=9)
ax3.grid(True, alpha=0.3, axis='y')
ax3.set_ylim(0, 110)

# 4. Average power utilization
ax4 = axes[1, 1]
for i, (scenario, label) in enumerate(SCENARIOS.items()):
    avg_powers = []

```

```

        for vessels in VESSEL_COUNTS:
            db_path = get_db_path(vessels, scenario)
            power_data = load_power_data(db_path)
            avg_powers.append(power_data['consumption'].mean() if not power_data.empty else 0)

        ax4.bar(x + i * width, avg_powers, width, label=label, color=COLORS[scenario])

    ax4.set_xlabel('Number of Vessels', fontsize=11)
    ax4.set_ylabel('Average Power (kW)', fontsize=11)
    ax4.set_title('Average Power Consumption by Scenario', fontsize=12, fontweight='bold')
    ax4.set_xticks(x + width)
    ax4.set_xticklabels(VESSEL_COUNTS)
    ax4.legend(fontsize=9)
    ax4.grid(True, alpha=0.3, axis='y')

plt.suptitle('Port Electrification Study - Scenario Comparison', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()

output_path = os.path.join(output_dir, 'combined_comparison.png')
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
print(f" Saved: {output_path}")

def load_charger_power_data(db_path: str) -> pd.DataFrame:
    """Load individual charger power data from the database."""
    if not os.path.exists(db_path):
        return pd.DataFrame()

    conn = sqlite3.connect(db_path)

    # Get power_active metric ID
    try:
        power_id = pd.read_sql_query(
            "SELECT metric_id FROM metric WHERE metric_name = 'power_active'",
            conn
        ).iloc[0, 0]
    except:
        conn.close()
        return pd.DataFrame()

    # Get all charger sources
    charger_sources = pd.read_sql_query(
        "SELECT source_id, source_name FROM source WHERE source_type = 'charger'",
        conn
    )

    all_data = []
    for _, row in charger_sources.iterrows():
        query = f"""
            SELECT timestamp, CAST(value AS FLOAT) as power, '{row['source_name']}' as charger
            FROM measurements
            WHERE source_id = {row['source_id']} AND metric_id = {power_id}
            ORDER BY timestamp
        """
        df = pd.read_sql_query(query, conn)
        all_data.append(df)

    conn.close()

    if all_data:
        result = pd.concat(all_data, ignore_index=True)
        result['timestamp'] = pd.to_datetime(result['timestamp'])
        return result
    return pd.DataFrame()

def plot_soc_grid(output_dir: str):

```

```

"""
Generate a 3x3 grid plot for boat SOC.

Rows: 5 vessels (25%), 10 vessels (50%), 20 vessels (100%)
Columns: No Opt No DER, Opt No DER, Opt + DER

Each subplot shows ONLY:
- SOC lines for all boats
- Two vertical lines for departure times (9:00 and 14:00)
"""
fig, axes = plt.subplots(3, 3, figsize=(16, 12))

vessel_labels = {
    5: "5 Vessels (25%)",
    10: "10 Vessels (50%)",
    20: "20 Vessels (100%)"
}

scenario_order = ["no_opt_no_der", "opt_no_der", "opt_der"]
scenario_labels = {
    "no_opt_no_der": "No Optimization, No DER",
    "opt_no_der": "Optimization, No DER",
    "opt_der": "Optimization + DER"
}

# Distinct colors for boats
boat_colors = [
    '#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
    '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf',
    '#aec7e8', '#ffbb78', '#98df8a', '#ff9896', '#c5b0d5',
    '#c49c94', '#f7b6d2', '#c7c7c7', '#bdbd8d', '#9edae5'
]

for row_idx, vessels in enumerate(VESSEL_COUNTS):
    for col_idx, scenario in enumerate(scenario_order):
        ax = axes[row_idx, col_idx]
        db_path = get_db_path(vessels, scenario)

        soc_data = load_boat_soc_data(db_path)

        if not soc_data.empty:
            boats = sorted(soc_data['boat'].unique())

            for b_idx, boat in enumerate(boats):
                boat_data = soc_data[soc_data['boat'] == boat].sort_values('timestamp')
                hours = np.array([(t.hour + t.minute/60.0) for t in boat_data['timestamp']])
                # SOC is already stored as percentage (0-100) in database
                soc_values = boat_data['soc'].values

                # Plot SOC line with thicker lines
                line_width = 2.0 if vessels <= 5 else (1.5 if vessels <= 10 else 1.0)
                ax.plot(hours, soc_values,
                        color=boat_colors[b_idx % len(boat_colors)],
                        linewidth=line_width,
                        alpha=0.85)

                # Add two vertical lines for departure times
                ax.axvline(x=9, color='red', linestyle='--', linewidth=2.5, label='Departure')
                ax.axvline(x=14, color='red', linestyle='--', linewidth=2.5)

            # Formatting
            ax.set_xlim(0, 24)
            ax.set_ylim(0, 105)
            ax.set_xticks([0, 3, 6, 9, 12, 14, 17, 20, 24])
            ax.grid(True, alpha=0.3)

            # Labels
            if row_idx == 0:

```

```

        ax.set_title(scenario_labels[scenario], fontsize=11, fontweight='bold')
    if col_idx == 0:
        ax.set_ylabel(f'{vessel_labels[vessels]}\nSOC (%)', fontsize=10)
    if row_idx == 2:
        ax.set_xlabel('Hour of Day', fontsize=10)

# Add legend
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='red', linestyle='--', linewidth=2.5, label='Departure Time (9:00 & 14:00)'),
]
fig.legend(handles=legend_elements, loc='upper right', fontsize=10)

plt.suptitle('Boat State of Charge (SOC) Comparison', fontsize=14, fontweight='bold')
plt.tight_layout()

output_path = os.path.join(output_dir, 'soc_grid_comparison.png')
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
print(f" Saved: {output_path}")

def plot_power_grid(output_dir: str):
    """
    Generate a 3x3 grid plot for charger power consumption.

    Rows: 5 vessels (25%), 10 vessels (50%), 20 vessels (100%)
    Columns: No Opt No DER, Opt No DER, Opt + DER

    Each subplot shows total charger power + contracted power horizontal line.
    """
    fig, axes = plt.subplots(3, 3, figsize=(18, 14))

    vessel_labels = {
        5: "5 Vessels (25% Fleet)",
        10: "10 Vessels (50% Fleet)",
        20: "20 Vessels (100% Fleet)"
    }

    contracted_powers = {5: 80, 10: 80, 20: 80} # kW

    scenario_order = ["no_opt_no_der", "opt_no_der", "opt_der"]
    scenario_labels = {
        "no_opt_no_der": "No Optimization\nNo DER",
        "opt_no_der": "Optimization\nNo DER",
        "opt_der": "Optimization\n+ DER"
    }

    # Color palette for chargers
    charger_colors = plt.cm.Set3(np.linspace(0, 1, 20))

    for row_idx, vessels in enumerate(VESSEL_COUNTS):
        for col_idx, scenario in enumerate(scenario_order):
            ax = axes[row_idx, col_idx]
            db_path = get_db_path(vessels, scenario)

            # Load port-level power data
            power_data = load_power_data(db_path)
            charger_data = load_charger_power_data(db_path)

            if not power_data.empty:
                hours = [(t.hour + t.minute/60) for t in power_data['timestamp']]

                # Plot total consumption
                ax.fill_between(hours, power_data['consumption'],
                               alpha=0.4, color=COLORS[scenario], label='Total Consumption')
                ax.plot(hours, power_data['consumption'],
                        color=COLORS[scenario], linewidth=1.5)

```

```

# Plot individual chargers as stacked (if data available)
if not charger_data.empty:
    chargers = charger_data['charger'].unique()

    # Create pivot table for stacked area
    pivot = charger_data.pivot_table(
        index='timestamp', columns='charger', values='power',
        aggfunc='first'
    ).fillna(0)

    if len(pivot) > 0:
        pivot_hours = [(t.hour + t.minute/60) for t in pivot.index]

        # Plot individual charger lines (thinner)
        for c_idx, charger in enumerate(sorted(chargers)):
            if charger in pivot.columns:
                ax.plot(pivot_hours, pivot[charger],
                        color=charger_colors[c_idx % 12],
                        linewidth=0.8, alpha=0.5)

# Add contracted power horizontal line
contracted = contracted_powers[vessels]
ax.axhline(y=contracted, color='red', linestyle='--',
           linewidth=2.5, label=f'Contracted Power ({contracted} kW)')

# Add departure time vertical lines
for dep_hour in TRIP_DEPARTURE_HOURS:
    ax.axvline(x=dep_hour, color='gray', linestyle=':',
               linewidth=1.5, alpha=0.5)

# Formatting
ax.set_xlim(0, 24)
ax.set_ylim(0, max(180, contracted * 2))
ax.set_xticks([0, 6, 9, 12, 14, 18, 24])
ax.grid(True, alpha=0.3)

# Add row/column labels
if row_idx == 0:
    ax.set_title(scenario_labels[scenario], fontsize=12, fontweight='bold')
if col_idx == 0:
    ax.set_ylabel(f'{vessel_labels[vessels]}\nPower (kW)', fontsize=10)
else:
    ax.set_ylabel('Power (kW)', fontsize=10)
if row_idx == 2:
    ax.set_xlabel('Hour of Day', fontsize=10)

# Add legend
from matplotlib.lines import Line2D
legend_elements = [
    mpatches.Patch(facecolor=COLORS["opt_der"], alpha=0.4, label='Total Consumption'),
    Line2D([0], [0], color='red', linestyle='--', linewidth=2.5, label='Contracted Power (80 kW)'),
    Line2D([0], [0], color='gray', linestyle=':', linewidth=1.5, label='Departure Time'),
]
fig.legend(handles=legend_elements, loc='upper right', fontsize=10,
           bbox_to_anchor=(0.98, 0.98))

plt.suptitle('Charger Power Consumption by Scenario and Fleet Size',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()

output_path = os.path.join(output_dir, 'power_grid_comparison.png')
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
print(f" Saved: {output_path}")

def main():
    """Main function to run the comparison analysis."""

```

```

print("=" * 60)
print("Port Electrification Study - Scenario Comparison")
print("=" * 60)

# Create output directory
output_dir = "comparison_results"
os.makedirs(output_dir, exist_ok=True)
print(f"\nOutput directory: {output_dir}")

# Generate plots for each vessel count
print("\n1. Generating SOC comparison plots...")
for vessels in VESSEL_COUNTS:
    plot_soc_comparison(vessels, output_dir)

print("\n2. Generating power usage comparison plots...")
for vessels in VESSEL_COUNTS:
    plot_power_comparison(vessels, output_dir)

print("\n3. Generating reliability summary plot...")
plot_reliability_summary(output_dir)

print("\n4. Generating combined comparison plot...")
plot_combined_comparison(output_dir)

print("\n5. Creating summary table...")
create_summary_table(output_dir)

print("\n6. Generating SOC grid comparison (3x3)...")
plot_soc_grid(output_dir)

print("\n7. Generating Power grid comparison (3x3)...")
plot_power_grid(output_dir)

print("\n" + "=" * 60)
print("Analysis complete! All plots saved to:", output_dir)
print("=" * 60)

if __name__ == "__main__":
    main()

```

# tests/port\_eletrification\_studies/run\_all.py

---

```
import subprocess
import sys
from pathlib import Path

THIS_DIR = Path(__file__).parent

# Only run scenario scripts (exclude helpers)
EXCLUDE = {
    "run_all.py",
    "calculate_kpis.py",
    "compare_scenarios.py",
}

scripts = sorted(
    p
    for p in THIS_DIR.glob("*.py")
    if p.name not in EXCLUDE and not p.name.startswith("_")
)

if not scripts:
    print("No test scripts found.")
    sys.exit(1)

failed = []

for script in scripts:
    module = f"tests.port_eletrification_studies.{script.stem}"
    print(f"\n■ Running {module}")

    result = subprocess.run(
        [sys.executable, "-m", module],
        stdout=sys.stdout,
        stderr=sys.stderr,
    )

    if result.returncode != 0:
        failed.append(script.name)

print("\n" + "=" * 50)
if failed:
    print("■ Failed scripts:")
    for f in failed:
        print(f" - {f}")
    sys.exit(1)
else:
    print("■ All tests completed successfully")
```

# tests/test\_bess.py

---

```
"""
BESS Model Verification (BESS class).

Objective:
    Verify correct energy balance during charging and discharging, enforcement
    of maximum power limits, and compliance with SOC constraints.

Test Case 1 - Charging Dynamics:
    The battery is initialized at a defined SOC and subjected to a charging
    command at rated power for a known timestep. The stored energy is
         $E_{\text{stored}} = P_{\text{charge}} \cdot \eta \cdot \Delta t$ 
    and the resulting SOC variation is compared against the analytical
    expectation.

Test Case 2 - Discharging Dynamics:
    The battery is discharged at rated power for a fixed timestep. The energy
    removed from the battery is
         $E_{\text{removed}} = (P_{\text{discharge}} / \eta) \cdot \Delta t$ 
    and the SOC reduction is verified accordingly.

Test Case 3 - SOC Limit Enforcement:
    Charging and discharging commands are applied near SOC_max and SOC_min.
    The model must prevent overcharge and overdischarge by adjusting the
    actual power and clamping SOC within its allowable range.

Test Case 4 - Power Constraints and Round-Trip Behavior:
    Requested power values exceeding rated limits are evaluated to confirm
    internal clamping. A complete charge-discharge cycle is performed to
    verify the expected round-trip efficiency behaviour.

Run with --generate to produce plots and CSV in tests/output/.
"""

import csv
import sys
from datetime import timedelta
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent))

import pytest

from models.bess import BESS

OUTPUT_DIR = Path(__file__).parent / "output" / "bess"

BESS_CAPACITY = 100.0 # kWh
BESS_MAX_POWER = 50.0 # kW (charge and discharge)
BESS EFFICIENCY = 0.90 # round-trip efficiency factor
BESS_SOC_MIN = 0.10
BESS_SOC_MAX = 0.90

def _make_bess(**overrides) -> BESS:
    base = {
        "name": "TestBESS",
        "capacity": BESS_CAPACITY,
        "max_charge_power": BESS_MAX_POWER,
        "max_discharge_power": BESS_MAX_POWER,
        "efficiency": BESS EFFICIENCY,
        "soc_min": BESS_SOC_MIN,
```



```

        "soc_max": BESS_SOC_MAX,
        "initial_soc": 0.5,
    }
    base.update(overrides)
    return BESS(**base)

class TestChargingDynamics:
    """Test Case 1: Charging dynamics and energy balance."""

    def test_charging_energy_balance(self):
        """
        Charge at rated power for a known duration and compare SOC change to
         $E_{\text{stored}} = P_{\text{charge}} \cdot \eta \cdot \Delta t$ .
        """
        bess = _make_bess(initial_soc=0.2)
        initial_soc = bess.current_soc

        power = BESS_MAX_POWER # kW
        duration_seconds = 3600 # 1 hour

        actual_power = bess.charge(power, duration_seconds)
        assert actual_power <= BESS_MAX_POWER + 1e-9

        # Analytical expectation
        stored_energy = power * BESS EFFICIENCY * (duration_seconds / 3600.0)
        expected_delta_soc = stored_energy / BESS_CAPACITY
        expected_soc = initial_soc + expected_delta_soc

        assert bess.current_soc == pytest.approx(expected_soc, rel=1e-6)

class TestDischargingDynamics:
    """Test Case 2: Discharging dynamics and energy balance."""

    def test_discharging_energy_balance(self):
        """
        Discharge at rated power for a known duration and compare SOC change to
         $E_{\text{removed}} = (P_{\text{discharge}} / \eta) \cdot \Delta t$ .
        """
        bess = _make_bess(initial_soc=0.8)
        initial_soc = bess.current_soc

        power = BESS_MAX_POWER # kW
        duration_seconds = 3600 # 1 hour

        actual_power = bess.discharge(power, duration_seconds)
        assert actual_power <= BESS_MAX_POWER + 1e-9

        delivered_energy = power * (duration_seconds / 3600.0)
        removed_from_battery = delivered_energy / BESS EFFICIENCY
        expected_delta_soc = removed_from_battery / BESS_CAPACITY
        expected_soc = initial_soc - expected_delta_soc

        assert bess.current_soc == pytest.approx(expected_soc, rel=1e-6)

class TestSocLimitEnforcement:
    """Test Case 3: SOC clamping at SOC_min and SOC_max."""

    def test_prevents_overcharge_at_soc_max(self):
        """Charging near SOC_max must clamp SOC and adjust actual power."""
        bess = _make_bess(initial_soc=BESS_SOC_MAX - 0.02)

        power = BESS_MAX_POWER
        duration_seconds = 3600 # enough to overshoot without clamp

        actual_power = bess.charge(power, duration_seconds)

```

```

    assert bess.current_soc == pytest.approx(BESS_SOC_MAX, rel=1e-9)
    assert 0.0 < actual_power <= BESS_MAX_POWER + 1e-9

def test_prevents_overdischarge_at_soc_min(self):
    """Discharging near SOC_min must clamp SOC and adjust actual power."""
    bess = _make_bess(initial_soc=BESS_SOC_MIN + 0.02)

    power = BESS_MAX_POWER
    duration_seconds = 3600

    actual_power = bess.discharge(power, duration_seconds)

    assert bess.current_soc == pytest.approx(BESS_SOC_MIN, rel=1e-9)
    assert 0.0 < actual_power <= BESS_MAX_POWER + 1e-9

class TestPowerConstraintsAndRoundTrip:
    """Test Case 4: Power limits and round-trip efficiency behaviour."""

    def test_charge_and_discharge_power_clamping(self):
        """Requested power above ratings must be internally clamped."""
        bess = _make_bess(initial_soc=0.5)

        # Charge side
        requested_charge = BESS_MAX_POWER * 2
        actual_charge = bess.charge(requested_charge, 60)
        assert actual_charge <= BESS_MAX_POWER + 1e-9

        # Discharge side
        requested_discharge = BESS_MAX_POWER * 2
        actual_discharge = bess.discharge(requested_discharge, 60)
        assert actual_discharge <= BESS_MAX_POWER + 1e-9

    def test_round_trip_cycle(self):
        """
        Charge then discharge at rated power; final SOC must be below initial
        and match analytical round-trip expectation.
        """
        bess = _make_bess(initial_soc=0.5)

        power = BESS_MAX_POWER
        duration = 1800 # 0.5 h each phase
        initial_soc = bess.current_soc

        # Charge
        bess.charge(power, duration)
        soc_after_charge = bess.current_soc
        assert soc_after_charge > initial_soc

        # Discharge
        bess.discharge(power, duration)
        final_soc = bess.current_soc
        assert final_soc < initial_soc

        # Analytical expectation:
        e_charge = power * (duration / 3600.0) * BESS_EFFICIENCY
        e_discharge = power * (duration / 3600.0) / BESS_EFFICIENCY
        expected_final_soc = initial_soc + (e_charge - e_discharge) / BESS_CAPACITY

        assert final_soc == pytest.approx(expected_final_soc, rel=1e-6)

# -----
# Output generation: CSV and plots for documentation / thesis
# -----

class TestBESSwithOutput:

```

```

"""Generate CSV and plots from BESS tests for reporting."""

@pytest.fixture(autouse=True)
def setup_output_dir(self):
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

def test_charge_discharge_profile_csv_and_plot(self):
    """
    Time-series of SOC and power for a charge-idle-discharge sequence.
    CSV + plot saved under tests/output/bess.
    """
    pytest.importorskip("matplotlib")
    import matplotlib.pyplot as plt

    bess = _make_bess(initial_soc=0.5)

    timestep_s = 60
    minutes_charge = 30
    minutes_idle = 5
    minutes_discharge = 30

    t_min = 0
    rows = []
    times = []
    socs = []
    powers = []

    def record(phase: str):
        rows.append(
            {
                "time_min": t_min,
                "phase": phase,
                "soc_percent": round(bess.current_soc * 100.0, 3),
                "power_kw": round(bess.current_power, 3),
                "energy_stored_kwh": round(bess.get_energy_stored(), 3),
            }
        )
        times.append(t_min)
        socs.append(bess.current_soc * 100.0)
        powers.append(bess.current_power)

    record("initial")

    # Charging phase
    for _ in range(minutes_charge):
        t_min += 1
        bess.charge(BESS_MAX_POWER, timestep_s)
        record("charging")

    # Idle phase
    for _ in range(minutes_idle):
        t_min += 1
        bess.idle()
        record("idle")

    # Discharging phase
    for _ in range(minutes_discharge):
        t_min += 1
        bess.discharge(BESS_MAX_POWER, timestep_s)
        record("discharging")

    csv_path = OUTPUT_DIR / "bess_charge_discharge_profile.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=[
                "time_min",
                "phase",
            ]

```

```

        "soc_percent",
        "power_kw",
        "energy_stored_kwh",
    ],
)
writer.writeheader()
writer.writerows(rows)
print(f"\n\u2713 CSV saved: {csv_path}")

fig, (ax_soc, ax_p) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)

ax_soc.plot(times, socs, "b-", linewidth=2)
ax_soc.axhline(BESS_SOC_MIN * 100, color="orange", linestyle="--", alpha=0.7)
ax_soc.axhline(BESS_SOC_MAX * 100, color="red", linestyle="--", alpha=0.7)
ax_soc.set_ylabel("SOC (%)")
ax_soc.set_ylim(0, 100)
ax_soc.grid(True, alpha=0.3)

ax_p.bar(times, powers, color=["green" if p > 0 else "red" if p < 0 else "gray" for p in powers])
ax_p.axhline(0.0, color="black", linewidth=1)
ax_p.axhline(BESS_MAX_POWER, color="green", linestyle="--", alpha=0.5)
ax_p.axhline(-BESS_MAX_POWER, color="red", linestyle="--", alpha=0.5)
ax_p.set_xlabel("Time (minutes)")
ax_p.set_ylabel("Power (kW)")
ax_p.grid(True, alpha=0.3)

fig.suptitle(
    f"BESS charge-idle-discharge sequence\n"
    f"Capacity {BESS_CAPACITY} kWh,  $\pm$ {BESS_MAX_POWER} kW,  $\eta$ ={BESS EFFICIENCY:.0%}"
)
fig.tight_layout()

plot_path = OUTPUT_DIR / "bess_charge_discharge_profile.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"\u2713 Plot saved: {plot_path}")

# Sanity checks: SOC remains within [SOC_min, SOC_max]
assert min(socs) >= BESS_SOC_MIN * 100 - 1e-6
assert max(socs) <= BESS_SOC_MAX * 100 + 1e-6

def test_soc_limits_summary_csv(self):
    """CSV summarising operation near SOC_min and SOC_max constraints."""
    bess = _make_bess(initial_soc=0.5)

    rows = []

    # Near SOC_max: repeated charging
    for _ in range(10):
        before_soc = bess.current_soc
        actual_power = bess.charge(BESS_MAX_POWER, 600) # 10 min
        after_soc = bess.current_soc
        rows.append(
            {
                "phase": "charge_near_max",
                "soc_before": round(before_soc, 4),
                "soc_after": round(after_soc, 4),
                "requested_power_kw": BESS_MAX_POWER,
                "actual_power_kw": round(actual_power, 4),
            }
        )

    # Then near SOC_min: repeated discharging
    for _ in range(20):
        before_soc = bess.current_soc
        actual_power = bess.discharge(BESS_MAX_POWER, 600)
        after_soc = bess.current_soc
        rows.append(

```

```

        {
            "phase": "discharge_near_min",
            "soc_before": round(before_soc, 4),
            "soc_after": round(after_soc, 4),
            "requested_power_kw": BESS_MAX_POWER,
            "actual_power_kw": round(actual_power, 4),
        }
    )

csv_path = OUTPUT_DIR / "bess_soc_limits.csv"
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "phase",
            "soc_before",
            "soc_after",
            "requested_power_kw",
            "actual_power_kw",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)
print(f"\n\u2713 CSV saved: {csv_path}")

# All SOC values must remain within [SOC_min, SOC_max].
for r in rows:
    assert BESS_SOC_MIN <= r["soc_after"] <= BESS_SOC_MAX + 1e-9

def test_round_trip_summary_csv(self):
    """CSV summarising a simple charge-discharge round-trip."""
    bess = _make_bess(initial_soc=0.5)

    power = BESS_MAX_POWER
    duration = 1800 # 0.5 h

    initial_energy = bess.get_energy_stored()
    bess.charge(power, duration)
    energy_after_charge = bess.get_energy_stored()
    bess.discharge(power, duration)
    final_energy = bess.get_energy_stored()

    energy_in = power * (duration / 3600.0)
    stored = energy_after_charge - initial_energy
    delivered = initial_energy - final_energy + stored # net to load in cycle

    rows = [
        {
            "capacity_kwh": BESS_CAPACITY,
            "max_power_kw": BESS_MAX_POWER,
            "efficiency": BESS EFFICIENCY,
            "energy_input_kwh": round(energy_in, 4),
            "energy_stored_kwh": round(stored, 4),
            "energy_delivered_kwh": round(delivered, 4),
        }
    ]

csv_path = OUTPUT_DIR / "bess_round_trip_summary.csv"
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "capacity_kwh",
            "max_power_kw",
            "efficiency",
            "energy_input_kwh",
            "energy_stored_kwh",
            "energy_delivered_kwh",
        ],
    )

```

```

        ],
    )
    writer.writeheader()
    writer.writerows(rows)
    print(f"\n\u2713 CSV saved: {csv_path}")

def generate_all_outputs():
    """Generate all BESS test outputs (CSV + plots) without running full pytest."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    instance = TestBESSwithOutput()
    instance.setup_output_dir()
    instance.test_charge_discharge_profile_csv_and_plot()
    instance.test_soc_limits_summary_csv()
    instance.test_round_trip_summary_csv()
    print(f"\nBESS test outputs written to: {OUTPUT_DIR}")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--generate":
        generate_all_outputs()
    else:
        pytest.main([__file__, "-v", "-s"])

```

# tests/test\_boat.py

---

```
"""
Vessel Model Verification (Boat class).

Objective:
    Verify the correct computation of the hydrodynamic coefficient k and
    the enforcement of constructor validation rules and state management.

Test Case 1 - Propeller Law Coefficient:
     $k = P_{\text{motor}} / v_{\text{range}}^3$ ; with  $P_{\text{motor}}=500$  kW,  $v_{\text{range}}=10$  knots  $\rightarrow k=0.5$ .
    Cubic relation  $P = k v^3$  is checked for multiple speed values.

Test Case 2 - Physical Boundary Conditions:
    Non-physical inputs (non-positive motor power, invalid battery capacity,
    SOC not in  $[0,1]$ ) must raise appropriate exceptions.

Test Case 3 - State Management:
    Only valid enumerated states (IDLE, CHARGING, SAILING) are accepted.

Run with --generate to produce plots and CSV in tests/output/.
"""

import csv
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent))

import pytest
from models.boat import Boat, BoatState

OUTPUT_DIR = Path(__file__).parent / "output" / "boat"

def _valid_boat_kwargs(**overrides):
    base = dict(
        name="TestBoat",
        motor_power=500,
        weight=5000.0,
        length=10.0,
        battery_capacity=1000.0,
        range_speed=10.0,
        soc=1.0,
    )
    base.update(overrides)
    return base

class TestPropellerLawCoefficient:
    """Test Case 1: Hydrodynamic coefficient k and cubic scaling  $P = k v^3$ ."""

    def test_k_computation(self):
        """ $k = P_{\text{motor}} / v_{\text{range}}^3$ ; with 500 kW and 10 knots, k must be 0.5."""
        boat = Boat(**_valid_boat_kwargs())
        expected_k = 0.5
        assert boat.k == pytest.approx(expected_k, rel=1e-9), (
            f"k should be {expected_k}, got {boat.k}"
        )

    def test_cubic_scaling_at_multiple_speeds(self):
        """ $P = k v^3$  for several speeds;  $k=0.5$  from  $P_{\text{motor}}=500$  kW,  $v_{\text{range}}=10$  kn."""
        boat = Boat(**_valid_boat_kwargs())
        k = boat.k
```

```

    assert k == pytest.approx(0.5, rel=1e-9)

    cases = [
        (5.0, 62.5),
        (10.0, 500.0),
        (15.0, 1687.5),
    ]
    for speed, expected_power in cases:
        P = k * (speed ** 3)
        assert P == pytest.approx(expected_power, rel=1e-9), (
            f"At v={speed} kn, P should be {expected_power} kW, got {P}"
        )

class TestPhysicalBoundaryConditions:
    """Test Case 2: Constructor raises on non-physical inputs."""

    def test_non_positive_motor_power_raises(self):
        with pytest.raises(ValueError, match="Motor power must be positive"):
            Boat(**_valid_boat_kwargs(motor_power=0))
        with pytest.raises(ValueError, match="Motor power must be positive"):
            Boat(**_valid_boat_kwargs(motor_power=-100))

    def test_invalid_battery_capacity_raises(self):
        with pytest.raises(ValueError, match="Battery capacity must be positive"):
            Boat(**_valid_boat_kwargs(battery_capacity=0))
        with pytest.raises(ValueError, match="Battery capacity must be positive"):
            Boat(**_valid_boat_kwargs(battery_capacity=-50))

    def test_soc_out_of_range_raises(self):
        with pytest.raises(ValueError, match="SOC must be between 0 and 1"):
            Boat(**_valid_boat_kwargs(soc=-0.1))
        with pytest.raises(ValueError, match="SOC must be between 0 and 1"):
            Boat(**_valid_boat_kwargs(soc=1.1))

class TestStateManagement:
    """Test Case 3: Only valid BoatState values (IDLE, CHARGING, SAILING) accepted."""

    def test_accepts_valid_states(self):
        boat = Boat(**_valid_boat_kwargs())
        boat.state = BoatState.IDLE
        assert boat.state is BoatState.IDLE
        boat.state = BoatState.CHARGING
        assert boat.state is BoatState.CHARGING
        boat.state = BoatState.SAILING
        assert boat.state is BoatState.SAILING

    def test_rejects_invalid_state_type(self):
        boat = Boat(**_valid_boat_kwargs())
        with pytest.raises(ValueError, match="State must be a BoatState enum"):
            boat.state = "idle"
        with pytest.raises(ValueError, match="State must be a BoatState enum"):
            boat.state = 42

# -----
# Output generation: CSV and plots for documentation / thesis
# -----

class TestBoatWithOutput:
    """Generate CSV and plots from boat tests for reporting."""

    @pytest.fixture(autouse=True)
    def setup_output_dir(self):
        OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    def test_propeller_law_csv_and_plot(self):

```



```

"""Propeller law:  $k = P/v^3$ ,  $P = k*v^3$ . CSV + power-vs-speed plot."""
pytest.importorskip("matplotlib")
import numpy as np
import matplotlib.pyplot as plt

boat = Boat(**_valid_boat_kwargs())
k = boat.k
speeds = [0, 2.5, 5.0, 7.5, 10.0, 12.5, 15.0]
rows = []
for v in speeds:
    P = k * (v ** 3)
    rows.append({"speed_knots": v, "power_kw": round(P, 2), "k": k})

csv_path = OUTPUT_DIR / "boat_propeller_law.csv"
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["speed_knots", "power_kw", "k"])
    writer.writeheader()
    writer.writerows(rows)
print(f"\n✓ CSV saved: {csv_path}")

v_curve = np.linspace(0, 16, 200)
P_curve = k * (v_curve ** 3)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(v_curve, P_curve, "b-", linewidth=2, label=r"$P = k \cdot v^3$")
ax.plot(10, 500, "ro", markersize=10, label="Reference (10 kn, 500 kW)")
ax.axhline(500, color="r", linestyle="--", alpha=0.5)
ax.axvline(10, color="r", linestyle="--", alpha=0.5)
ax.set_xlabel("Speed (knots)")
ax.set_ylabel("Power (kW)")
ax.set_title(f"Boat propeller law (k = {k}, $P_{{motor}}$=500 kW, $v_{{range}}$=10 kn)")
ax.legend()
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 16)
ax.set_ylim(0, None)
fig.tight_layout()
plot_path = OUTPUT_DIR / "boat_propeller_law.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"✓ Plot saved: {plot_path}")

assert k == pytest.approx(0.5, rel=1e-9)

def test_validation_summary_csv(self):
    """Summary of physical boundary checks: what was tested and expected."""
    rows = [
        {"check": "motor_power > 0", "invalid_example": "0, -100", "expected": "ValueError"},
        {"check": "battery_capacity > 0", "invalid_example": "0, -50", "expected": "ValueError"},
        {"check": "0 <= SOC <= 1", "invalid_example": "-0.1, 1.1", "expected": "ValueError"},
    ]
    csv_path = OUTPUT_DIR / "boat_validation_summary.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["check", "invalid_example", "expected"])
        writer.writeheader()
        writer.writerows(rows)
    print(f"\n✓ CSV saved: {csv_path}")

def test_states_plot(self):
    """Plot valid boat states (IDLE, CHARGING, SAILING)."""
    pytest.importorskip("matplotlib")
    import matplotlib.pyplot as plt

    boat = Boat(**_valid_boat_kwargs())
    states = [BoatState.IDLE, BoatState.CHARGING, BoatState.SAILING]
    labels = [s.value for s in states]
    x = range(len(states))
    colors = ["#2ecc71", "#3498db", "#e74c3c"]

    fig, ax = plt.subplots(figsize=(6, 4))

```

```

bars = ax.bar(x, [1] * len(states), color=colors, edgecolor="black", linewidth=1.2)
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.set_ylabel("Valid")
ax.set_title("Boat state enum (only these values accepted by state setter)")
ax.set_ylim(0, 1.2)
fig.tight_layout()
plot_path = OUTPUT_DIR / "boat_states.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"✓ Plot saved: {plot_path}")

boat.state = BoatState.SAILING
assert boat.state is BoatState.SAILING

def test_boat_summary_csv(self):
    """Single summary CSV: k, reference speeds/powers, test result."""
    boat = Boat(**_valid_boat_kwargs())
    k = boat.k
    ref_speeds = [5.0, 10.0, 15.0]
    summary = [
        {
            "parameter": "k (P_motor / v_range^3)",
            "value": k,
            "unit": "-",
        },
        {
            "parameter": "P at 5 kn",
            "value": round(k * 5**3, 2),
            "unit": "kW",
        },
        {
            "parameter": "P at 10 kn",
            "value": round(k * 10**3, 2),
            "unit": "kW",
        },
        {
            "parameter": "P at 15 kn",
            "value": round(k * 15**3, 2),
            "unit": "kW",
        },
    ]
    csv_path = OUTPUT_DIR / "boat_summary.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["parameter", "value", "unit"])
        writer.writeheader()
        writer.writerows(summary)
    print(f"\n✓ CSV saved: {csv_path}")

def generate_all_outputs():
    """Generate all boat test outputs (CSV + plots) without running full pytest."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    instance = TestBoatWithOutput()
    instance.setup_output_dir()
    instance.test_propeller_law_csv_and_plot()
    instance.test_validation_summary_csv()
    instance.test_states_plot()
    instance.test_boat_summary_csv()
    print(f"\nBoat test outputs written to: {OUTPUT_DIR}")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--generate":
        generate_all_outputs()
    else:
        pytest.main([__file__, "-v", "-s"])

```

# tests/test\_charger.py

---

```
"""
Charger Model Verification (Charger class).

Objective:
    Verify that the charger enforces its rated power limits, correctly applies
    efficiency losses, and maintains consistent operational states.

Test Case 1 - Power Constraints:
    The charger is initialized with a maximum power rating (P_max). The
    constructor is tested to ensure that invalid configurations raise
    exceptions. During operation, requested power values exceeding P_max are
    externally clamped, and the assigned power is verified to remain within
    allowable limits.

Test Case 2 - Efficiency Handling:
    The effective charging power delivered to the battery is evaluated using
     $P_{\text{effective}} = P \cdot \eta_{\text{evse}}$ 
    for different power levels. The returned value is compared against
    reference calculations to confirm correct loss modeling.

Test Case 3 - State Transitions:
    The state setter is tested to ensure that only valid enumerated states
    (IDLE, CHARGING) are accepted. Additionally, transitions to the IDLE
    state are verified to automatically reset the output power and disconnect
    any associated vessel.

Run with --generate to produce plots and CSV in tests/output/.
"""

import csv
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent))

import pytest

from models.charger import Charger, ChargerState

OUTPUT_DIR = Path(__file__).parent / "output" / "charger"

def _valid_charger_kwargs(**overrides):
    base = dict(
        name="TestCharger",
        max_power=22, # kW
        efficiency=0.95, # 95 %
        power=0.0,
    )
    base.update(overrides)
    return base

class TestPowerConstraints:
    """Test Case 1: Constructor validation and power limits."""

    def test_invalid_max_power_raises(self):
        """Non-positive max_power must raise a ValueError."""
        with pytest.raises(ValueError, match="Max power must be positive"):
            Charger(**_valid_charger_kwargs(max_power=0))
        with pytest.raises(ValueError, match="Max power must be positive"):
            Charger(**_valid_charger_kwargs(max_power=-5))
```

```

def test_invalid_efficiency_raises(self):
    """Efficiency must lie strictly in (0, 1]."""
    with pytest.raises(ValueError, match="Efficiency must be between 0 and 1"):
        Charger(**_valid_charger_kwargs(eficiency=0.0))
    with pytest.raises(ValueError, match="Efficiency must be between 0 and 1"):
        Charger(**_valid_charger_kwargs(eficiency=-0.1))
    with pytest.raises(ValueError, match="Efficiency must be between 0 and 1"):
        Charger(**_valid_charger_kwargs(eficiency=1.1))

def test_invalid_initial_power_raises(self):
    """Initial power must be within [0, P_max]."""
    with pytest.raises(ValueError, match="Power cannot be negative"):
        Charger(**_valid_charger_kwargs(power=-1.0))
    with pytest.raises(ValueError, match="Power cannot exceed max_power"):
        Charger(**_valid_charger_kwargs(power=23.0))

def test_requested_power_is_clamped_externally(self):
    """
    Simulate external clamping: requested power above P_max
    must not be assigned to the charger.
    """
    charger = Charger(**_valid_charger_kwargs())

    cases = [
        (0.0, 0.0),
        (10.0, 10.0),
        (22.0, 22.0),
        (30.0, 22.0),
        (50.0, 22.0),
    ]

    for requested, expected_assigned in cases:
        assigned = min(requested, charger.max_power)
        charger.power = assigned
        assert 0.0 <= charger.power <= charger.max_power
        assert charger.power == pytest.approx(
            expected_assigned, rel=1e-9
        ), f"Requested {requested} kW should result in {expected_assigned} kW assigned"

class TestEfficiencyHandling:
    """Test Case 2: P_effective = P * η_evse for different power levels."""

    def test_effective_power_at_max_rating(self):
        """
        For P = P_max = 22 kW and η = 0.95, P_effective must be 20.9 kW.
        """
        charger = Charger(**_valid_charger_kwargs())
        charger.power = charger.max_power

        expected = charger.max_power * charger.eficiency # 22 * 0.95 = 20.9
        assert charger.efective_power == pytest.approx(
            expected, rel=1e-9
        ), f"Effective power should be {expected} kW, got {charger.efective_power} kW"

    def test_effective_power_for_multiple_power_levels(self):
        """
        Check P_effective = P * η_evse over a range of powers below and at P_max.
        """
        charger = Charger(**_valid_charger_kwargs())

        test_cases = [
            0.0,
            5.0,
            10.0,
            15.0,
            22.0,
        ]

```

```

        for power in test_cases:
            charger.power = power
            expected = power * charger.efficiency
            assert charger.effective_power == pytest.approx(
                expected, rel=1e-9
            ), f"At P={power} kW, effective power should be {expected} kW"

class TestStateTransitions:
    """Test Case 3: Valid states and automatic reset on IDLE."""

    def test_accepts_valid_states(self):
        """Only ChargerState.IDLE and ChargerState.CHARGING are accepted."""
        charger = Charger(**_valid_charger_kwargs())

        # Initial state from dataclass
        assert charger.state is ChargerState.IDLE

        charger.state = ChargerState.CHARGING
        assert charger.state is ChargerState.CHARGING

        charger.state = ChargerState.IDLE
        assert charger.state is ChargerState.IDLE

    def test_rejects_invalid_state_type(self):
        """Setting state to non-enum types must raise a ValueError."""
        charger = Charger(**_valid_charger_kwargs())

        with pytest.raises(ValueError, match="State must be a ChargerState enum"):
            charger.state = "idle" # type: ignore[assignment]

        with pytest.raises(ValueError, match="State must be a ChargerState enum"):
            charger.state = 1 # type: ignore[assignment]

    def test_idle_state_resets_power_and_connected_boat(self):
        """
        Transitioning to IDLE resets power to 0 kW and disconnects any boat.
        """
        charger = Charger(**_valid_charger_kwargs())

        charger.state = ChargerState.CHARGING
        charger.power = 10.0
        charger.connected_boat = "DemoBoat"

        assert charger.state is ChargerState.CHARGING
        assert charger.power == 10.0
        assert charger.connected_boat == "DemoBoat"

        charger.state = ChargerState.IDLE

        assert charger.state is ChargerState.IDLE
        assert charger.power == 0.0
        assert charger.connected_boat is None

# -----
# Output generation: CSV and plots for documentation / thesis
# -----

class TestChargerWithOutput:
    """Generate CSV and plots from charger tests for reporting."""

    @pytest.fixture(autouse=True)
    def setup_output_dir(self):
        OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    def test_power_clamping_csv_and_plot(self):

```

```

"""
Power clamping: requested vs assigned vs effective power.
CSV + plot saved under tests/output/charger.
"""

pytest.importorskip("matplotlib")
import numpy as np
import matplotlib.pyplot as plt

charger = Charger(**_valid_charger_kwargs())

requested_powers = [0.0, 5.0, 10.0, 15.0, 22.0, 30.0, 40.0, 50.0]
rows = []
for p_req in requested_powers:
    p_assigned = min(p_req, charger.max_power)
    charger.power = p_assigned
    p_eff = charger.effective_power
    rows.append(
        {
            "requested_power_kw": p_req,
            "assigned_power_kw": p_assigned,
            "effective_power_kw": round(p_eff, 4),
            "max_power_kw": charger.max_power,
            "efficiency": charger.efficiency,
            "was_clamped": p_req > charger.max_power,
        }
    )

csv_path = OUTPUT_DIR / "charger_power_clamping.csv"
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "requested_power_kw",
            "assigned_power_kw",
            "effective_power_kw",
            "max_power_kw",
            "efficiency",
            "was_clamped",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)
print(f"\n\u2713 CSV saved: {csv_path}")

# Smooth curves for requested, assigned (clamped) and effective power
p_req_curve = np.linspace(0.0, 50.0, 200)
p_assigned_curve = np.minimum(p_req_curve, charger.max_power)
p_eff_curve = p_assigned_curve * charger.efficiency

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(
    p_req_curve,
    p_req_curve,
    "b--",
    linewidth=1.5,
    label="Requested power",
    alpha=0.7,
)
ax.plot(
    p_req_curve,
    p_assigned_curve,
    "g-",
    linewidth=2,
    label=f"Assigned power (clamped, $P_{{max}}$ = {charger.max_power} kW)",
)
ax.plot(
    p_req_curve,
    p_eff_curve,

```

```

        "r-",
        linewidth=2,
        label=r"Effective power  $P_{\mathrm{eff}} = P \cdot \eta_{\mathrm{evse}}$ ",
    )

    # Highlight a representative over-limit request (e.g. 50 kW)
    p_test = 50.0
    p_assigned_test = min(p_test, charger.max_power)
    p_eff_test = p_assigned_test * charger.efficiency
    ax.plot(p_test, p_assigned_test, "ko", markersize=8, label="Example clamped point")
    ax.plot(p_test, p_eff_test, "ro", markersize=6)

    ax.axhline(charger.max_power, color="g", linestyle=":", alpha=0.5)
    ax.axvline(charger.max_power, color="gray", linestyle=":", alpha=0.5)

    ax.set_xlabel("Requested power (kW)")
    ax.set_ylabel("Power (kW)")
    ax.set_title("Charger power clamping and efficiency")
    ax.legend()
    ax.grid(True, alpha=0.3)
    ax.set_xlim(0, 50)
    ax.set_ylim(0, 50)
    fig.tight_layout()

    plot_path = OUTPUT_DIR / "charger_power_clamping.png"
    fig.savefig(plot_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"\u2713 Plot saved: {plot_path}")

    # Sanity check on one of the tabulated rows
    ref = next(r for r in rows if r["requested_power_kw"] == 22.0)
    assert ref["assigned_power_kw"] == pytest.approx(22.0, rel=1e-9)
    assert ref["effective_power_kw"] == pytest.approx(22.0 * charger.efficiency, rel=1e-9)

def test_efficiency_summary_csv(self):
    """Summary of  $P_{\mathrm{effective}} = P \cdot \eta_{\mathrm{evse}}$  for several operating points."""
    charger = Charger(**_valid_charger_kwargs())

    powers = [5.0, 10.0, 15.0, 22.0]
    rows = []
    for p in powers:
        charger.power = p
        rows.append(
            {
                "power_kw": p,
                "efficiency": charger.efficiency,
                "effective_power_kw": round(charger.effective_power, 4),
            }
        )

    csv_path = OUTPUT_DIR / "charger_efficiency_summary.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=["power_kw", "efficiency", "effective_power_kw"],
        )
        writer.writeheader()
        writer.writerows(rows)
    print(f"\n\u2713 CSV saved: {csv_path}")

def test_states_plot(self):
    """Plot valid charger states (IDLE, CHARGING)."""
    pytest.importorskip("matplotlib")
    import matplotlib.pyplot as plt

    states = [ChargerState.IDLE, ChargerState.CHARGING]
    labels = [s.value for s in states]
    x = range(len(states))

```

```

colors = ["#2ecc71", "#3498db"]

fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(x, [1] * len(states), color=colors, edgecolor="black", linewidth=1.2)
ax.set_xticks(list(x))
ax.set_xticklabels(labels)
ax.set_ylabel("Valid")
ax.set_title(
    "Charger state enum (only these values accepted by state setter)"
)
ax.set_ylim(0, 1.2)
fig.tight_layout()

plot_path = OUTPUT_DIR / "charger_states.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"\u2713 Plot saved: {plot_path}")

# Simple behavioural check: switching to CHARGING is accepted.
charger = Charger(**_valid_charger_kwargs())
charger.state = ChargerState.CHARGING
assert charger.state is ChargerState.CHARGING

def test_charger_summary_csv(self):
    """Single summary CSV: P_max, efficiency, and effective power at P_max."""
    charger = Charger(**_valid_charger_kwargs())
    charger.power = charger.max_power

    summary = [
        {
            "parameter": "P_max",
            "value": charger.max_power,
            "unit": "kW",
        },
        {
            "parameter": "efficiency",
            "value": charger.efficiency,
            "unit": "-",
        },
        {
            "parameter": "P_effective at P_max",
            "value": round(charger.effective_power, 4),
            "unit": "kW",
        },
    ]

    csv_path = OUTPUT_DIR / "charger_summary.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["parameter", "value", "unit"])
        writer.writeheader()
        writer.writerows(summary)
    print(f"\n\u2713 CSV saved: {csv_path}")

def generate_all_outputs():
    """Generate all charger test outputs (CSV + plots) without running full pytest."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    instance = TestChargerWithOutput()
    instance.setup_output_dir()
    instance.test_power_clamping_csv_and_plot()
    instance.test_efficiency_summary_csv()
    instance.test_states_plot()
    instance.test_charger_summary_csv()
    print(f"\nCharger test outputs written to: {OUTPUT_DIR}")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--generate":

```



```
    generate_all_outputs()
else:
    pytest.main([__file__, "-v", "-s"])
```

# tests/test\_integration.py

---

```
"""System Integration Validation Scenario.
```

```
Scenario:
```

- One electric vessel connected to a dedicated charger
- Active PV generation with BESS
- Fixed grid connection with contracted power limit

```
Objective:
```

```
Verify correct energy flow consistency, module interaction, and global  
power balance across a complete 24-hour simulation cycle.
```

```
Validation criteria:
```

- Power balance at each timestep:  
     $P_{\text{grid}} + P_{\text{PV}} + P_{\text{BESS}} = P_{\text{load}}$   
    where:  
         $P_{\text{grid}} = P_{\text{import}} - P_{\text{export}}$  (port metrics)  
         $P_{\text{PV}} = \text{power\_active\_production}$  (port metric)  
         $P_{\text{BESS}} = \text{bess\_discharge} - \text{bess\_charge}$  (port metrics)  
         $P_{\text{load}} = \text{power\_active\_consumption}$  (port metric)
- Compliance with component constraints:
  - Grid import  $\leq$  contracted power
  - SOC bounds respected for vessel and BESS
  - BESS power limited by its rated charge/discharge power

```
This script both RUNS the simulation (writing an SQLite DB) and then  
QUERIES the DB to perform the above validations and generate plots/CSVs.
```

```
"""
```

```
from __future__ import annotations
```

```
import sqlite3
```

```
from pathlib import Path
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
from models import Port, Boat, Charger, PV, BESS, BESSControlStrategy
```

```
from config import Settings, SimulationMode
```

```
from database import DatabaseManager
```

```
from simulation import SimulationEngine
```

```
PROJECT_ROOT = Path(__file__).parent.parent
```

```
DB_PATH = PROJECT_ROOT / "system_integration_validation.db"
```

```
OUTPUT_DIR = PROJECT_ROOT / "tests" / "output" / "sim"
```

```
# -----  
# Scenario configuration helpers  
# -----
```

```
def build_port_scenario() -> Port:
```

```
    """Create port + components for the integration scenario."""
```

```
    port = Port(
```

```
        name="Funchal_System_Test",
```

```
        contracted_power=80, # kW grid limit
```

```
        lat=32.64542,
```

```
        lon=-16.90841,
```

```
        tariff_path="assets/tariff/default_tariff.json",
```

```
    )
```

```
    # One vessel
```

```

boat = Boat(
    name="Boat",
    motor_power=120,
    weight=3500,
    length=9.0,
    battery_capacity=120.0,
    range_speed=18.0,
    soc=0.20,
)

# Dedicated charger
charger = Charger(name="Charger", max_power=22.0, efficiency=0.95)

# PV + BESS
pv_system = PV(
    name="PV",
    capacity=20.0, # kW DC
    tilt=30.0,
    azimuth=180.0,
    latitude=port.lat,
    longitude=port.lon,
)

bess = BESS(
    name="BESS",
    capacity=100.0, # kWh
    max_charge_power=30.0, # kW
    max_discharge_power=30.0, # kW
    efficiency=0.90,
    soc_min=0.10,
    soc_max=0.90,
    initial_soc=0.5,
    control_strategy=BESSControlStrategy.DEFAULT,
)

port.add_boat(boat)
port.add_charger(charger)
port.add_pv(pv_system)
port.add_bess(bess)

return port

def run_simulation() -> None:
    """Run a 24h batch simulation and populate the SQLite DB."""
    settings = Settings(
        timestep=900, # 15 minutes
        mode=SimulationMode.BATCH,
        db_path=str(DB_PATH),
        use_optimizer=False,
        power_limit_mode=True,
    )

    db_manager = DatabaseManager(settings.db_path)
    db_manager.initialize_schema()
    db_manager.initialize_default_metrics()

    port = build_port_scenario()
    sim = SimulationEngine(
        port=port,
        settings=settings,
        db_manager=db_manager,
        start_date="2025-06-20", # clear-sky summer day
        days=1,
    )
    sim.run()
# -----
# DB-based validation & plotting

```

```

# -----

def _load_port_timeseries(db_path: Path) -> pd.DataFrame:
    """Load port-level power metrics and derive P_grid, P_PV, P_BESS, P_load."""
    conn = sqlite3.connect(db_path)

    # Port source
    port_src = pd.read_sql_query(
        "SELECT source_id FROM source WHERE source_type = 'port'", conn
    )
    if port_src.empty:
        conn.close()
        raise RuntimeError("No port source found in DB.")
    port_id = int(port_src.iloc[0, 0])

    # Metric mapping
    metrics = pd.read_sql_query(
        "SELECT metric_id, metric_name FROM metric", conn
    )
    metric_map = dict(zip(metrics["metric_name"], metrics["metric_id"]))

    def fetch_metric(name: str) -> pd.DataFrame:
        mid = metric_map.get(name)
        if mid is None:
            return pd.DataFrame(columns=["timestamp", name])
        q = f"""
            SELECT timestamp, CAST(value AS FLOAT) AS {name}
            FROM measurements
            WHERE source_id = {port_id} AND metric_id = {mid}
            ORDER BY timestamp
        """
        return pd.read_sql_query(q, conn)

    df_load = fetch_metric("power_active_consumption")
    df_pv = fetch_metric("power_active_production")
    df_bess_dis = fetch_metric("bess_discharge")
    df_bess_ch = fetch_metric("bess_charge")
    df_imp = fetch_metric("power_active_import")
    df_exp = fetch_metric("power_active_export")

    conn.close()

    # Merge on timestamp
    dfs = [df_load, df_pv, df_bess_dis, df_bess_ch, df_imp, df_exp]
    ts = dfs[0]
    for extra in dfs[1:]:
        ts = ts.merge(extra, on="timestamp", how="outer")

    ts = ts.sort_values("timestamp").reset_index(drop=True)
    ts = ts.fillna(0.0)

    # Derived quantities
    ts["P_load"] = ts["power_active_consumption"]
    ts["P_PV"] = ts["power_active_production"]
    ts["P_BESS"] = ts["bess_discharge"] - ts["bess_charge"]
    ts["P_grid"] = ts["power_active_import"] - ts["power_active_export"]

    return ts

def _load_soc_timeseries(db_path: Path) -> pd.DataFrame:
    """Load SOC and BESS power for constraint checks."""
    conn = sqlite3.connect(db_path)

    metrics = pd.read_sql_query(
        "SELECT metric_id, metric_name FROM metric", conn
    )

```

```

metric_map = dict(zip(metrics["metric_name"], metrics["metric_id"]))
soc_id = metric_map.get("soc")
power_id = metric_map.get("power_active")

boats = pd.read_sql_query(
    "SELECT source_id, source_name FROM source WHERE source_type = 'boat'", conn
)
besses = pd.read_sql_query(
    "SELECT source_id, source_name FROM source WHERE source_type = 'bess'", conn
)

series = []

def fetch_soc_power(source_row, src_type: str):
    sid = int(source_row["source_id"])
    sname = source_row["source_name"]
    q_soc = f"""
        SELECT timestamp, CAST(value AS FLOAT) AS soc_pct
        FROM measurements
        WHERE source_id = {sid} AND metric_id = {soc_id}
        ORDER BY timestamp
    """
    df_soc = pd.read_sql_query(q_soc, conn)
    df_soc["source_name"] = sname
    df_soc["source_type"] = src_type
    if power_id is not None and src_type == "bess":
        q_p = f"""
            SELECT timestamp, CAST(value AS FLOAT) AS power_kw
            FROM measurements
            WHERE source_id = {sid} AND metric_id = {power_id}
            ORDER BY timestamp
        """
        df_p = pd.read_sql_query(q_p, conn)
        df = df_soc.merge(df_p, on="timestamp", how="left")
    else:
        df = df_soc
    series.append(df)

for _, row in boats.iterrows():
    fetch_soc_power(row, "boat")
for _, row in besses.iterrows():
    fetch_soc_power(row, "bess")

conn.close()
if not series:
    return pd.DataFrame()
return pd.concat(series, ignore_index=True)

def validate_and_plot_from_db() -> None:
    """Perform DB-based validations and generate plots/CSVs."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    ts = _load_port_timeseries(DB_PATH)
    soc_df = _load_soc_timeseries(DB_PATH)

    # -----
    # Validation 1: Power balance P_grid + P_PV + P_BEES = P_load
    # -----
    ts["balance_error"] = ts["P_grid"] + ts["P_PV"] + ts["P_BEES"] - ts["P_load"]
    max_abs_err = ts["balance_error"].abs().max()
    print(f"Max absolute power balance error: {max_abs_err:.6e} kW")
    assert max_abs_err < 1e-6 or max_abs_err < 1e-3, "Power balance error too large."

    # -----
    # Validation 2: Grid import within contracted power
    # -----
    conn = sqlite3.connect(DB_PATH)

```

```

cp_df = pd.read_sql_query(
    "SELECT DISTINCT value FROM measurements "
    "JOIN metric USING(metric_id) "
    "JOIN source USING(source_id) "
    "WHERE metric.metric_name = 'contracted_power' "
    "AND source.source_type = 'port'",
    conn,
)
conn.close()
contracted = float(cp_df.iloc[0, 0]) if not cp_df.empty else 80.0
max_import = ts["power_active_import"].max()
print(f"Max grid import: {max_import:.3f} kW (contracted {contracted:.3f} kW)")
assert max_import <= contracted + 1e-6

# -----
# Validation 3: SOC limits & BESS power rating
# -----
if not soc_df.empty:
    bess_soc = soc_df[soc_df["source_type"] == "bess"]
    boat_soc = soc_df[soc_df["source_type"] == "boat"]

    if not bess_soc.empty:
        min_bess_soc = bess_soc["soc_pct"].min()
        max_bess_soc = bess_soc["soc_pct"].max()
        print(
            f"BESS SOC range: {min_bess_soc:.2f}% - {max_bess_soc:.2f}% "
            "(expected within 10-90%)"
        )
        assert 9.9 <= min_bess_soc <= 90.1
        assert 9.9 <= max_bess_soc <= 90.1

        if "power_kw" in bess_soc.columns:
            max_bess_p = bess_soc["power_kw"].abs().max()
            print(f"Max |BESS power|: {max_bess_p:.3f} kW (limit 30 kW)")
            assert max_bess_p <= 30.0 + 1e-6

    if not boat_soc.empty:
        min_boat_soc = boat_soc["soc_pct"].min()
        max_boat_soc = boat_soc["soc_pct"].max()
        print(f"Boat SOC range: {min_boat_soc:.2f}% - {max_boat_soc:.2f}%")
        assert 0.0 <= min_boat_soc <= 100.0
        assert 0.0 <= max_boat_soc <= 100.0

# -----
# CSV export
# -----
csv_path = OUTPUT_DIR / "system_profile_db.csv"
ts.to_csv(csv_path, index=False)
print(f"✓ DB-based system profile CSV saved to: {csv_path}")

# -----
# Plots (power flows + SOC)
# -----
times = pd.to_datetime(ts["timestamp"])
hours = times.dt.hour + times.dt.minute / 60.0

fig, (ax_p, ax_soc) = plt.subplots(2, 1, figsize=(10, 7), sharex=True)

ax_p.plot(hours, ts["P_load"], "k-", linewidth=2, label="Load (P_load)")
ax_p.plot(hours, ts["P_PV"], "gold", linewidth=2, label="P_PV")
ax_p.plot(hours, ts["P_BEES"], "purple", linewidth=1.5, label="P_BEES (discharge-charge)")
ax_p.plot(hours, ts["P_grid"], "r--", linewidth=1.5, label="P_grid (import-export)")
ax_p.set_ylabel("Power (kW)")
ax_p.set_title("System power balance from DB")
ax_p.grid(True, alpha=0.3)
ax_p.legend(loc="upper right", fontsize=9)

if not soc_df.empty:

```

```

soc_df_local = soc_df.copy()
soc_df_local["hour"] = pd.to_datetime(soc_df_local["timestamp"]).dt.hour + (
    pd.to_datetime(soc_df_local["timestamp"]).dt.minute / 60.0
)
for sname, grp in soc_df_local.groupby("source_name"):
    style = "-" if grp["source_type"].iloc[0] == "boat" else "--"
    ax_soc.plot(
        grp["hour"],
        grp["soc_pct"],
        style,
        linewidth=2,
        label=f"{sname} SOC",
    )

ax_soc.set_xlabel("Hour of day")
ax_soc.set_ylabel("SOC (%)")
ax_soc.set_ylim(0, 100)
ax_soc.grid(True, alpha=0.3)
ax_soc.legend(loc="upper right", fontsize=9)

fig.tight_layout()
plot_path = OUTPUT_DIR / "system_profile_db.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"✓ DB-based system profile plot saved to: {plot_path}")

def main():
    print("=" * 80)
    print("SYSTEM INTEGRATION VALIDATION - DB-BASED CHECK")
    print("=" * 80)

    run_simulation()
    print(f"\nSimulation completed. Results stored in: {DB_PATH}")

    validate_and_plot_from_db()

    print("\n" + "=" * 80)
    print("All DB-based validations completed successfully.")
    print("=" * 80)

if __name__ == "__main__":
    main()

```

# tests/test\_pv.py

---

```
"""
PV Model Verification (PV class).

Objective:
    Verify correct solar elevation computation, zero production during night
    hours, realistic diurnal production behavior, and enforcement of rated
    capacity limits.

Test Case 1 - Night Condition:
    For timestamps where the apparent solar elevation satisfies  $\alpha \leq 0$ , the
    model must return zero power output. Multiple night-time hours are
    evaluated to confirm consistent behaviour.

Test Case 2 - Diurnal Production Profile:
    A clear-sky day is simulated using representative irradiance inputs. The
    resulting production profile is evaluated to confirm a bell-shaped curve
    during daylight hours, with peak generation occurring near solar noon.

Test Case 3 - Capacity Constraint:
    The instantaneous production is verified to never exceed the rated DC
    capacity of the system.

Run with --generate to produce plots and CSV in tests/output/.
"""

import csv
import sys
from dataclasses import dataclass
from datetime import datetime, timedelta
from pathlib import Path
from typing import List, Tuple

sys.path.insert(0, str(Path(__file__).parent.parent))

import pytest

from models.pv import PV

OUTPUT_DIR = Path(__file__).parent / "output" / "pv"

# Port of Funchal, Madeira coordinates
FUNCHAL_LAT = 32.6514 # degrees North
FUNCHAL_LON = -16.9084 # degrees West

# PV system parameters
PV_CAPACITY = 22.0 # kW peak
PV_TILT = 9.0 # degrees
PV_AZIMUTH = 180.0 # South-facing

@dataclass
class ClearSkyPoint:
    timestamp: datetime
    ghi: float
    dni: float
    dhi: float
    solar_elevation: float

def _make_pv(**overrides) -> PV:
    base = {
```



```

        "name": "TestPV",
        "capacity": PV_CAPACITY,
        "tilt": PV_TILT,
        "azimuth": PV_AZIMUTH,
        "latitude": FUNCHAL_LAT,
        "longitude": FUNCHAL_LON,
    }
    base.update(overrides)
    return PV(**base)

def _sample_day(
    date: datetime, step_minutes: int = 15
) -> List[Tuple[datetime, float]]:
    """Helper: sample a full day at regular time steps."""
    times = []
    start = date.replace(hour=0, minute=0, second=0, microsecond=0)
    end = start + timedelta(days=1)
    t = start
    while t < end:
        times.append(t)
        t += timedelta(minutes=step_minutes)
    return times

class TestNightCondition:
    """Test Case 1: Zero production when apparent solar elevation  $\alpha \leq 0$ ."""

    def test_zero_production_for_night_hours(self):
        """
        Several night-time hours are checked; production must be exactly 0 kW.
        """
        pv = _make_pv()

        # Representative date for clear conditions (summer solstice)
        date = datetime(2025, 6, 21)
        night_hours = [0, 1, 2, 3, 4, 5, 22, 23]

        for hour in night_hours:
            ts = date.replace(hour=hour, minute=0)
            # No need for irradiance, model will detect  $\alpha \leq 0$  internally
            production = pv.calculate_production(
                ghi=0.0,
                dni=0.0,
                dhi=0.0,
                temperature=20.0,
                timestamp=ts,
            )
            # Use an absolute tolerance to avoid strict float equality.
            assert production == pytest.approx(
                0.0, abs=1e-12
            ), f"Production should be 0 at {hour}:00, got {production} kW"

    def test_negative_elevation_implies_zero_production(self):
        """
        At midnight the solar elevation is strictly negative;
        production must be zero.
        """
        pv = _make_pv()
        ts = datetime(2025, 6, 21, 0, 0)

        # We indirectly test elevation through the model's internal check.
        production = pv.calculate_production(
            ghi=0.0, dni=0.0, dhi=0.0, temperature=15.0, timestamp=ts
        )
        assert production == pytest.approx(0.0, abs=1e-12)

class TestDiurnalProductionProfile:
    """Test Case 2: Bell-shaped daily production profile under clear sky."""

```

```

def test_clear_sky_bell_shaped_profile(self):
    """
    Simulate a clear-sky day; production must rise from 0, peak near noon,
    and go back to 0, with peak occurring around solar noon.
    """
    pv = _make_pv()
    date = datetime(2025, 6, 21)

    # We drive the model with a generic clear-sky GHI envelope that
    # increases and then decreases around midday; DNI/DHI chosen
    # for a simple clear-sky-like split.
    times = _sample_day(date, step_minutes=30)
    productions = []
    for ts in times:
        # Simple smooth envelope: sin over daylight window [6h,18h]
        hour = ts.hour + ts.minute / 60
        if 6 <= hour <= 18:
            # Map [6,18] → [0, π]
            x = (hour - 6) / 12.0 * 3.141592653589793
            ghi = max(0.0, 900.0 * abs(__import__("math").sin(x)))
            dni = ghi * 0.8
            dhi = ghi * 0.2
        else:
            ghi = dni = dhi = 0.0

        p = pv.calculate_production(
            ghi=ghi, dni=dni, dhi=dhi, temperature=25.0, timestamp=ts
        )
        productions.append((hour, p))

    # Extract only positive production hours (daylight)
    daylight = [(h, p) for h, p in productions if p > 0]
    assert daylight, "There should be at least some daylight production."

    # Peak near noon: within ±2 hours of 12:00
    peak_hour, peak_value = max(daylight, key=lambda x: x[1])
    assert 10 <= peak_hour <= 14, f"Peak should occur near noon, got hour {peak_hour}"
    assert peak_value > PV_CAPACITY * 0.3, "Peak should be a significant fraction of capacity."

    # Monotonic rise before peak (coarsely) and fall after peak
    before = [p for h, p in daylight if h < peak_hour]
    after = [p for h, p in daylight if h > peak_hour]
    if len(before) >= 3:
        assert before[0] <= before[len(before) // 2] <= before[-1]
    if len(after) >= 3:
        assert after[0] >= after[len(after) // 2] >= after[-1]

class TestCapacityConstraint:
    """Test Case 3: Instantaneous production never exceeds rated capacity."""

    def test_production_never_exceeds_capacity(self):
        """
        + For a full clear-sky day, validate production  $P(t) \leq \text{capacity}$  for
          every sample.
        """
        pv = _make_pv()
        date = datetime(2025, 6, 21)

        times = _sample_day(date, step_minutes=15)
        for ts in times:
            # Use a high, potentially exaggerated irradiance to stress the model.
            hour = ts.hour + ts.minute / 60
            if 6 <= hour <= 18:
                ghi = 1000.0
                dni = 800.0
                dhi = 200.0
            else:

```

```

        ghi = dni = dhi = 0.0

    p = pv.calculate_production(
        ghi=ghi, dni=dni, dhi=dhi, temperature=25.0, timestamp=ts
    )
    assert p <= PV_CAPACITY + 1e-6, f"Production {p} kW exceeds capacity {PV_CAPACITY} kW"

# -----
# Output generation: CSV and plots for documentation / thesis
# -----

class TestPVWithOutput:
    """Generate CSV and plots from PV tests for reporting."""

    @pytest.fixture(autouse=True)
    def setup_output_dir(self):
        OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    def test_daily_profile_csv_and_plot(self):
        """
        Daily PV profile on a clear-sky summer solstice day.
        CSV + production plot saved under tests/output/pv.
        """
        pytest.importorskip("matplotlib")
        import numpy as np
        import matplotlib.pyplot as plt

        pv = _make_pv(name="Funchal_PV")
        date = datetime(2025, 6, 21)

        rows = []
        times = []
        productions = []

        for ts in _sample_day(date, step_minutes=15):
            hour = ts.hour + ts.minute / 60
            if 6 <= hour <= 18:
                # Smooth bell-shaped irradiance envelope
                x = (hour - 6) / 12.0 * 3.141592653589793
                ghi = max(0.0, 900.0 * abs(__import__("math").sin(x)))
                dni = ghi * 0.8
                dhi = ghi * 0.2
            else:
                ghi = dni = dhi = 0.0

            p = pv.calculate_production(
                ghi=ghi, dni=dni, dhi=dhi, temperature=25.0, timestamp=ts
            )

            rows.append(
                {
                    "timestamp": ts.isoformat(),
                    "hour_decimal": round(hour, 2),
                    "ghi_wm2": round(ghi, 1),
                    "dni_wm2": round(dni, 1),
                    "dhi_wm2": round(dhi, 1),
                    "production_kw": round(p, 4),
                }
            )
            times.append(hour)
            productions.append(p)

        csv_path = OUTPUT_DIR / "pv_daily_profile.csv"
        with open(csv_path, "w", newline="") as f:
            writer = csv.DictWriter(
                f,

```

```

        fieldnames=[
            "timestamp",
            "hour_decimal",
            "ghi_wm2",
            "dni_wm2",
            "dhi_wm2",
            "production_kw",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)
print(f"\n\u2713 CSV saved: {csv_path}")

# Plot production curve
fig, ax = plt.subplots(figsize=(9, 5))
ax.fill_between(times, productions, step="mid", alpha=0.3, color="orange")
ax.plot(times, productions, "orange", linewidth=2, label="PV production")
ax.axhline(
    PV_CAPACITY,
    color="red",
    linestyle="--",
    linewidth=1.5,
    label=f"Rated capacity ({PV_CAPACITY} kW)",
)

# Mark peak
peak_idx = int(np.argmax(productions))
ax.plot(times[peak_idx], productions[peak_idx], "ro", markersize=8)
annotation_text = (
    f"Peak: {productions[peak_idx]:.1f} kW\n"
    f"at {times[peak_idx]:.1f} h"
)
ax.annotate(
    annotation_text,
    xy=(times[peak_idx], productions[peak_idx]),
    xytext=(times[peak_idx] + 1.5, productions[peak_idx]),
    fontsize=9,
    arrowprops={"arrowstyle": "->", "color": "black"},
)

ax.set_xlabel("Hour of day (UTC)")
ax.set_ylabel("Power (kW)")
ax.set_title(
    "PV daily production profile\n"
    f"Port of Funchal ({FUNCHAL_LAT:.2f}\u00b0N, {abs(FUNCHAL_LON):.2f}\u00b0W)"
)
ax.set_xlim(0, 24)
ax.set_ylim(0, PV_CAPACITY * 1.1)
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()

plot_path = OUTPUT_DIR / "pv_daily_profile.png"
fig.savefig(plot_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"\u2713 Plot saved: {plot_path}")

# Expectations: no night production, peak within daylight
for t, p in zip(times, productions):
    if t < 6 or t > 18:
        assert p == pytest.approx(0.0, abs=1e-9)
assert 10 <= times[peak_idx] <= 14

def test_night_condition_summary_csv(self):
    """Summary CSV explicitly listing several night-time samples."""
    pv = _make_pv()
    date = datetime(2025, 6, 21)
    night_hours = [0, 2, 4, 22]

```

```

rows = []
for h in night_hours:
    ts = date.replace(hour=h, minute=0)
    p = pv.calculate_production(
        ghi=0.0, dni=0.0, dhi=0.0, temperature=18.0, timestamp=ts
    )
    rows.append(
        {
            "hour": f"{h:02d}:00",
            "timestamp": ts.isoformat(),
            "expected_alpha_condition": "alpha <= 0",
            "production_kw": round(p, 4),
        }
    )

csv_path = OUTPUT_DIR / "pv_night_condition.csv"
with open(csv_path, "w", newline="") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=[
            "hour",
            "timestamp",
            "expected_alpha_condition",
            "production_kw",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)
print(f"\n\u2713 CSV saved: {csv_path}")

# All listed samples must be zero production
# Allow tiny numerical noise, but enforce values are effectively zero.
for r in rows:
    assert r["production_kw"] == pytest.approx(0.0, abs=1e-9)

def test_capacity_constraint_summary_csv(self):
    """Single summary CSV for capacity constraint checks."""
    pv = _make_pv()
    date = datetime(2025, 6, 21)

    rows = []
    for ts in _sample_day(date, step_minutes=60):
        hour = ts.hour
        if 6 <= hour <= 18:
            ghi = 1000.0
            dni = 800.0
            dhi = 200.0
        else:
            ghi = dni = dhi = 0.0

        p = pv.calculate_production(
            ghi=ghi, dni=dni, dhi=dhi, temperature=25.0, timestamp=ts
        )
        rows.append(
            {
                "hour": f"{hour:02d}:00",
                "ghi_wm2": ghi,
                "dni_wm2": dni,
                "dhi_wm2": dhi,
                "production_kw": round(p, 4),
                "capacity_kw": PV_CAPACITY,
                "within_capacity": p <= PV_CAPACITY + 1e-6,
            }
        )

    csv_path = OUTPUT_DIR / "pv_capacity_constraint.csv"
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(

```

```

        f,
        fieldnames=[
            "hour",
            "ghi_wm2",
            "dni_wm2",
            "dhi_wm2",
            "production_kw",
            "capacity_kw",
            "within_capacity",
        ],
    )
    writer.writeheader()
    writer.writerows(rows)
    print(f"\n\u2713 CSV saved: {csv_path}")

    assert all(r["within_capacity"] for r in rows)

def generate_all_outputs():
    """Generate all PV test outputs (CSV + plots) without running full pytest."""
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    instance = TestPVWithOutput()
    instance.setup_output_dir()
    instance.test_daily_profile_csv_and_plot()
    instance.test_night_condition_summary_csv()
    instance.test_capacity_constraint_summary_csv()
    print(f"\nPV test outputs written to: {OUTPUT_DIR}")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--generate":
        generate_all_outputs()
    else:
        pytest.main([__file__, "-v", "-s"])

```

## weather/\_\_init\_\_.py

---

```
"""Weather data management."""  
  
from .openmeteo import OpenMeteoClient  
  
__all__ = ["OpenMeteoClient"]
```

# weather/openmeteo.py

---

```
"""Weather data fetcher using Open-Meteo API."""

import requests
from datetime import datetime, timedelta
from typing import Dict, List, Optional

class OpenMeteoClient:
    """Client for fetching weather data from Open-Meteo API."""

    FORECAST_URL = "https://api.open-meteo.com/v1/forecast"
    HISTORICAL_URL = "https://archive-api.open-meteo.com/v1/archive"

    def __init__(self, latitude: float, longitude: float):
        """
        Initialize Open-Meteo client.

        Args:
            latitude: Location latitude
            longitude: Location longitude
        """
        self.latitude = latitude
        self.longitude = longitude

    def fetch_forecast(
        self, start_date: datetime, days: int = 7
    ) -> Optional[Dict[str, List]]:
        """
        Fetch weather data from Open-Meteo (forecast or historical based on date).

        Args:
            start_date: Start date for weather data
            days: Number of days to fetch (max 7 for free tier forecast)

        Returns:
            Dictionary with weather data, or None on error
        """
        # Calculate end date
        end_date = start_date + timedelta(days=days)

        # Format dates as strings
        start_str = start_date.strftime("%Y-%m-%d")
        end_str = end_date.strftime("%Y-%m-%d")

        # Determine if we need historical or forecast API
        today = datetime.now().replace(hour=0, minute=0, second=0, microsecond=0)
        is_historical = start_date.replace(tzinfo=None) < today

        if is_historical:
            return self._fetch_historical(start_str, end_str)
        else:
            return self._fetch_forecast(start_str, end_str)

    def _fetch_forecast(self, start_str: str, end_str: str) -> Optional[Dict[str, List]]:
        """Fetch from forecast API (for current/future dates)."""
        params = {
            "latitude": self.latitude,
            "longitude": self.longitude,
            "start_date": start_str,
            "end_date": end_str,
            "hourly": [
                "temperature_2m",
            ]
        }
```



```

        "relative_humidity_2m",
        "dew_point_2m",
        "precipitation",
        "weather_code",
        "cloud_cover",
        "wind_speed_10m",
        "wind_direction_10m",
        "shortwave_radiation", # GHI equivalent
        "direct_radiation", # DNI equivalent
        "diffuse_radiation", # DHI equivalent
        "direct_normal_irradiance", # Actual DNI
    ],
    "timezone": "UTC",
}

try:
    response = requests.get(self.FORECAST_URL, params=params, timeout=30)
    response.raise_for_status()
    data = response.json()
    return self._parse_response(data)

except requests.exceptions.RequestException as e:
    print(f"Error fetching forecast data: {e}")
    return None

def _fetch_historical(self, start_str: str, end_str: str) -> Optional[Dict[str, List]]:
    """Fetch from historical/archive API (for past dates)."""
    params = {
        "latitude": self.latitude,
        "longitude": self.longitude,
        "start_date": start_str,
        "end_date": end_str,
        "hourly": [
            "temperature_2m",
            "relative_humidity_2m",
            "dew_point_2m",
            "precipitation",
            "weather_code",
            "cloud_cover",
            "wind_speed_10m",
            "wind_direction_10m",
            "shortwave_radiation", # GHI equivalent
            "direct_radiation", # DNI equivalent
            "diffuse_radiation", # DHI equivalent
            "direct_normal_irradiance", # Actual DNI
        ],
        "timezone": "UTC",
    }

    try:
        print(f" (Using historical weather API for past date)")
        response = requests.get(self.HISTORICAL_URL, params=params, timeout=30)
        response.raise_for_status()
        data = response.json()
        return self._parse_response(data)

    except requests.exceptions.RequestException as e:
        print(f"Error fetching historical data: {e}")
        return None

def _parse_response(self, data: dict) -> Dict[str, List]:
    """
    Parse Open-Meteo API response.

    Args:
        data: Raw API response

    Returns:

```

```

        Parsed weather data with timestamps and values
    """
    if "hourly" not in data:
        return {}

    hourly = data["hourly"]
    timestamps = hourly.get("time", [])

    # Parse timestamps
    parsed_timestamps = [
        datetime.strptime(ts, "%Y-%m-%dT%H:%M") for ts in timestamps
    ]

    # Extract all available metrics
    parsed_data = {"timestamps": parsed_timestamps}

    # Map API fields to our metric names
    metric_mapping = {
        "temperature_2m": "temperature",
        "relative_humidity_2m": "humidity",
        "dew_point_2m": "dew_point",
        "precipitation": "precipitation",
        "weather_code": "weather_code",
        "cloud_cover": "cloud_cover",
        "wind_speed_10m": "wind_speed",
        "wind_direction_10m": "wind_direction",
        "shortwave_radiation": "ghi", # Global Horizontal Irradiance
        "direct_radiation": "direct_radiation",
        "diffuse_radiation": "dhi", # Diffuse Horizontal Irradiance
        "direct_normal_irradiance": "dni", # Direct Normal Irradiance
    }

    for api_field, metric_name in metric_mapping.items():
        if api_field in hourly:
            parsed_data[metric_name] = hourly[api_field]

    return parsed_data

def get_current_conditions(self, current_time: datetime) -> Optional[Dict]:
    """
    Get weather conditions for a specific time from forecast data.

    Args:
        current_time: Time to get conditions for

    Returns:
        Dictionary with weather conditions, or None if not available
    """
    # Fetch forecast if not cached
    forecast = self.fetch_forecast(current_time, days=1)

    if not forecast or "timestamps" not in forecast:
        return None

    # Find closest timestamp
    timestamps = forecast["timestamps"]
    closest_idx = min(
        range(len(timestamps)),
        key=lambda i: abs((timestamps[i] - current_time).total_seconds()),
    )

    # Extract conditions for that timestamp
    conditions = {"timestamp": timestamps[closest_idx]}

    for key, values in forecast.items():
        if key != "timestamps" and len(values) > closest_idx:
            conditions[key] = values[closest_idx]

    return conditions

```

```
def __repr__(self) -> str:  
    return f"OpenMeteoClient(lat={self.latitude}, lon={self.longitude})"
```